

The Amazon Book

Ejercicios LeetCode para entrevistas

Aldo Zetina

7 de julio de 2026

Índice general

Introducción	3
1 Grafos	4
200. Number of Islands (M)	5
994. Rotting Oranges (M)	8
695. Max Area of Island (M)	11
542. 01 Matrix (M)	14
1091. Shortest Path in Binary Matrix (M)	17
417. Pacific Atlantic Water Flow (M)	20
79. Word Search (M)	25
207. Course Schedule (M)	28
210. Course Schedule II (M)	31
269. Alien Dictionary (H)	35
127. Word Ladder (H)	40
126. Word Ladder II (H)	43
261. Graph Valid Tree (M)	48
323. Number of Connected Components (M)	52
128. Longest Consecutive Sequence (M)	56
547. Number of Provinces (M)	59
743. Network Delay Time (M)	63
787. Cheapest Flights Within K Stops (M)	67
133. Clone Graph (M)	71
1192. Critical Connections in a Network (H)	76
2 Árboles	81
236. Lowest Common Ancestor of a Binary Tree (M)	82
1644. Lowest Common Ancestor of a Binary Tree II (M)	86
103. Binary Tree Zigzag Level Order Traversal (M)	90
199. Binary Tree Right Side View (M)	93
987. Vertical Order Traversal of a Binary Tree (H)	97
124. Binary Tree Maximum Path Sum (H)	101
543. Diameter of Binary Tree (Fácil)	106
863. All Nodes Distance K in Binary Tree (Media)	110
98. Validate Binary Search Tree (M)	114
105. Construct Binary Tree from Preorder and Inorder Traversal (M)	118
337. House Robber III (M)	122
297. Serialize and Deserialize Binary Tree (D)	126

Introducción

Este libro reúne ejercicios de LeetCode frecuentes en entrevistas de Amazon, resueltos en C++ y explicados para todo público. Cada encabezado de problema es un enlace: al hacer clic se abre el archivo `.cpp` correspondiente en Visual Studio Code, donde el código es compilable y trae casos de prueba.

Dónde está el código. Todo el código fuente vive en la carpeta `src/` del repositorio, organizada por bloque (`src/01-grafos/`, `src/02-arboles/`, ...). El libro lo importa directamente desde ahí (no se duplica). Bajo el título de cada problema se muestra la ruta exacta del archivo, por si quieres abrirlo o consultarlo directamente en `src/`.

Animaciones interactivas. Además del código, cada problema incluye el enlace ▷ *Ver animación* con dos accesos: **local** (abre el visualizador del repo descargado, funciona sin internet) y **web** (abre la versión publicada en GitHub Pages desde cualquier equipo). Muestra una visualización del algoritmo paso a paso: la cuadrícula se pinta, la cola del BFS avanza, el árbol se recorre. El visualizador vive en la carpeta `rep-visual/`, funciona sin instalar nada (basta un navegador) y permite alternar el idioma entre español e inglés. Los enlaces piden abrirse en una ventana nueva para no reemplazar la vista del PDF.

Cómo compilar. Ejecuta `build.ps1` en PowerShell. El script detecta la ruta del repositorio e inyecta las rutas de los enlaces, luego compila con `latexmk -xelatex`.

Capítulo 1

Grafos

200. Number of Islands

(M)

Código fuente: `src/01-grafos/200-number-of-islands.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan una cuadrícula de caracteres donde '1' es tierra y '0' es agua. Una *isla* es un grupo de tierras conectadas en horizontal o vertical. Queremos contar cuántas islas hay.

Intuición. Imagina que sobrevuelas el mapa. Cada vez que ves un pedazo de tierra que aún no has pisado, plantas una bandera (una isla nueva) y luego caminas por toda la tierra conectada hundiéndola en el mapa, para no volver a contarla. Repites hasta que no quede tierra nueva.

```

1  1  0  0  0
1  1  0  0  0
0  0  1  0  0
0  0  0  1  1

```

En esta cuadrícula hay **3** islas: el bloque 2×2 de arriba a la izquierda, la celda central, y el par de abajo a la derecha.

Solución. Recorremos la cuadrícula; al encontrar tierra nueva sumamos una isla y usamos DFS (hundir) para marcar como agua toda su tierra conectada. El código completo (abre el archivo con clic en el título):

```

1  //
   =====
2  // 200. Number of Islands   (Medium)
3  //
   -----
4  // Dada una cuadrícula de '1' (tierra) y '0' (agua), cuenta cuántas islas hay.
5  // Una isla se forma conectando tierras adyacentes en horizontal o vertical
6  // (no en diagonal). Los bordes de la cuadrícula son todos agua.
7  //
8  // Ejemplo:
9  //   grid =
10 //       1 1 0 0 0           Aquí hay 3 islas:
11 //       1 1 0 0 0           - bloque 2x2 arriba-izquierda
12 //       0 0 1 0 0           - celda central
13 //       0 0 0 1 1           - bloque de 2 abajo-derecha
14 //
15 // Diagrama de la exploración DFS desde una celda de tierra:
16 //
17 //       (r-1,c)
18 //       |
19 //   (r,c-1)-(r,c)-(r,c+1)   Visitamos las 4 vecinas; cada tierra
20 //       |                   visitada la marcamos como '0' (hundida)
21 //       (r+1,c)             para no contarla otra vez.
22 //
23 // Idea: recorreremos la cuadrícula. Cuando encontramos un '1' no visitado,
24 // sumamos una isla y "hundimos" (marcamos como agua) toda su tierra conectada
25 // con DFS. Así cada isla se cuenta exactamente una vez.

```

```

26 //
27 // Complejidad: tiempo  $O(m*n)$  (cada celda se visita una vez),
28 //                espacio  $O(m*n)$  peor caso por la pila de recursión.
29 //
=====

30 #include <vector>
31 #include <cassert>
32 #include <iostream>
33 using namespace std;
34
35 class Solution {
36 public:
37     int numIslands(vector<vector<char>>& grid) {
38         if (grid.empty() || grid[0].empty()) return 0;
39         int m = grid.size(), n = grid[0].size();
40         int islas = 0;
41         for (int r = 0; r < m; ++r) {
42             for (int c = 0; c < n; ++c) {
43                 if (grid[r][c] == '1') { // tierra sin visitar => nueva isla
44                     ++islas;
45                     hundir(grid, r, c, m, n);
46                 }
47             }
48         }
49         return islas;
50     }
51
52 private:
53     // Marca como agua toda la tierra conectada a (r,c).
54     void hundir(vector<vector<char>>& grid, int r, int c, int m, int n) {
55         if (r < 0 || r >= m || c < 0 || c >= n) return; // fuera de límites
56         if (grid[r][c] != '1') return; // agua o ya
57                                     // visitada
58         grid[r][c] = '0'; // hundir esta celda
59         hundir(grid, r - 1, c, m, n); // arriba
60         hundir(grid, r + 1, c, m, n); // abajo
61         hundir(grid, r, c - 1, m, n); // izquierda
62         hundir(grid, r, c + 1, m, n); // derecha
63     }
64 };
65
66 int main() {
67     Solution s;
68
69     vector<vector<char>> g1 = {
70         {'1', '1', '0', '0', '0'},
71         {'1', '1', '0', '0', '0'},
72         {'0', '0', '1', '0', '0'},
73         {'0', '0', '0', '1', '1'},
74     };
75     assert(s.numIslands(g1) == 3);
76
77     vector<vector<char>> g2 = {

```

```
77         {'1','1','1','1','0'},
78         {'1','1','0','1','0'},
79         {'1','1','0','0','0'},
80         {'0','0','0','0','0'},
81     };
82     assert(s.numIslands(g2) == 1);
83
84     vector<vector<char>> g3 = {{'0','0'},{'0','0'}};
85     assert(s.numIslands(g3) == 0);
86
87     vector<vector<char>> vacio = {};
88     assert(s.numIslands(vacio) == 0);
89
90     cout << "200 Number of Islands: todas las pruebas pasaron.\n";
91     return 0;
92 }
```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

Notas de entrevista (Amazon). Es el clásico de Amazon. Menciona que modificar la cuadrícula in situ evita el arreglo `visited`; si no puedes mutar la entrada, usa un conjunto de visitados o BFS con cola. Casos borde: cuadrícula vacía, toda agua, toda tierra (una isla).

994. Rotting Oranges

(M)

Código fuente: [src/01-grafos/994-rotting-oranges.cpp](#)▷ Ver animación · [local](#) | [web](#)

Tenemos una cuadrícula donde 0 es una celda vacía, 1 es una naranja fresca y 2 una naranja podrida. Cada minuto, toda naranja fresca adyacente (arriba, abajo, izquierda, derecha) a una podrida se pudre. Queremos el número de minutos hasta que no quede ninguna fresca, o -1 si alguna nunca se pudre.

Intuición. La podredumbre se expande como una onda desde todas las naranjas podridas *al mismo tiempo*. Eso es exactamente un BFS *multi-fuente*: metemos todas las podridas en la cola y expandimos por capas; cada capa es un minuto. Contamos las frescas al inicio y las restamos conforme se pudren; si al final queda alguna, es imposible.

	2	1	1		2	2	2	
minuto 0	1	1	0	→	minuto 4	2	2	0
	0	1	1			0	2	2

Solución. BFS por niveles desde todas las podridas. La clave del manejo de casos borde: si no hay frescas al inicio, la respuesta es 0; si terminan las capas y aún quedan frescas, es -1.

```

1 //
  =====

2 // 994. Rotting Oranges (Medium)
3 //
  -----

4 // Cuadrícula con valores: 0 = vacío, 1 = naranja fresca, 2 = naranja podrida.
5 // Cada minuto, toda naranja fresca adyacente (4 direcciones) a una podrida
6 // se pudre. Devuelve los minutos hasta que ninguna naranja quede fresca, o
7 // -1 si es imposible (queda alguna fresca aislada).
8 //
9 // Ejemplo:
10 //      minuto 0          minuto 1          minuto 2
11 //      2 1 1          2 2 1          2 2 2
12 //      1 1 0  =>      2 1 0  =>      2 2 0
13 //      0 1 1          0 1 1          0 2 1  ... hasta minuto 4
14 //
15 // Diagrama del BFS multi-fuente (todas las podridas empiezan en la cola):
16 //
17 //      cola inicial: [todas las (r,c) con valor 2]
18 //      por cada nivel (minuto): sacamos la capa actual y pudrimos vecinas
19 //      frescas, que entran al siguiente nivel. El número de niveles-1 = minutos.
20 //
21 // Idea: BFS por niveles desde TODAS las podridas a la vez (multi-fuente).
22 // Contamos cuántas frescas hay; cada vez que pudrimos una, la restamos.
23 // Si al final quedan frescas => -1. Si no había frescas al inicio => 0.
24 //
25 // Complejidad: tiempo O(m*n), espacio O(m*n) por la cola.
26 //
  =====

```

```

27 #include <vector>
28 #include <queue>
29 #include <cassert>
30 #include <iostream>
31 using namespace std;
32
33 class Solution {
34 public:
35     int orangesRotting(vector<vector<int>>& grid) {
36         int m = grid.size(), n = grid[0].size();
37         queue<pair<int,int>> cola;    // naranjas podridas (fuentes del BFS)
38         int frescas = 0;
39         for (int r = 0; r < m; ++r)
40             for (int c = 0; c < n; ++c) {
41                 if (grid[r][c] == 2) cola.push({r, c});
42                 else if (grid[r][c] == 1) ++frescas;
43             }
44
45         if (frescas == 0) return 0;    // nada que pudrir
46
47         int minutos = 0;
48         int dr[] = {-1, 1, 0, 0};
49         int dc[] = {0, 0, -1, 1};
50         while (!cola.empty() && frescas > 0) {
51             int capa = cola.size();    // naranjas podridas en este minuto
52             for (int i = 0; i < capa; ++i) {
53                 auto [r, c] = cola.front(); cola.pop();
54                 for (int d = 0; d < 4; ++d) {
55                     int nr = r + dr[d], nc = c + dc[d];
56                     if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
57                     if (grid[nr][nc] != 1) continue;    // vacío o ya podrida
58                     grid[nr][nc] = 2;                // se pudre
59                     --frescas;
60                     cola.push({nr, nc});
61                 }
62             }
63             ++minutos;    // pasó un minuto completo
64         }
65         return frescas == 0 ? minutos : -1;
66     }
67 };
68
69 int main() {
70     Solution s;
71
72     vector<vector<int>> g1 = {{2,1,1},{1,1,0},{0,1,1}};
73     assert(s.orangesRotting(g1) == 4);
74
75     // Imposible: la naranja de abajo-izquierda nunca se pudre.
76     vector<vector<int>> g2 = {{2,1,1},{0,1,1},{1,0,1}};
77     assert(s.orangesRotting(g2) == -1);
78
79     // Sin naranjas frescas => 0 minutos.
80     vector<vector<int>> g3 = {{0,2}};

```

```
81     assert(s.orangesRotting(g3) == 0);
82
83     // Todo vacío => 0.
84     vector<vector<int>> g4 = {{0,0},{0,0}};
85     assert(s.orangesRotting(g4) == 0);
86
87     cout << "994 Rotting Oranges: todas las pruebas pasaron.\n";
88     return 0;
89 }
```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

Notas de entrevista (Amazon). El error típico es empezar `minutos` en -1 o contar un minuto de más cuando la última capa no pudre nada. Aquí sólo incrementamos `minutos` mientras haya frescas que pudrir. Casos borde a mencionar: sin frescas (0), fresca aislada (-1), todo vacío (0).

695. Max Area of Island

(M)

Código fuente: `src/01-grafos/695-max-area-of-island.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan una cuadrícula de 0 (agua) y 1 (tierra). Una *isla* es un grupo de tierras conectadas en horizontal o vertical, y su *área* es cuántas celdas de tierra la forman. Queremos el área de la isla más grande (0 si no hay tierra).

Intuición. Es el mismo sobrevuelo de *Number of Islands*, pero ahora no contamos islas: medimos cada una. Al pisar tierra nueva recorremos toda la isla conectada sumando 1 por celda y hundiéndola para no recontarla. Nos quedamos con el área máxima vista.

```

0  0  1  0  0
0  0  1  1  0
0  1  1  0  0
0  0  0  0  1

```

La isla central conecta 5 celdas; la celda suelta de abajo a la derecha es otra isla de área 1. La respuesta es 5.

Solución. DFS que, en lugar de un valor booleano, devuelve el tamaño de la isla: 1 por la celda actual más la suma recursiva de sus cuatro vecinas. Hundimos cada celda visitada marcándola como agua.

```

1  //
   =====
2  // 695. Max Area of Island (Medium)
3  //
   -----
4  // Cuadrícula de 0 (agua) y 1 (tierra). Una isla es un grupo de tierras
5  // conectadas en horizontal o vertical (no diagonal). El "área" de una isla
6  // es cuántas celdas de tierra la forman. Devuelve el área de la isla más
7  // grande; si no hay tierra, devuelve 0.
8  //
9  // Ejemplo:
10 //   grid =
11 //       0 0 1 0 0           La isla más grande tiene área 5
12 //       0 0 1 1 0           (el bloque conectado del centro).
13 //       0 1 1 0 0           La celda suelta de abajo-derecha es otra isla (área
14 //       0 0 0 0 1           1).
15 //
16 // Diagrama del DFS que mide el área desde una celda de tierra:
17 //
18 //       (r-1,c)
19 //       |
20 //   (r,c-1)-(r,c)-(r,c+1)   Cada tierra visitada aporta 1 al área y se
21 //       |                   hunde (marca como agua) para no recontarla.
22 //       (r+1,c)             El área de la isla = 1 + suma de las 4 vecinas.
23 //

```

```

24 // Idea: es el patrón de "Number of Islands", pero en vez de contar islas,
25 // el DFS devuelve el TAMAÑO de la isla que explora. Guardamos el máximo.
26 //
27 // Complejidad: tiempo  $O(m*n)$  (cada celda se visita una vez),
28 //                espacio  $O(m*n)$  peor caso por la pila de recursión.
29 //
=====

30 #include <vector>
31 #include <cassert>
32 #include <algorithm>
33 #include <iostream>
34 using namespace std;
35
36 class Solution {
37 public:
38     int maxAreaOfIsland(vector<vector<int>>& grid) {
39         if (grid.empty() || grid[0].empty()) return 0;
40         int m = grid.size(), n = grid[0].size();
41         int maximo = 0;
42         for (int r = 0; r < m; ++r)
43             for (int c = 0; c < n; ++c)
44                 if (grid[r][c] == 1) // tierra sin visitar
45                     maximo = max(maximo, area(grid, r, c, m, n));
46         return maximo;
47     }
48
49 private:
50     // Devuelve el área de la isla conectada a (r,c), hundiéndola a su paso.
51     int area(vector<vector<int>>& grid, int r, int c, int m, int n) {
52         if (r < 0 || r >= m || c < 0 || c >= n) return 0; // fuera de límites
53         if (grid[r][c] != 1) return 0; // agua o visitada
54         grid[r][c] = 0; // hundir esta
55             celda
56         return 1
57             + area(grid, r - 1, c, m, n) // arriba
58             + area(grid, r + 1, c, m, n) // abajo
59             + area(grid, r, c - 1, m, n) // izquierda
60             + area(grid, r, c + 1, m, n); // derecha
61     }
62 };
63
64 int main() {
65     Solution s;
66
67     vector<vector<int>> g1 = {
68         {0,0,1,0,0},
69         {0,0,1,1,0},
70         {0,1,1,0,0},
71         {0,0,0,0,1},
72     };
73     assert(s.maxAreaOfIsland(g1) == 5);
74
75     // Toda agua => 0.

```

```
75     vector<vector<int>>> g2 = {{0,0,0},{0,0,0}};  
76     assert(s.maxAreaOfIsland(g2) == 0);  
77  
78     // Una sola celda de tierra => 1.  
79     vector<vector<int>>> g3 = {{0,1,0}};  
80     assert(s.maxAreaOfIsland(g3) == 1);  
81  
82     // Toda tierra => área = m*n.  
83     vector<vector<int>>> g4 = {{1,1},{1,1}};  
84     assert(s.maxAreaOfIsland(g4) == 4);  
85  
86     cout << "695 Max Area of Island: todas las pruebas pasaron.\n";  
87     return 0;  
88 }
```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

Notas de entrevista (Amazon). Si te acaban de preguntar *Number of Islands*, este es el seguimiento natural: cambia el DFS de “contar” a “medir”. Comenta que hundir la cuadrícula ahorra el arreglo `visited`; si no puedes mutar la entrada, usa BFS con cola o un conjunto de visitados. El máximo se puede llevar como acumulador global.

542. 01 Matrix

(M)

Código fuente: [src/01-grafos/542-01-matrix.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una matriz de 0 y 1. Para cada celda debemos calcular su distancia (en pasos horizontales o verticales) al 0 más cercano. La distancia entre celdas adyacentes es 1.

Intuición. La tentación es, para cada 1, buscar el cero más cercano con un BFS propio; eso es lentísimo ($O((mn)^2)$). El truco es invertir la mirada: en vez de partir de cada uno, arrancamos de *todos* los ceros a la vez. Imagina que cada cero suelta una onda que crece un paso por minuto; la primera onda que toca a un 1 viene de su cero más cercano. Eso es un BFS multi-fuente.

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{array} \longrightarrow \begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 2 & 1 \end{array}$$

Los ceros valen 0; el 1 del centro tiene un cero pegado (distancia 1); la celda de en medio de la última fila está a 2 pasos.

Solución. Metemos todos los ceros a la cola con distancia 0 y el resto en infinito. El BFS expande la onda: al visitar una vecina con una distancia mejor que la que tenía, la actualizamos y la encolamos. Como cada celda se fija una sola vez, el costo es lineal.

```

1 //
2 // =====
3 //
4 // 542. 01 Matrix (Medium)
5 //
6 // Dada una matriz de 0 y 1, devuelve una matriz del mismo tamaño donde cada
7 // celda contiene la distancia (número de pasos en 4 direcciones) al 0 más
8 // cercano. La distancia entre celdas adyacentes es 1.
9 //
10 // Ejemplo:
11 // entrada          salida
12 // 0 0 0            0 0 0
13 // 0 1 0            0 1 0
14 // 1 1 1            1 2 1
15 //
16 // Diagrama del BFS multi-fuente (todos los 0 arrancan en la cola a la vez):
17 //
18 // cola inicial: [todas las celdas con valor 0] dist = 0
19 // nivel 1: vecinas de los 0 que aún no tienen distancia -> dist = 1
20 // nivel 2: vecinas de esas -> dist = 2 ...
21 //
22 // La "ola" se expande desde TODOS los ceros simultáneamente, así la primera
23 // vez que una celda 1 es alcanzada, es por su cero más cercano.
24 //
25 // Idea (truco clave): NO hacer un BFS por cada 1 (sería  $O((m*n)^2)$ ). En vez
26 // de
27 // eso, hacer UN BFS multi-fuente partiendo de todos los ceros. Inicializamos

```

```

25 // dist=0 en los ceros e infinito en los unos; la onda del BFS asigna la
26 // distancia mínima a cada uno automáticamente.
27 //
28 // Complejidad: tiempo  $O(m*n)$  (cada celda entra a la cola una vez),
29 //                espacio  $O(m*n)$  por la cola y la matriz de distancias.
30 //
=====
31 #include <vector>
32 #include <queue>
33 #include <cassert>
34 #include <climits>
35 #include <iostream>
36 using namespace std;
37
38 class Solution {
39 public:
40     vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
41         int m = mat.size(), n = mat[0].size();
42         vector<vector<int>> dist(m, vector<int>(n, INT_MAX));
43         queue<pair<int,int>> cola;
44
45         // Fuentes del BFS: todos los ceros con distancia 0.
46         for (int r = 0; r < m; ++r)
47             for (int c = 0; c < n; ++c)
48                 if (mat[r][c] == 0) {
49                     dist[r][c] = 0;
50                     cola.push({r, c});
51                 }
52
53         int dr[] = {-1, 1, 0, 0};
54         int dc[] = {0, 0, -1, 1};
55         while (!cola.empty()) {
56             auto [r, c] = cola.front(); cola.pop();
57             for (int d = 0; d < 4; ++d) {
58                 int nr = r + dr[d], nc = c + dc[d];
59                 if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
60                 // Solo mejora si encontramos un camino más corto (aún sin
61                 // fijar).
62                 if (dist[nr][nc] > dist[r][c] + 1) {
63                     dist[nr][nc] = dist[r][c] + 1;
64                     cola.push({nr, nc});
65                 }
66             }
67         }
68         return dist;
69     };
70
71 int main() {
72     Solution s;
73
74     vector<vector<int>> g1 = {{0,0,0},{0,1,0},{1,1,1}};
75     vector<vector<int>> e1 = {{0,0,0},{0,1,0},{1,2,1}};

```



```
76     assert(s.updateMatrix(g1) == e1);
77
78     vector<vector<int>> g2 = {{0,0,0},{0,1,0},{0,0,0}};
79     vector<vector<int>> e2 = {{0,0,0},{0,1,0},{0,0,0}};
80     assert(s.updateMatrix(g2) == e2);
81
82     // Fila con un solo cero al inicio.
83     vector<vector<int>> g3 = {{0,1,1,1}};
84     vector<vector<int>> e3 = {{0,1,2,3}};
85     assert(s.updateMatrix(g3) == e3);
86
87     cout << "542 01 Matrix: todas las pruebas pasaron.\n";
88     return 0;
89 }
```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

Notas de entrevista (Amazon). La palabra clave que buscan es *BFS multi-fuente*: reconocer que sembrar todas las fuentes a la vez convierte un problema aparentemente cuadrático en lineal. Alternativa clásica: dos pasadas de programación dinámica (arriba-izquierda y luego abajo-derecha), también $O(mn)$ pero sin cola.

1091. Shortest Path in Binary Matrix

(M)

Código fuente: [src/01-grafos/1091-shortest-path-binary-matrix.cpp](#)▷ Ver animación · [local](#) | [web](#)

En una cuadrícula $n \times n$ de 0 y 1, un *camino claro* va de la esquina $(0,0)$ a la esquina $(n-1, n-1)$ pisando solo celdas con 0 y moviéndose en las 8 direcciones (incluidas las diagonales). Su longitud es el número de celdas visitadas. Devolvemos la longitud del camino más corto, o -1 si no existe.

Intuición. En un grafo sin pesos, “camino más corto” es sinónimo de BFS: la onda que sale del origen alcanza cada celda por su ruta mínima. La única diferencia con los BFS de cuadrícula previos es que aquí un paso puede ser diagonal, así que cada celda tiene hasta 8 vecinas (movimiento de rey de ajedrez).

$(r-1, c-1)$	$(r-1, c)$	$(r-1, c+1)$
$(r, c-1)$	(r, c)	$(r, c+1)$
$(r+1, c-1)$	$(r+1, c)$	$(r+1, c+1)$

Las 8 celdas que rodean a (r, c) . La primera vez que la onda toca $(n-1, n-1)$, esa es la distancia mínima en celdas.

Solución. BFS por niveles desde $(0,0)$ (que ya cuenta como celda 1). En cada nivel avanzamos un paso; al sacar el destino de la cola devolvemos la longitud acumulada. Marcamos visitadas mutando la celda a 1 para no reencollarla. Si el origen o el destino están bloqueados, o nunca llegamos, devolvemos -1 .

```

1 //
2 // =====
3 //
4 // 1091. Shortest Path in Binary Matrix (Medium)
5 //
6 // -----
7 // Cuadrícula n x n de 0 y 1. Un "camino claro" va de la esquina superior
8 // izquierda (0,0) a la inferior derecha (n-1,n-1) pisando solo celdas con 0,
9 // moviéndose en las 8 direcciones (incluye diagonales). La longitud del
10 // camino es el NÚMERO DE CELDAS visitadas. Devuelve la longitud del camino
11 // más corto, o -1 si no existe.
12 //
13 // Ejemplo:
14 // grid =                                Camino más corto: (0,0)->(1,1)->(2,2)
15 //      0 0 0                                longitud 4 ... espera, n=3 aquí da 4 celdas
16 //      1 1 0                                si vamos 0,0 -> 0,1 -> 1,2 -> 2,2 (por bordes)
17 //
18 //      1 1 0
19 //
20 // Diagrama de las 8 vecinas (movimiento tipo rey de ajedrez):
21 //
22 //      (r-1,c-1) (r-1,c) (r-1,c+1)
23 //      (r ,c-1)  (r ,c)  (r ,c+1)      BFS: la primera vez que alcanzamos
24 //      (r+1,c-1) (r+1,c) (r+1,c+1)      (n-1,n-1) es por el camino más corto.
25 //

```

```

22 // Idea: BFS clásico de camino más corto en grafo no ponderado. Cada celda 0
23 // es un nodo; hay arista entre celdas 0 adyacentes en las 8 direcciones. BFS
24 // desde (0,0) garantiza que la primera vez que tocamos el destino la
    distancia
25 // (en celdas) es mínima. Marcamos visitadas mutando la celda a 1.
26 //
27 // Complejidad: tiempo  $O(n^2)$  (cada celda entra a la cola una vez),
28 //                espacio  $O(n^2)$  por la cola.
29 //
    =====

30 #include <vector>
31 #include <queue>
32 #include <cassert>
33 #include <iostream>
34 using namespace std;
35
36 class Solution {
37 public:
38     int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
39         int n = grid.size();
40         if (grid[0][0] != 0 || grid[n-1][n-1] != 0) return -1; // extremo
            bloqueado
41
42         queue<pair<int,int>> cola;
43         cola.push({0, 0});
44         grid[0][0] = 1; // marcar visitada
45         int longitud = 1; // (0,0) ya cuenta como 1 celda
46
47         int dr[] = {-1,-1,-1, 0, 0, 1, 1, 1};
48         int dc[] = {-1, 0, 1,-1, 1,-1, 0, 1};
49         while (!cola.empty()) {
50             int capa = cola.size(); // celdas a distancia actual
51             for (int i = 0; i < capa; ++i) {
52                 auto [r, c] = cola.front(); cola.pop();
53                 if (r == n-1 && c == n-1) return longitud; // llegamos
54                 for (int d = 0; d < 8; ++d) {
55                     int nr = r + dr[d], nc = c + dc[d];
56                     if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
57                     if (grid[nr][nc] != 0) continue; // muro o ya visitada
58                     grid[nr][nc] = 1; // marcar visitada
59                     cola.push({nr, nc});
60                 }
61             }
62             ++longitud; // avanzamos una capa (una celda más de camino)
63         }
64         return -1; // nunca alcanzamos el destino
65     }
66 };
67
68 int main() {
69     Solution s;
70
71     vector<vector<int>> g1 = {{0,1},{1,0}};

```

```
72     assert(s.shortestPathBinaryMatrix(g1) == 2);
73
74     vector<vector<int>> g2 = {{0,0,0},{1,1,0},{1,1,0}};
75     assert(s.shortestPathBinaryMatrix(g2) == 4);
76
77     // Origen bloqueado => -1.
78     vector<vector<int>> g3 = {{1,0},{0,0}};
79     assert(s.shortestPathBinaryMatrix(g3) == -1);
80
81     // Una sola celda libre => longitud 1.
82     vector<vector<int>> g4 = {{0}};
83     assert(s.shortestPathBinaryMatrix(g4) == 1);
84
85     // Destino inalcanzable => -1.
86     vector<vector<int>> g5 = {{0,1,0},{1,1,0},{0,0,0}};
87     assert(s.shortestPathBinaryMatrix(g5) == -1);
88
89     cout << "1091 Shortest Path in Binary Matrix: todas las pruebas pasaron.\n";
90     return 0;
91 }
```

Complejidad: tiempo $O(n^2)$, espacio $O(n^2)$.

Notas de entrevista (Amazon). Recalca por qué BFS y no DFS: DFS encuentra *un* camino, no el más corto. Cuida los casos borde de una línea: origen o destino en 1 (respuesta -1 inmediata) y la cuadrícula 1×1 con un solo 0 (longitud 1). Si pidieran pesos, el salto natural sería Dijkstra o 0-1 BFS.

417. Pacific Atlantic Water Flow

(M)

Código fuente: `src/01-grafos/417-pacific-atlantic.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan una matriz `heights` de tamaño $m \times n$ con la altura de cada celda. El océano Pacífico toca el borde superior y el borde izquierdo de la matriz; el océano Atlántico toca el borde inferior y el borde derecho. El agua fluye de una celda a una vecina (arriba, abajo, izquierda, derecha) si la altura de la vecina es **menor o igual** a la de la celda actual (el agua desciende o se mantiene, nunca sube). Queremos las coordenadas de las celdas desde las que el agua puede llegar a **ambos** océanos.

Intuición. Hacer un BFS/DFS de escurrimiento desde cada una de las $m \cdot n$ celdas para ver a qué océano llega sería $O((m \cdot n)^2)$: muy lento. La clave es invertir el problema: en lugar de preguntar “¿a dónde llega el agua desde aquí?”, partimos de los bordes de cada océano y exploramos hacia adentro *subiendo* de altura. Una celda vecina es alcanzable en este recorrido inverso si su altura es mayor o igual a la de la celda actual, porque en el sentido original el agua fluiría de esa vecina hacia la celda que estamos visitando. Así obtenemos, en una sola pasada por océano, el conjunto de celdas que logran drenar hasta él. La respuesta final es la intersección de ambos conjuntos.

```

1  2  2  3  5
3  2  3  4  4
2  4  5  3  1
6  7  1  4  5
5  1  1  2  4

```

En este ejemplo, el resultado es $(0, 4), (1, 3), (1, 4), (2, 2), (3, 0), (3, 1), (4, 0)$: desde cada una de estas celdas el agua puede descender hasta tocar tanto el borde superior/izquierdo como el inferior/derecho.

Diagrama. DFS inverso desde cada océano, subiendo de altura hacia adentro de la matriz:

Pacífico (fila 0 y columna 0)	Atlántico (fila $m-1$ y columna $n-1$)
P P P P P	A
P	A
P	(subiendo de altura) A
P	A
P	A A A A A A A A

Solución. Dos matrices booleanas `pacifico` y `atlantico` marcan qué celdas alcanzan cada océano. Lanzamos un DFS desde cada celda del borde correspondiente; el DFS sólo avanza hacia una vecina cuya altura sea mayor o igual (nunca hacia una más baja, porque el agua no podría subir en el sentido inverso). Al final, las celdas marcadas en ambas matrices forman la respuesta:

```

1 //
  =====
2 // 417. Pacific Atlantic Water Flow (Medium)
3 //
  -----

```

```

4 // Dada una matriz de alturas `heights` de tamaño m x n, el océano Pacífico
5 // toca el borde superior y el borde izquierdo; el océano Atlántico toca el
6 // borde inferior y el borde derecho. El agua fluye de una celda a una vecina
7 // (arriba, abajo, izquierda, derecha) si la altura de la vecina es MENOR O
8 // IGUAL a la altura de la celda actual (el agua desciende o se mantiene).
9 // Debemos devolver las coordenadas de las celdas desde las que el agua puede
10 // llegar a AMBOS océanos.
11 //
12 // Ejemplo (el clásico de LeetCode):
13 // heights =
14 //   1  2  2  3  5
15 //   3  2  3  4  4
16 //   2  4  5  3  1
17 //   6  7  1  4  5
18 //   5  1  1  2  4
19 //
20 // Resultado: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
21 //
22 // Diagrama de la idea (BFS/DFS INVERSO desde cada océano):
23 //
24 // Pacífico toca Atlántico toca
25 // fila 0 y columna 0 fila m-1 y columna n-1
26 // ↓↓
27 // +-----+ +-----+
28 // |P P P P P | | A A |
29 // |P | | | | | | | A |
30 // |P | | | | | | | A |
31 // |P | | | | | | | A |
32 // |P | | | | | | | A |
33 // +-----+ +-----+
34 //
35 // Idea: hacer BFS/DFS desde cada celda por separado sería  $O((m*n)^2)$ . En su
36 // lugar invertimos el problema: partimos de los bordes de cada océano y
37 // exploramos hacia adentro SUBIENDO de altura (el reverso de "fluir hacia
38 // abajo"). Una celda vecina es alcanzable en el grafo inverso si su altura es
39 // MAYOR O IGUAL a la de la celda actual (porque en el grafo original el agua
40 // fluiría de esa vecina hacia la celda actual). Así marcamos qué celdas
41 // alcanzan el Pacífico y cuáles alcanzan el Atlántico; la respuesta es la
42 // intersección de ambos conjuntos.
43 //
44 // Complejidad: tiempo  $O(m*n)$  (cada celda se visita a lo más una vez por
45 // océano), espacio  $O(m*n)$  por las matrices de visitados y la
46 // pila de recursión.
47 //
=====
48 #include <vector>
49 #include <algorithm>
50 #include <cassert>
51 #include <iostream>
52 using namespace std;
53
54 class Solution {
55 public:

```

```

56     vector<vector<int>>> pacificAtlantic(vector<vector<int>>& heights) {
57         vector<vector<int>>> resultado;
58         if (heights.empty() || heights[0].empty()) return resultado;
59         int m = heights.size(), n = heights[0].size();
60
61         vector<vector<bool>>> pacifico(m, vector<bool>(n, false));
62         vector<vector<bool>>> atlantico(m, vector<bool>(n, false));
63
64         // Bordes del Pacífico: fila 0 y columna 0.
65         // Bordes del Atlántico: fila m-1 y columna n-1.
66         for (int c = 0; c < n; ++c) {
67             dfs(heights, pacifico, 0, c, m, n);
68             dfs(heights, atlantico, m - 1, c, m, n);
69         }
70         for (int r = 0; r < m; ++r) {
71             dfs(heights, pacifico, r, 0, m, n);
72             dfs(heights, atlantico, r, n - 1, m, n);
73         }
74
75         for (int r = 0; r < m; ++r)
76             for (int c = 0; c < n; ++c)
77                 if (pacifico[r][c] && atlantico[r][c])
78                     resultado.push_back({r, c});
79
80         return resultado;
81     }
82
83 private:
84     // DFS inverso: desde (r,c) avanza hacia una vecina sólo si su altura es
85     // mayor o igual (en el grafo original el agua fluiría de esa vecina hacia
86     // aquí, ya que fluye de mayor/igual altura hacia menor/igual altura).
87     void dfs(vector<vector<int>>& heights, vector<vector<bool>>& visitado,
88             int r, int c, int m, int n) {
89         visitado[r][c] = true;
90         int dr[] = {-1, 1, 0, 0};
91         int dc[] = {0, 0, -1, 1};
92         for (int d = 0; d < 4; ++d) {
93             int nr = r + dr[d], nc = c + dc[d];
94             if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
95             if (visitado[nr][nc]) continue;
96             if (heights[nr][nc] < heights[r][c]) continue; // agua no podría
97                 subir
98             dfs(heights, visitado, nr, nc, m, n);
99         }
100 };
101
102 // Compara dos listas de coordenadas sin importar el orden.
103 bool mismasCeldas(vector<vector<int>>& a, vector<vector<int>>& b) {
104     sort(a.begin(), a.end());
105     sort(b.begin(), b.end());
106     return a == b;
107 }
108

```

```
109 int main() {
110     Solution s;
111
112     // Caso clásico de LeetCode.
113     vector<vector<int>> h1 = {
114         {1, 2, 2, 3, 5},
115         {3, 2, 3, 4, 4},
116         {2, 4, 5, 3, 1},
117         {6, 7, 1, 4, 5},
118         {5, 1, 1, 2, 4},
119     };
120     vector<vector<int>> esperado1 = {
121         {0, 4}, {1, 3}, {1, 4}, {2, 2}, {3, 0}, {3, 1}, {4, 0}
122     };
123     assert(mismasCeldas(s.pacificAtlantic(h1), esperado1));
124
125     // Caso borde: matriz 1x1. La única celda toca los cuatro bordes a la
126     // vez, así que siempre alcanza ambos océanos sin importar su altura.
127     vector<vector<int>> h2 = {{5}};
128     vector<vector<int>> esperado2 = {{0, 0}};
129     assert(mismasCeldas(s.pacificAtlantic(h2), esperado2));
130
131     // Caso borde: una sola fila. Como m=1, esa fila es a la vez la fila
132     // superior (Pacífico) y la inferior (Atlántico): todas las celdas
133     // alcanzan ambos océanos de inmediato, sin importar la altura.
134     vector<vector<int>> h3 = {{1, 2, 3}};
135     vector<vector<int>> esperado3 = {{0, 0}, {0, 1}, {0, 2}};
136     assert(mismasCeldas(s.pacificAtlantic(h3), esperado3));
137
138     // Caso borde: una sola columna. Como n=1, esa columna es a la vez la
139     // columna izquierda (Pacífico) y la derecha (Atlántico): todas las
140     // celdas alcanzan ambos océanos de inmediato.
141     vector<vector<int>> h4 = {{1}, {2}, {3}};
142     vector<vector<int>> esperado4 = {{0, 0}, {1, 0}, {2, 0}};
143     assert(mismasCeldas(s.pacificAtlantic(h4), esperado4));
144
145     // Meseta plana: toda la matriz tiene la misma altura, así que el agua
146     // fluye libremente entre vecinas y todas las celdas llegan a ambos
147     // océanos.
148     vector<vector<int>> h5 = {
149         {3, 3, 3},
150         {3, 3, 3},
151         {3, 3, 3},
152     };
153     vector<vector<int>> esperado5 = {
154         {0, 0}, {0, 1}, {0, 2},
155         {1, 0}, {1, 1}, {1, 2},
156         {2, 0}, {2, 1}, {2, 2},
157     };
158     assert(mismasCeldas(s.pacificAtlantic(h5), esperado5));
159
160     cout << "417 Pacific Atlantic: todas las pruebas pasaron.\n";
161     return 0;
162 }
```


Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

Notas de entrevista (Amazon). El truco central es reconocer que invertir el sentido del recorrido (partir de los océanos en vez de partir de cada celda) baja la complejidad de $O((m \cdot n)^2)$ a $O(m \cdot n)$; es un patrón que vale la pena mencionar en voz alta al entrevistador. Casos borde importantes: matriz 1×1 (la única celda toca los cuatro bordes a la vez), una sola fila o columna (la misma fila/columna es simultáneamente borde de ambos océanos), y mesetas de altura uniforme (el agua fluye libremente entre vecinas de igual altura porque la condición es “menor o igual”).

79. Word Search

(M)

Código fuente: [src/01-grafos/79-word-search.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una cuadrícula de letras (**board**) y una palabra (**word**). Queremos saber si la palabra se puede formar recorriendo celdas adyacentes en horizontal o vertical (nunca en diagonal), sin usar la misma celda dos veces dentro de un mismo camino.

Intuición. Piensa que vas letra por letra de la palabra, parado en una celda, y buscas a cuál vecina moverte para encontrar la siguiente letra. Si te atorras, retrocedes (*backtracking*) y pruebas otro camino. Para no pisar dos veces la misma celda en el mismo intento, la marcamos temporalmente (por ejemplo con '#') mientras la estamos usando, y la restauramos a su letra original en cuanto retrocedemos, de modo que otros caminos sí puedan usarla.

A	B	C	E
S	F	C	S
A	D	E	E

En esta cuadrícula, "ABCCED" existe (bajando por la columna central de la C y luego a la E y la D), y "SEE" también existe. En cambio "ABCB" no existe: tras usar la primera B, la única forma de volver a una B obligaría a reutilizar una celda ya visitada en ese mismo camino.

Solución. Probamos cada celda de la cuadrícula como posible punto de partida (**word[0]**) y hacemos DFS con backtracking: si la celda actual coincide con la letra esperada, la marcamos como visitada y probamos las 4 vecinas buscando la siguiente letra; si ningún vecino completa el camino, restauramos la celda antes de volver.

```

1 //
  =====
2 // 79. Word Search (Medium)
3 //
  -----
4 // Dada una cuadrícula de letras (board) y una palabra (word), decide si la
5 // palabra existe formando un camino de celdas adyacentes en horizontal o
6 // vertical (no diagonal), sin reutilizar una misma celda dos veces.
7 //
8 // Ejemplo:
9 //   board =
10 //       A B C E       word = "ABCCED"  -> true (A-B-C-C-E-D, bajando y
11 //       S F C S       recorriendo por la columna central)
12 //       A D E E       word = "SEE"     -> true
13 //                   word = "ABCB"     -> false (la segunda B reutilizaría
14 //                                           la celda ya usada por la primera)
15 //
16 // Diagrama de la búsqueda DFS con backtracking desde una celda (r,c) que
17 // coincide con word[i]:
18 //
19 //       (r-1,c)
20 //       |

```

```

21 //      (r,c-1)-(r,c)-(r,c+1)      Si board[r][c] == word[i], marcamos la celda
22 //      |                           como visitada ('#') y probamos las 4 vecinas
23 //      (r+1,c)                     buscando word[i+1]. Si ninguna funciona,
24 //                                 restauramos la celda (backtrack) antes de
25 //                                 volver, para que otros caminos puedan usarla.
26 //
27 // Idea: para cada celda de la cuadrícula, intentamos que sea el punto de
28 // partida (word[0]) y hacemos DFS con backtracking. Al entrar a una celda la
29 // marcamos temporalmente para no reusarla en el mismo camino; al salir (ya
30 // sea que el camino tuvo éxito o fracasó) la restauramos a su valor original.
31 //
32 // Complejidad: tiempo  $O(m*n*4^L)$  donde  $L$  = longitud de word (en el peor caso,
33 // desde cada una de las  $m*n$  celdas se abren hasta 4 ramas por
34 // cada letra de la palabra),
35 // espacio  $O(L)$  por la profundidad de la pila de recursión.
36 //
=====

37 #include <vector>
38 #include <string>
39 #include <cassert>
40 #include <iostream>
41 using namespace std;
42
43 class Solution {
44 public:
45     bool exist(vector<vector<char>>& board, string word) {
46         if (board.empty() || board[0].empty() || word.empty()) return false;
47         int m = board.size(), n = board[0].size();
48         for (int r = 0; r < m; ++r) {
49             for (int c = 0; c < n; ++c) {
50                 if (buscar(board, word, r, c, 0, m, n)) return true;
51             }
52         }
53         return false;
54     }
55
56 private:
57     // Intenta continuar el camino de word desde word[i] partiendo de (r,c).
58     bool buscar(vector<vector<char>>& board, const string& word,
59                 int r, int c, int i, int m, int n) {
60         if (i == (int)word.size()) return true; // ya se emparejó toda la
61         palabra
62         if (r < 0 || r >= m || c < 0 || c >= n) return false; // fuera de lí
63         mites
64         if (board[r][c] != word[i]) return false; // no coincide o ya
65         visitada ('#')
66
67         char original = board[r][c];
68         board[r][c] = '#'; // marcamos la celda como usada en este camino
69
70         bool encontrada =
71             buscar(board, word, r - 1, c, i + 1, m, n) || // arriba
72             buscar(board, word, r + 1, c, i + 1, m, n) || // abajo

```

```

70         buscar(board, word, r, c - 1, i + 1, m, n) || // izquierda
71         buscar(board, word, r, c + 1, i + 1, m, n);    // derecha
72
73         board[r][c] = original; // backtrack: restauramos antes de volver
74         return encontrada;
75     }
76 };
77
78 int main() {
79     Solution s;
80
81     // Palabra presente: A-B-C-C-E-D es un camino válido (baja por la columna
82     // central de la C y luego va a la E y a la D).
83     vector<vector<char>> b1 = {
84         {'A', 'B', 'C', 'E'},
85         {'S', 'F', 'C', 'S'},
86         {'A', 'D', 'E', 'E'},
87     };
88     assert(s.exist(b1, "ABCCED") == true);
89
90     // Palabra presente, camino corto por la columna derecha.
91     vector<vector<char>> b2 = b1;
92     assert(s.exist(b2, "SEE") == true);
93
94     // Palabra ausente: no existe ningún camino que forme "ABCB" porque,
95     // tras usar la primera B, la única forma de volver a una B requeriría
96     // reutilizar la celda ya visitada.
97     vector<vector<char>> b3 = b1;
98     assert(s.exist(b3, "ABCB") == false);
99
100    // Palabra de una sola letra: basta con que exista esa letra en la
101    // cuadrícula (no requiere adyacencia con nada más).
102    vector<vector<char>> b4 = b1;
103    assert(s.exist(b4, "F") == true);
104    assert(s.exist(b4, "Z") == false);
105
106    // Cuadrícula de una sola celda.
107    vector<vector<char>> b5 = {{'A'}};
108    assert(s.exist(b5, "A") == true);
109    assert(s.exist(b5, "AA") == false); // reutilizaría la única celda
110
111    cout << "79 Word Search: todas las pruebas pasaron.\n";
112    return 0;
113 }

```

Complejidad: tiempo $O(m \cdot n \cdot 4^L)$, con L la longitud de `word`, espacio $O(L)$ por la recursión.

Notas de entrevista (Amazon). El punto clave es el backtracking: marcar la celda al entrar y **siempre** restaurarla al salir, incluso si el camino tuvo éxito antes de regresar la llamada. Si no puedes mutar la cuadrícula de entrada, usa una matriz `visited` paralela en vez de '#'. Casos borde: palabra de una sola letra (basta con que exista en la cuadrícula), cuadrícula de una sola celda, y palabras que reutilizarían una celda (deben fallar).

207. Course Schedule

(M)

Código fuente: `src/01-grafos/207-course-schedule.cpp`▷ Ver animación · [local](#) | [web](#)

Hay `numCourses` cursos numerados de 0 a `numCourses - 1`, y una lista `prerequisites` de pares $[a, b]$ que significan “para tomar el curso a primero hay que tomar el curso b ”. Queremos saber si es posible tomar todos los cursos.

Intuición. Cada par $[a, b]$ es una dependencia dirigida $b \rightarrow a$ (“ b habilita a a ”). Si dibujamos todas las dependencias como un grafo dirigido, la pregunta “¿puedo tomar todos los cursos?” es exactamente “¿este grafo tiene un ciclo?”. Si un grupo de cursos se necesita entre sí en círculo, ninguno de ellos puede ser el primero: es imposible. Si no hay ciclos, siempre existe un **orden topológico**: una forma de ordenar los cursos tal que cada uno aparece después de todos sus prerequisites.

Con ciclo (imposible):	Sin ciclo (posible):
$0 \rightarrow 1 \rightarrow 0$ (0 necesita 1, 1 necesita 0)	$0 \rightarrow 1 \rightarrow 3, \quad 0 \rightarrow 2 \rightarrow 3$ orden topológico: 0,1,2,3

Grado de entrada (in-degree). Es un concepto nuevo en este libro: el *grado de entrada* de un curso es cuántos prerequisites le faltan por cumplir. Un curso con grado de entrada 0 no depende de nada y se puede tomar de inmediato.

Solución (algoritmo de Kahn). Construimos la lista de adyacencia ($\text{adj}[b]$ = cursos que dependen de b) y el grado de entrada de cada curso. Metemos en una cola todos los cursos con grado de entrada 0. Sacamos uno a la vez, lo contamos como “tomado” y disminuimos en 1 el grado de entrada de todo lo que dependía de él; cuando un curso llega a grado 0, entra a la cola. Este proceso es un **BFS sobre grados de entrada** y va generando un orden topológico capa por capa. Si al final procesamos todos los cursos, no había ciclo; si alguno se quedó sin procesar (su grado de entrada nunca bajó a 0 porque estaba atrapado en un ciclo), la respuesta es falsa.

```

1 //
  =====
2 // 207. Course Schedule (Medium)
3 //
  -----
4 // Hay numCourses cursos, numerados de 0 a numCourses-1. La lista
  // prerequisites
5 // contiene pares [a, b] que significan "para tomar el curso a, primero hay
6 // que tomar el curso b". Determina si es posible tomar todos los cursos.
7 //
8 // Esto equivale a preguntar si el grafo dirigido de dependencias (b -> a,
9 // "b habilita a a") tiene un ciclo. Si hay un ciclo, esos cursos se necesitan
10 // entre sí circularmente y nunca se puede empezar. Si no hay ciclo, siempre
11 // existe un orden válido para cursarlos: un orden topológico.
12 //
13 // Ejemplo con ciclo (imposible):
14 //   numCourses = 2, prerequisites = [[1,0],[0,1]]
15 //   0 --> 1

```

```

16 //      ^-----/      para tomar 1 necesitas 0, y para tomar 0 necesitas 1:
17 //                      ciclo, nunca se puede empezar.
18 //
19 // Ejemplo sin ciclo (posible):
20 //   numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
21 //   0 --> 1 --> 3
22 //   0 --> 2 --> 3      orden topológico válido: 0, 1, 2, 3
23 //
24 // Idea (algoritmo de Kahn, BFS con grados de entrada):
25 // - El "grado de entrada" (in-degree) de un curso es cuántos prerequisites
26 //   le faltan por cumplir antes de poder tomarlo.
27 // - Empezamos metiendo en una cola todos los cursos con grado de entrada 0
28 //   (los que no necesitan nada, se pueden tomar de inmediato).
29 // - Sacamos un curso de la cola, lo "tomamos" (lo contamos como procesado) y
30 //   reducimos en 1 el grado de entrada de todos los cursos que dependían de
31 //   él. Si alguno llega a 0, ya cumplió todos sus prerequisites y entra a
32 //   la cola.
33 // - Si al final procesamos exactamente numCourses cursos, no había ciclo y
34 //   existe un orden topológico completo => true. Si sobran cursos sin
35 //   procesar (se quedaron atorados en un ciclo, su grado de entrada nunca
36 //   baja a 0) => false.
37 //
38 // Complejidad: tiempo  $O(V+E)$  ( $V$  cursos,  $E$  prerequisites),
39 //               espacio  $O(V+E)$  por la lista de adyacencia y los grados.
40 //
=====

41 #include <vector>
42 #include <queue>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 class Solution {
48 public:
49     bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
50         vector<vector<int>> adj(numCourses);    // adj[b] = cursos que dependen
51         // de b
52         vector<int> gradoEntrada(numCourses, 0);
53
54         for (auto& pre : prerequisites) {
55             int a = pre[0], b = pre[1];    // para tomar a, primero hay que
56             // tomar b
57             adj[b].push_back(a);
58             ++gradoEntrada[a];
59         }
60
61         queue<int> cola;    // cursos listos para tomarse (sin prerequisites
62         // pendientes)
63         for (int curso = 0; curso < numCourses; ++curso)
64             if (gradoEntrada[curso] == 0) cola.push(curso);
65
66         int procesados = 0;
67         while (!cola.empty()) {

```

```

65         int curso = cola.front(); cola.pop();
66         ++procesados;    // "tomamos" este curso
67         for (int siguiente : adj[curso]) {
68             if (--gradoEntrada[siguiente] == 0) cola.push(siguiente);
69         }
70     }
71
72     return procesados == numCourses;    // si falta alguno, había un ciclo
73 }
74 };
75
76 int main() {
77     Solution s;
78
79     // Sin prerequisites: siempre se puede tomar todo.
80     vector<vector<int>> p1 = {};
81     assert(s.canFinish(2, p1) == true);
82
83     // Cadena lineal: 1 necesita 0, 2 necesita 1, 3 necesita 2. Sin ciclos.
84     vector<vector<int>> p2 = {{1,0},{2,1},{3,2}};
85     assert(s.canFinish(4, p2) == true);
86
87     // Ciclo simple: 0 necesita 1 y 1 necesita 0. Imposible.
88     vector<vector<int>> p3 = {{0,1},{1,0}};
89     assert(s.canFinish(2, p3) == false);
90
91     // Varios cursos con un ciclo escondido entre 1, 2 y 3 (0 sí se puede
92     // tomar).
93     vector<vector<int>> p4 = {{1,0},{2,1},{3,2},{1,3}};
94     assert(s.canFinish(4, p4) == false);
95
96     cout << "207 Course Schedule: todas las pruebas pasaron.\n";
97     return 0;
98 }

```

Complejidad: tiempo $O(V + E)$, espacio $O(V + E)$.

Notas de entrevista (Amazon). Este problema es la puerta de entrada al patrón de **orden topológico**, muy común en preguntas de planeación de tareas y resolución de dependencias (build systems, compiladores, gestores de paquetes). Menciona ambas variantes: BFS con grados de entrada (algoritmo de Kahn, el que usamos aquí, útil también si se pide devolver el orden explícito) o DFS con detección de ciclo por colores (blanco/gris/negro). Casos borde: sin prerequisites (siempre posible), un ciclo que involucra solo a algunos cursos y no a todos, y cursos con múltiples prerequisites o múltiples dependientes.

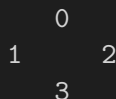
210. Course Schedule II

(M)

Código fuente: `src/01-grafos/210-course-schedule-ii.cpp`▷ Ver animación · [local](#) | [web](#)

Hay que cursar `numCourses` materias numeradas de 0 a `numCourses-1`. Nos dan una lista de prerequisites `prerequisites[i] = [a, b]`, que significa que para tomar el curso `a` primero hay que haber tomado el curso `b`. Queremos devolver *un* orden válido para cursar todas las materias, o un vector vacío si es imposible (existe un ciclo de dependencias).

Intuición. Es la continuación natural del problema 207 (Course Schedule): ahí solo preguntaban si era *posible* terminar todos los cursos; aquí además hay que construir el orden concreto. La idea sigue siendo la misma: un curso solo se puede tomar cuando ya se tomaron todos sus prerequisites, así que vamos "liberando" cursos conforme se cumplen sus dependencias, con el algoritmo de Kahn (BFS sobre el grado de entrada).



Con `prerequisites = [[1,0],[2,0],[3,1],[3,2]]`: para tomar 1 y 2 hace falta 0; para tomar 3 hacen falta 1 y 2. Un orden válido es `[0, 1, 2, 3]` (o `[0, 2, 1, 3]`): el orden topológico no es único cuando hay ramas independientes.

Solución. Construimos la lista de adyacencia `curso -> [dependientes]` y el grado de entrada de cada curso (cuántos prerequisites le faltan). Metemos en una cola todos los cursos con grado de entrada 0; al sacar un curso de la cola lo agregamos al resultado y restamos 1 al grado de entrada de sus dependientes, y el que llegue a 0 entra a la cola. Si al final el resultado no incluye a los `numCourses` cursos, hay un ciclo y devolvemos un vector vacío.

```

1 //
2 // =====
3 //
4 // 210. Course Schedule II (Medium)
5 //
6 // Hay que cursar "numCourses" materias (0 .. numCourses-1). Nos dan una lista
7 // de prerequisites "prerequisites[i] = [a, b]", que significa: para tomar el
8 // curso a hace falta haber tomado antes el curso b. Devuelve UN orden válido
9 // en el que se pueden cursar todas las materias, o un vector vacío si es
10 // imposible (existe un ciclo de dependencias).
11 //
12 // Es la continuación natural del problema 207 (Course Schedule): ahí solo
13 // preguntaban si era posible terminar todos los cursos (true/false); aquí
14 // además hay que construir el orden topológico concreto.
15 //
16 // Ejemplo:
17 //   numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
18 //   Para tomar 1 y 2 hace falta 0; para tomar 3 hacen falta 1 y 2.
19 //
20 // Diagrama del grafo dirigido (a -> b significa "b depende de a"):

```



```

19 //
20 //      0
21 //     / \
22 //    1   2
23 //     \ /
24 //      3
25 //
26 // Un orden válido es [0, 1, 2, 3] (o [0, 2, 1, 3]): el orden topológico no
27 // es único cuando hay ramas independientes.
28 //
29 // Idea (algoritmo de Kahn, BFS con grado de entrada):
30 // 1) Construimos la lista de adyacencia curso -> [cursos que dependen de él
31 // ]
32 // y calculamos el grado de entrada (in-degree) de cada curso: cuántos
33 // prerequisitos le faltan.
34 // 2) Metemos en una cola todos los cursos con grado de entrada 0 (se pueden
35 // tomar de inmediato, sin prerequisitos pendientes).
36 // 3) Sacamos un curso de la cola, lo agregamos al resultado, y "liberamos"
37 // a sus dependientes restándoles 1 al grado de entrada; el que llegue a
38 // 0 entra a la cola.
39 // 4) Si al final el resultado no incluye a los "numCourses" cursos, hay un
40 // ciclo (algunos cursos nunca bajan a grado de entrada 0) => devolvemos
41 // un vector vacío.
42 // Complejidad: tiempo  $O(V + E)$  ( $V$  cursos,  $E$  prerequisitos),
43 // espacio  $O(V + E)$  por la lista de adyacencia y la cola.
44 //
=====

45 #include <vector>
46 #include <queue>
47 #include <cassert>
48 #include <iostream>
49 using namespace std;
50
51 class Solution {
52 public:
53     vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites)
54     {
55         vector<vector<int>> adj(numCourses); // adj[b] = cursos que dependen
56         // de b
57         vector<int> grado(numCourses, 0); // grado de entrada de cada
58         // curso
59
60         for (auto& pr : prerequisites) {
61             int a = pr[0], b = pr[1]; // b es prerequisito de a
62             adj[b].push_back(a);
63             ++grado[a];
64         }
65
66         queue<int> cola; // cursos listos para tomar (sin prerequisitos
67         // pendientes)
68         for (int curso = 0; curso < numCourses; ++curso)
69             if (grado[curso] == 0) cola.push(curso);

```

```

66     vector<int> orden;
67     orden.reserve(numCourses);
68
69     while (!cola.empty()) {
70         int actual = cola.front(); cola.pop();
71         orden.push_back(actual);
72         for (int dependiente : adj[actual]) {
73             if (--grado[dependiente] == 0) cola.push(dependiente);
74         }
75     }
76
77     if ((int)orden.size() != numCourses) return {}; // había un ciclo
78     return orden;
79 }
80 };
81
82 // -----
83 // Verifica que "orden" sea un orden topológico válido para "numCourses"
84 // cursos
85 // con las dependencias dadas en "prerequisitos": debe incluir a todos los
86 // cursos exactamente una vez, y cada prerequisito debe aparecer antes que el
87 // curso que lo necesita. Como el orden topológico no es único, esta función
88 // nos permite comparar sin depender de un vector fijo.
89 bool esOrdenValido(const vector<int>& orden, int numCourses,
90                   const vector<vector<int>>& prerequisitos) {
91     if ((int)orden.size() != numCourses) return false;
92
93     vector<int> posicion(numCourses, -1);
94     vector<bool> visto(numCourses, false);
95     for (int i = 0; i < (int)orden.size(); ++i) {
96         int curso = orden[i];
97         if (curso < 0 || curso >= numCourses || visto[curso]) return false;
98         // repetido o fuera de rango
99         visto[curso] = true;
100        posicion[curso] = i;
101    }
102
103    for (auto& pr : prerequisitos) {
104        int a = pr[0], b = pr[1]; // b debe ir antes que a
105        if (posicion[b] >= posicion[a]) return false;
106    }
107    return true;
108 }
109
110 int main() {
111     Solution s;
112
113     // Sin prerequisitos: cualquier permutación de 0..numCourses-1 es válida.
114     vector<vector<int>> p1 = {};
115     auto o1 = s.findOrder(2, p1);
116     assert(esOrdenValido(o1, 2, p1));
117
118     // Cadena lineal: 0 <- 1 <- 2 <- 3 (cada curso depende del anterior).

```

```

118 // El orden topológico es único: [0, 1, 2, 3].
119 vector<vector<int>> p2 = {{1, 0}, {2, 1}, {3, 2}};
120 auto o2 = s.findOrder(4, p2);
121 vector<int> esperado2 = {0, 1, 2, 3};
122 assert(o2 == esperado2);
123 assert(esOrdenValido(o2, 4, p2));
124
125 // Grafo con ramas independientes (0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3):
126 // hay más de un orden válido, así que solo verificamos con esOrdenValido.
127 vector<vector<int>> p3 = {{1, 0}, {2, 0}, {3, 1}, {3, 2}};
128 auto o3 = s.findOrder(4, p3);
129 assert(esOrdenValido(o3, 4, p3));
130
131 // Ciclo: 0 depende de 1 y 1 depende de 0 => imposible, vector vacío.
132 vector<vector<int>> p4 = {{0, 1}, {1, 0}};
133 auto o4 = s.findOrder(2, p4);
134 assert(o4.empty());
135
136 // Ciclo más grande escondido entre cursos válidos: 0 -> 1 -> 2 -> 1.
137 vector<vector<int>> p5 = {{1, 0}, {2, 1}, {1, 2}};
138 auto o5 = s.findOrder(3, p5);
139 assert(o5.empty());
140
141 cout << "210 Course Schedule II: todas las pruebas pasaron.\n";
142 return 0;
143 }

```

Complejidad: tiempo $O(V + E)$, espacio $O(V + E)$.

Notas de entrevista (Amazon). Como el orden topológico casi nunca es único, en la entrevista aclara que cualquier orden que respete las dependencias es una respuesta válida; si te piden verificar tu propia solución, basta con checar que cada prerrequisito aparezca antes que su curso. Casos borde a mencionar: sin prerrequisitos (cualquier permutación sirve), cadena lineal (orden único), y ciclo (respuesta vacía, igual que "false" en el 207).

269. Alien Dictionary

(H)

Código fuente: [src/01-grafos/269-alien-dictionary.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una lista de palabras que ya están ordenadas según las reglas de un alfabeto alienígena que no conocemos. A partir de esas palabras hay que deducir un orden válido de las letras del alfabeto. Si la lista es inconsistente (no existe ningún orden posible), devolvemos "".

Intuición. Cada par de palabras *adyacentes* en la lista nos regala como mucho una pista: el primer carácter en el que difieren nos dice “esta letra va antes que esa otra”. El resto de los caracteres no aporta nada sobre el alfabeto. Con todas esas pistas armamos un grafo dirigido letra $\rightarrow$ letra, y basta un orden topológico de ese grafo para responder.

```
wrt wrf (diff. índice 2: t  $\rightarrow$  f)
wrf er (diff. índice 0: w  $\rightarrow$  e)
er ett (diff. índice 1: r  $\rightarrow$  t)
ett rftt (diff. índice 0: e  $\rightarrow$  r)
```

Grafo resultante: $w \rightarrow e \rightarrow r \rightarrow t \rightarrow f$

En este ejemplo las aristas forman una sola cadena, así que el orden topológico es único: "wertf".

Solución. Construimos el grafo comparando cada par de palabras consecutivas y tomando el primer par de caracteres distintos (esa es la única arista que aporta ese par). Luego aplicamos el algoritmo de Kahn: metemos en una cola las letras con grado de entrada 0, y las vamos sacando mientras liberamos a sus sucesoras. Si al final no procesamos todas las letras, había un ciclo y devolvemos "".

Hay un caso borde que es fácil pasar por alto: si dos palabras adyacentes comparten todo su prefijo común pero la primera es *más larga* que la segunda (por ejemplo ["abc", "ab"]), eso viola el orden alfabético — un prefijo siempre debe ir antes que la palabra que lo extiende — así que devolvemos "" de inmediato, sin llegar siquiera a construir el topológico.

```
1 //
2 // =====
3 //
4 // 269. Alien Dictionary (Hard)
5 //
6 // -----
7 //
8 // Nos dan una lista de palabras que ya vienen ORDENADAS según las reglas del
9 // alfabeto de un idioma alienígena (desconocido para nosotros). Hay que
10 // deducir un orden válido de las letras de ese alfabeto. Si la lista es
11 // inconsistente con cualquier orden posible, devolvemos "".
12 //
13 // Ejemplo:
14 // words = ["wrt", "wrf", "er", "ett", "rftt"]
15 // Comparando cada par de palabras adyacentes y viendo el primer carácter
16 // en el que difieren, obtenemos las reglas: w antes de e, r antes de t (via
17 // "er"/"ett"), e antes de r, y t antes de f. Un orden válido es "wertf".
18 //
19 // Diagrama: cada par de palabras adyacentes aporta UNA arista dirigida, la
```

```

16 // del primer carácter donde difieren (el resto de la palabra no dice nada
17 // sobre el orden del alfabeto):
18 //
19 //   wrt   wrf   w r t           diff en índice 2: t -> f
20 //   |     |
21 //   wrf   er    w r f
22 //   |     |
23 //   er    ett   e r           diff en índice 0: w -> e
24 //   |     |
25 //   ett   rftt   e t t       diff en índice 1: r -> t
26 //           r f t t       diff en índice 0: e -> r
27 //
28 //   Grafo resultante:   w -> e -> r -> t -> f
29 //
30 // Idea: construimos un grafo dirigido letra -> letra comparando cada par de
31 // palabras consecutivas y tomando el primer par de caracteres distintos.
32 // Luego sacamos UN orden topológico (algoritmo de Kahn) sobre ese grafo; si
33 // no existe (hay un ciclo), la respuesta es "".
34 //
35 // Casos especiales CRÍTICOS:
36 //   (a) Prefijo inválido: si la palabra anterior es más larga que la
37 //       siguiente y la siguiente es un prefijo de la anterior (por ejemplo
38 //       ["abc", "ab"]), eso viola el orden alfabético (un prefijo siempre
39 //       debe ir antes que la palabra que lo extiende) => "".
40 //   (b) Ciclo en el grafo de letras => "" (no existe orden topológico).
41 //
42 // Complejidad: tiempo  $O(C)$ , espacio  $O(1)$  sobre el alfabeto (a lo más 26
43 // letras), donde  $C$  es la suma de longitudes de todas las palabras (el tiempo
44 // que toma comparar palabras adyacentes domina sobre el topológico, que es
45 //  $O(26)$  en el peor caso).
46 //
=====

47 #include <vector>
48 #include <string>
49 #include <queue>
50 #include <unordered_map>
51 #include <unordered_set>
52 #include <cassert>
53 #include <iostream>
54 using namespace std;
55
56 class Solution {
57 public:
58     string alienOrder(vector<string>& words) {
59         unordered_map<char, unordered_set<char>>> adyacencia; // letra ->
60             letras que van después
61         unordered_map<char, int> gradoEntrada; // cuántas
62             letras deben ir antes
63
64         // Registramos todas las letras que aparecen (con grado de entrada 0
65         // por defecto), para no perder letras aisladas sin restricciones.
66         for (const string& palabra : words)
67             for (char c : palabra) {

```

```

66         adyacencia[c];
67         gradoEntrada[c] = 0;
68     }
69
70     // Comparamos cada par de palabras adyacentes para deducir aristas.
71     for (size_t i = 0; i + 1 < words.size(); ++i) {
72         const string& w1 = words[i];
73         const string& w2 = words[i + 1];
74         size_t minLen = min(w1.size(), w2.size());
75         bool difieren = false;
76         for (size_t j = 0; j < minLen; ++j) {
77             if (w1[j] != w2[j]) {
78                 if (!adyacencia[w1[j]].count(w2[j])) {
79                     adyacencia[w1[j]].insert(w2[j]);
80                     ++gradoEntrada[w2[j]];
81                 }
82                 difieren = true;
83                 break;
84             }
85         }
86         // Prefijo inválido: w1 es más larga que w2 pero coinciden en todo
87         // el prefijo común => w2 debería haber ido antes que w1.
88         if (!difieren && w1.size() > w2.size()) return "";
89     }
90
91     // Algoritmo de Kahn: arrancamos con las letras sin dependencias.
92     queue<char> cola;
93     for (auto& [letra, grado] : gradoEntrada)
94         if (grado == 0) cola.push(letra);
95
96     string resultado;
97     while (!cola.empty()) {
98         char actual = cola.front(); cola.pop();
99         resultado += actual;
100        for (char siguiente : adyacencia[actual]) {
101            if (--gradoEntrada[siguiente] == 0) cola.push(siguiente);
102        }
103    }
104
105    // Si no procesamos todas las letras, había un ciclo => imposible.
106    if (resultado.size() != gradoEntrada.size()) return "";
107    return resultado;
108 }
109 };
110
111 // -----
112 // Verifica que "resultado" sea un orden de letras compatible con "words":
113 debe
114 // contener exactamente el mismo conjunto de letras (sin repetidas) y respetar
115 // cada arista deducida (letra c1 antes que c2). Como el orden topológico no
116 es
117 // único cuando hay letras sin relación entre sí, esta función permite
118 // comparar sin depender de un string fijo.
119 bool ordenValido(const string& resultado, const vector<string>& words) {

```

```

118 unordered_set<char> letras;
119 for (const string& palabra : words)
120     for (char c : palabra) letras.insert(c);
121
122 if (resultado.size() != letras.size()) return false;
123
124 unordered_map<char, int> posicion;
125 for (int i = 0; i < (int)resultado.size(); ++i) {
126     char c = resultado[i];
127     if (!letras.count(c) || posicion.count(c)) return false; // letra
128         repetida o ajena
129     posicion[c] = i;
130 }
131
132 for (size_t i = 0; i + 1 < words.size(); ++i) {
133     const string& w1 = words[i];
134     const string& w2 = words[i + 1];
135     size_t minLen = min(w1.size(), w2.size());
136     bool difieren = false;
137     for (size_t j = 0; j < minLen; ++j) {
138         if (w1[j] != w2[j]) {
139             if (posicion[w1[j]] >= posicion[w2[j]]) return false; //
140                 arista violada
141             difieren = true;
142             break;
143         }
144     }
145     if (!difieren && w1.size() > w2.size()) return false; // prefijo invá
146         lido
147 }
148 return true;
149 }
150
151 int main() {
152     Solution s;
153
154     // Ejemplo clásico: las aristas forman una cadena w->e->r->t->f, así que
155     // el
156     // orden topológico es único y podemos comparar directo contra "wertf".
157     vector<string> w1 = {"wrt", "wrf", "er", "ett", "rftt"};
158     assert(s.alienOrder(w1) == "wertf");
159     assert(ordenValido(s.alienOrder(w1), w1));
160
161     // Prefijo inválido: "abc" es más larga que "ab" pero "ab" es su prefijo,
162     // eso viola el orden alfabético => "".
163     vector<string> w2 = {"abc", "ab"};
164     assert(s.alienOrder(w2) == "");
165
166     // Ciclo de 2 letras: hacen falta 3 palabras para que dos comparaciones
167     // consecutivas den las dos aristas opuestas. "z","x" da z->x; "x","z" da
168     // x->z => ciclo z<->x, imposible (caso clásico de LeetCode).
169     vector<string> w3 = {"z", "x", "z"};
170     assert(s.alienOrder(w3) == "");

```

```
168 // Ambiguo: solo se deduce b->d; la letra c no tiene ninguna relación con
169 // las demás, así que hay varios órdenes válidos (usamos el verificador).
170 vector<string> w4 = {"ab", "adc"};
171 string r4 = s.alienOrder(w4);
172 assert(ordenaValido(r4, w4));
173
174 // Una sola palabra: no hay pares que comparar, cualquier letra presente
175 // debe aparecer y no hay aristas que respetar.
176 vector<string> w5 = {"z"};
177 assert(s.alienOrder(w5) == "z");
178
179 // Ciclo más largo escondido entre tres letras: a->b (de "ab","bc"),
180 // b->c (de "bc","ca") y c->a (de "ca","a") cierran el ciclo a->b->c->a.
181 vector<string> w6 = {"ab", "bc", "ca", "a"};
182 assert(s.alienOrder(w6) == "");
183
184 cout << "269 Alien Dictionary: todas las pruebas pasaron.\n";
185 return 0;
186 }
```

Complejidad: tiempo $O(C)$, espacio $O(1)$.

donde C es la suma de las longitudes de todas las palabras (el recorrido para comparar palabras adyacentes domina sobre el topológico, que corre sobre a lo más 26 letras).

Notas de entrevista (Amazon). Es primo cercano de Course Schedule (207/210): mismo algoritmo de Kahn, pero aquí el reto está en construir correctamente el grafo a partir de las palabras. Los dos errores más comunes: (1) olvidar el caso de prefijo inválido y (2) tomar más de un carácter distinto por par de palabras (solo el *primero* importa; el resto no dice nada). Recuerda también que el orden topológico casi nunca es único — cualquier letra sin restricciones puede ir en varias posiciones — así que en una entrevista conviene aclarar que basta con devolver *un* orden válido, no “el” orden.

127. Word Ladder

(H)

Código fuente: `src/01-grafos/127-word-ladder.cpp`▷ Ver animación · [local](#) | [web](#)

Dadas `beginWord`, `endWord` y una lista `wordList`, buscamos la longitud de la secuencia de transformación *más corta* de `beginWord` a `endWord`, donde cada paso cambia exactamente una letra y cada palabra intermedia debe existir en `wordList`. La longitud cuenta las palabras (incluyendo `begin` y `end`); si no existe tal secuencia, o `endWord` no está en `wordList`, devolvemos 0.

Intuición. Cada palabra es un nodo de un grafo implícito: hay arista entre dos palabras si difieren en exactamente una letra. Buscar la transformación más corta es buscar el camino más corto en un grafo no ponderado, y eso es justo lo que resuelve un BFS por niveles: cada nivel del BFS corresponde a un paso de transformación.

```

      hit      dot  lot
      |        |   |
      hot      dog  log          cog

```

Solución. No construimos el grafo explícitamente porque sería carísimo con muchas palabras: en su lugar generamos los vecinos de una palabra “sobre la marcha”. Para cada posición probamos las 26 letras del alfabeto distintas a la original y comprobamos si la palabra resultante está en el conjunto de `wordList` (un `unordered_set` da pertenencia en $O(1)$). El BFS avanza por capas (`cola.size()` al inicio de cada nivel) y marcamos palabras visitadas para no reprocesarlas.

```

1  //
2  // =====
3  //
4  // 127. Word Ladder (Hard)
5  // -----
6  //
7  // Dadas beginWord, endWord y una lista wordList, encuentra la longitud de la
8  // secuencia de transformación MÁS CORTA de beginWord a endWord, tal que:
9  // - cada paso cambia exactamente UNA letra,
10 // - cada palabra intermedia resultante debe existir en wordList.
11 // La longitud cuenta las palabras (incluyendo begin y end). Si no existe tal
12 // secuencia (o endWord no está en wordList), devuelve 0.
13 //
14 // Ejemplo:
15 // beginWord = "hit", endWord = "cog"
16 // wordList = ["hot", "dot", "dog", "lot", "log", "cog"]
17 //
18 // hit -> hot -> dot -> dog -> cog (longitud 5)
19 //
20 // Diagrama del grafo implícito (arista si dos palabras difieren en 1 letra):
21 //
22 //      hit
23 //      |
24 //      hot
25 //      / \
26 //     dot  lot
27 //     |    |
28 //     dog  log
29 //          |
30 //          cog

```

```

25 //      dog  log
26 //      \  /
27 //      cog
28 //
29 // Idea: cada palabra es un nodo; hay arista entre dos palabras si difieren en
30 // exactamente una letra. Buscamos el camino más corto de beginWord a endWord
31 // en ese grafo => BFS por niveles (no ponderado). Cada nivel del BFS
32 // corresponde a un paso de transformación.
33 //
34 // Para no construir el grafo explícitamente (sería carísimo), generamos los
35 // vecinos de una palabra "sobre la marcha": para cada posición, probamos las
36 // 26 letras del alfabeto distintas a la original y verificamos si el
37 // resultado está en el conjunto de wordList (O(1) con unordered_set).
38 //
39 // Complejidad: tiempo O(N * L^2 * 26), espacio O(N * L),
40 //              donde N = número de palabras en wordList, L = longitud de
41 //              cada palabra (todas tienen la misma longitud).
42 //
=====
43 #include <vector>
44 #include <string>
45 #include <queue>
46 #include <unordered_set>
47 #include <cassert>
48 #include <iostream>
49 using namespace std;
50
51 class Solution {
52 public:
53     int ladderLength(string beginWord, string endWord, vector<string>&
wordList) {
54         unordered_set<string> diccionario(wordList.begin(), wordList.end());
55         if (diccionario.find(endWord) == diccionario.end()) return 0;
56
57         queue<string> cola;           // BFS por niveles
58         unordered_set<string> visitadas;
59         cola.push(beginWord);
60         visitadas.insert(beginWord);
61
62         int nivel = 1; // beginWord ya cuenta como la primera palabra
63         while (!cola.empty()) {
64             int capa = cola.size(); // palabras en el nivel actual
65             for (int i = 0; i < capa; ++i) {
66                 string palabra = cola.front(); cola.pop();
67                 if (palabra == endWord) return nivel;
68
69                 // Generamos todos los vecinos: cambiar cada posición a cada
70                 // letra de la 'a' a la 'z' y ver si el resultado es válido.
71                 for (size_t pos = 0; pos < palabra.size(); ++pos) {
72                     char original = palabra[pos];
73                     for (char c = 'a'; c <= 'z'; ++c) {
74                         if (c == original) continue;
75                         palabra[pos] = c;

```

```

76         if (diccionario.count(palabra) && !visitadas.count(
77             palabra)) {
78             visitadas.insert(palabra);
79             cola.push(palabra);
80         }
81         palabra[pos] = original;    // restauramos antes de la
            siguiente posición
82     }
83 }
84 ++nivel;    // pasamos al siguiente nivel (un paso más de
            transformación)
85 }
86 return 0;    // se agotó el BFS sin alcanzar endWord => imposible
87 }
88 };
89
90 int main() {
91     Solution s;
92
93     // Caso clásico: hit -> hot -> dot -> dog -> cog (longitud 5).
94     vector<string> w1 = {"hot", "dot", "dog", "lot", "log", "cog"};
95     assert(s.ladderLength("hit", "cog", w1) == 5);
96
97     // endWord no está en wordList => imposible.
98     vector<string> w2 = {"hot", "dot", "dog", "lot", "log"};
99     assert(s.ladderLength("hit", "cog", w2) == 0);
100
101     // Camino más corto entre dos palabras intermedias del mismo diccionario:
102     // hot -> dot -> dog (longitud 3). La ruta hot->lot->log->dog es más larga
103     .
104     vector<string> w3 = {"hot", "dot", "dog", "lot", "log", "cog"};
105     assert(s.ladderLength("hot", "dog", w3) == 3);
106
107     // beginWord y endWord difieren en una sola letra y endWord está en la
108     // lista => transformación directa, longitud 2.
109     vector<string> w4 = {"hot"};
110     assert(s.ladderLength("hit", "hot", w4) == 2);
111
112     cout << "127 Word Ladder: todas las pruebas pasaron.\n";
113     return 0;
114 }

```

Complejidad: tiempo $O(N \cdot L^2 \cdot 26)$, espacio $O(N \cdot L)$.

donde N es el número de palabras en `wordList` y L la longitud de cada palabra.

Notas de entrevista (Amazon). El primer chequeo obligatorio es verificar que `endWord` esté en `wordList`; si no, la respuesta es 0 sin hacer ningún BFS. El error típico es generar vecinos comparando contra cada palabra de la lista letra por letra ($O(N \cdot L)$ por palabra), en vez de generar las variaciones directamente y consultarlas en un conjunto ($O(L \cdot 26)$ por palabra), lo cual escala mucho mejor cuando `wordList` es grande. También vale la pena mencionar Bidirectional BFS (buscar simultáneamente desde `beginWord` y `endWord`) como optimización adicional para reducir el factor de ramificación efectivo.

126. Word Ladder II

(H)

Código fuente: [src/01-grafos/126-word-ladder-ii.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dadas `beginWord`, `endWord` y una lista `wordList`, cada palabra puede transformarse en otra cambiando exactamente una letra, siempre que el resultado exista en `wordList`. A diferencia del problema 127 (Word Ladder), donde sólo pedían la *longitud* del camino más corto, aquí hay que devolver **todas** las secuencias de transformación más cortas de `beginWord` a `endWord`. Si `endWord` es inalcanzable, se devuelve el vector vacío.

Intuición. Primero resolvemos el 127 mentalmente: BFS por niveles nos da la distancia mínima. La diferencia aquí es que, mientras hacemos ese BFS, en vez de quedarnos sólo con la distancia, anotamos de *qué palabra viene cada palabra nueva* (su “padre”). Como puede haber varios padres válidos para una misma palabra en el mismo nivel, ese mapa de padres es en realidad un grafo dirigido acíclico (DAG) por capas. Una vez construido, basta con recorrerlo hacia atrás desde `endWord` hasta `beginWord` para obtener, por construcción, únicamente caminos de longitud mínima.

```
nivel 0: hit
nivel 1: hot
nivel 2: dot  lot
nivel 3: dog  log
nivel 4: cog
```

Con `wordList = ["hot","dot","dog","lot","log","cog"]` hay dos caminos más cortos, ambos de longitud 5: `hit-hot-dot-dog-cog` y `hit-hot-lot-log-cog`, porque tanto `dog` como `log` tienen a `cog` como hija en el mismo nivel.

Solución. Dos fases:

1. **BFS por niveles** desde `beginWord`. En cada nivel, para cada palabra probamos las 26 letras en cada posición; si el resultado está en el diccionario lo mandamos al siguiente nivel y registramos la palabra actual como uno de sus padres. Al terminar de procesar un nivel completo, borramos esas palabras del diccionario para que un nivel futuro no pueda *regresar* a ellas (eso violaría la propiedad de camino más corto). Si `endWord` aparece en un nivel, terminamos de procesar ese nivel (puede tener varios padres) y detenemos el BFS.
2. **DFS con backtracking** desde `endWord`, siguiendo todos los padres registrados hasta llegar a `beginWord`, acumulando cada camino completo. Cada camino se arma en reversa y se invierte al final.

```
1 //
2 // =====
3 //
4 // 126. Word Ladder II (Hard)
5 //
6 // -----
7 //
8 // Dadas beginWord, endWord y una lista wordList, cada palabra (empezando por
9 // beginWord) puede transformarse en otra cambiando exactamente UNA letra,
```



```

58 //          devolverlos TODOS),
59 //          espacio  $O(N * L)$  para el diccionario y el mapa de padres.
60 //
=====
61 #include <vector>
62 #include <string>
63 #include <unordered_set>
64 #include <unordered_map>
65 #include <algorithm>
66 #include <cassert>
67 #include <iostream>
68 using namespace std;
69
70 class Solution {
71 public:
72     vector<vector<string>> findLadders(string beginWord, string endWord,
73                                     vector<string>& wordList) {
74         unordered_set<string> diccionario(wordList.begin(), wordList.end());
75         vector<vector<string>> resultado;
76         if (!diccionario.count(endWord)) return resultado; // endWord
77             inalcanzable
78
79         unordered_map<string, vector<string>> padres; // palabra ->
80             predecesores
81
82         unordered_set<string> nivelActual{beginWord};
83         bool encontrado = false;
84
85         while (!nivelActual.empty() && !encontrado) {
86             // Quitamos del diccionario las palabras de este nivel: ya tienen
87             // fijada su distancia mínima, y no queremos que un nivel futuro
88             // "regrese" a ellas (dejaría de ser el camino más corto).
89             for (const string& palabra : nivelActual) diccionario.erase(
90                 palabra);
91
92             unordered_set<string> siguienteNivel;
93             for (const string& palabra : nivelActual) {
94                 string variante = palabra;
95                 for (size_t pos = 0; pos < variante.size(); ++pos) {
96                     char original = variante[pos];
97                     for (char c = 'a'; c <= 'z'; ++c) {
98                         if (c == original) continue;
99                         variante[pos] = c;
100                         if (diccionario.count(variante)) {
101                             siguienteNivel.insert(variante);
102                             padres[variante].push_back(palabra);
103                             if (variante == endWord) encontrado = true;
104                         }
105                     }
106                 }
107                 variante[pos] = original; // restauramos antes de la
108                     siguiente posición
109             }
110             nivelActual = siguienteNivel;

```

```

106     }
107
108     if (!encontrado) return resultado; // BFS se agotó sin alcanzar
        endWord
109
110     vector<string> camino{endWord};
111     reconstruir(endWord, beginWord, padres, camino, resultado);
112     for (auto& c : resultado) reverse(c.begin(), c.end());
113     return resultado;
114 }
115
116 private:
117     // DFS con backtracking: desde 'palabra' retrocede por todos sus padres
118     // hasta llegar a beginWord, acumulando cada camino completo (en reversa,
119     // de endWord a beginWord) en resultado.
120     void reconstruir(const string& palabra, const string& beginWord,
121                     unordered_map<string, vector<string>>& padres,
122                     vector<string>& camino, vector<vector<string>>&
                        resultado) {
123         if (palabra == beginWord) {
124             resultado.push_back(camino);
125             return;
126         }
127         for (const string& padre : padres[palabra]) {
128             camino.push_back(padre);
129             reconstruir(padre, beginWord, padres, camino, resultado);
130             camino.pop_back();
131         }
132     }
133 };
134
135 int main() {
136     Solution s;
137
138     // Caso clásico: dos caminos más cortos, ambos de longitud 5.
139     {
140         vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog"};
141         auto res = s.findLadders("hit", "cog", wordList);
142         sort(res.begin(), res.end());
143         vector<vector<string>> esperado = {
144             {"hit", "hot", "dot", "dog", "cog"},
145             {"hit", "hot", "lot", "log", "cog"},
146         };
147         sort(esperado.begin(), esperado.end());
148         assert(res == esperado);
149     }
150
151     // Sin solución: endWord no está en wordList => vector vacío.
152     {
153         vector<string> wordList = {"hot", "dot", "dog", "lot", "log"};
154         auto res = s.findLadders("hit", "cog", wordList);
155         assert(res.empty());
156     }
157

```

```

158 // Un único camino más corto (sin bifurcaciones).
159 {
160     vector<string> wordList = {"dot", "dog"};
161     auto res = s.findLadders("hot", "dog", wordList);
162     vector<vector<string>> esperado = {{ "hot", "dot", "dog" }};
163     assert(res == esperado);
164 }
165
166 // beginWord no pertenece a wordList (sólo endWord y las intermedias
167 // deben estar); aquí sólo existe un camino más corto de longitud 3.
168 {
169     vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog", "
        hog"};
170     auto res = s.findLadders("hig", "cog", wordList);
171     vector<vector<string>> esperado = {{ "hig", "hog", "cog" }};
172     assert(res == esperado);
173 }
174
175 cout << "126 Word Ladder II: todas las pruebas pasaron.\n";
176 return 0;
177 }

```

Complejidad: tiempo $O(N \cdot L^2 \cdot 26)$ para el BFS más el costo de reconstruir todos los caminos (exponencial en el peor caso, inherente al problema), espacio $O(N \cdot L)$ para el diccionario y el mapa de padres.

Notas de entrevista (Amazon). Este problema suele plantearse como “ya resolviste el 127, ahora dame todos los caminos”; la clave que buscan es que reconozcas que un BFS normal con un solo padre por nodo pierde caminos igualmente cortos, y que hay que permitir múltiples padres por nivel antes de borrar las palabras visitadas. Errores típicos: borrar del diccionario palabra por palabra en vez de por nivel completo (rompe caminos válidos del mismo nivel), o mezclar BFS con DFS en una sola pasada (generar todos los caminos con DFS puro, sin la poda de BFS, es exponencial e inviable). Casos borde a mencionar: `endWord` no está en `wordList` (vacío), `beginWord` no pertenece a `wordList` (es válido, sólo debe empezar la cadena), y un único camino más corto sin bifurcaciones.

261. Graph Valid Tree

(M)

Código fuente: `src/01-grafos/261-graph-valid-tree.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan n nodos etiquetados de 0 a $n - 1$ y una lista de aristas no dirigidas. Queremos saber si ese grafo forma un *árbol* válido.

Intuición. Un árbol es, por definición, un grafo conectado y sin ciclos. Por lo tanto hay que revisar ambas condiciones. La observación clave es aritmética: un árbol con n nodos tiene **exactamente** $n - 1$ aristas. Si hay menos, es imposible que todo esté conectado; si hay más, forzosamente sobra alguna arista que cierra un ciclo. Así que primero descartamos por conteo, y luego sólo falta comprobar que, con exactamente $n - 1$ aristas, ninguna de ellas cierre un ciclo (si no hay ciclos y el conteo de aristas es $n - 1$, la conectividad total queda garantizada).

```

      árbol válido (n = 5)
          0
        1 2 3
          4
aristas: [0,1] [0,2] [0,3] [1,4] → 4 = n-1 → true

```

Union-Find (patrón nuevo). Para detectar ciclos de forma eficiente usamos *Union-Find* (o *Disjoint Set Union*, DSU), una estructura que no habíamos usado antes en este libro:

- Cada nodo arranca siendo su propio representante (su propio conjunto), guardado en un arreglo **padre**.
- **find(x)** sigue los punteros **padre** hasta llegar a la raíz del conjunto de **x**. Con **compresión de caminos**, cada nodo visitado en el trayecto se re-liga directamente a la raíz, así que consultas futuras son casi instantáneas.
- **unite(x, y)** busca las raíces de **x** y **y**. Si ya comparten raíz, **x** y **y** ya estaban conectados por otro camino distinto de esta arista, y añadirla formaría un **ciclo**: devolvemos **false**. Si las raíces son distintas, fusionamos los conjuntos usando **unión por rango**: la raíz del árbol de menor altura cuelga de la raíz del árbol más alto, para que los árboles del DSU se mantengan planos y las operaciones sigan siendo rápidas.

Solución. Primero descartamos si el número de aristas no es $n - 1$. Después recorremos cada arista llamando a **unite**; si alguna llamada detecta que ambos extremos ya comparten raíz, hay un ciclo y regresamos **false** de inmediato. Si procesamos todas las aristas sin encontrar ciclos, el grafo es un árbol válido.

```

1 //
  =====
2 // 261. Graph Valid Tree (Medium)
3 //
  -----
4 // Dados n nodos etiquetados de 0 a n-1 y una lista de aristas no dirigidas,
5 // determina si forman un árbol válido.

```



```

58 #include <cassert>
59 #include <iostream>
60 using namespace std;
61
62 class Solution {
63 public:
64     bool validTree(int n, vector<vector<int>>& edges) {
65         // Un árbol con n nodos tiene exactamente n-1 aristas.
66         if ((int)edges.size() != n - 1) return false;
67
68         padre.resize(n);
69         rango.assign(n, 0);
70         for (int i = 0; i < n; ++i) padre[i] = i;    // cada nodo es su propia
            raíz
71
72         for (auto& e : edges) {
73             int a = e[0], b = e[1];
74             if (!unite(a, b)) return false;    // ya estaban conectados =>
                ciclo
75         }
76         return true;    // n-1 aristas y sin ciclos => un solo árbol conectado
77     }
78
79 private:
80     vector<int> padre;
81     vector<int> rango;
82
83     // Encuentra la raíz del conjunto de x, comprimiendo el camino recorrido.
84     int find(int x) {
85         if (padre[x] != x) padre[x] = find(padre[x]);    // compresión de
            caminos
86         return padre[x];
87     }
88
89     // Une los conjuntos de a y b. Devuelve false si ya estaban en el mismo
90     // conjunto (es decir, unir esta arista formaría un ciclo).
91     bool unite(int a, int b) {
92         int ra = find(a), rb = find(b);
93         if (ra == rb) return false;    // mismo conjunto => ciclo
94
95         // Unión por rango: la raíz de menor rango cuelga de la de mayor rango
96
97         if (rango[ra] < rango[rb]) swap(ra, rb);
98         padre[rb] = ra;
99         if (rango[ra] == rango[rb]) ++rango[ra];
100        return true;
101    };
102
103    int main() {
104        Solution s;
105
106        // Árbol válido: 5 nodos, 4 aristas (n-1), todo conectado, sin ciclos.
107        vector<vector<int>> e1 = {{0,1},{0,2},{0,3},{1,4}};

```

```

108     assert(s.validTree(5, e1) == true);
109
110     // Con ciclo: 0-1-2-0 forma un ciclo; además 3-4 queda desconectado del
111     // resto. La detección de ciclo en unite() basta para regresar false.
112     vector<vector<int>> e2 = {{0,1},{1,2},{2,0},{3,4}};
113     assert(s.validTree(5, e2) == false);
114
115     // Desconectado: dos componentes separados {0,1} y {2,3}; sólo 2 aristas
116     // pero se necesitan 3 (n-1 con n=4) para siquiera intentar ser árbol.
117     vector<vector<int>> e3 = {{0,1},{2,3}};
118     assert(s.validTree(4, e3) == false);
119
120     // Un solo nodo sin aristas: componente trivial, es un árbol válido.
121     vector<vector<int>> e4 = {};
122     assert(s.validTree(1, e4) == true);
123
124     cout << "261 Graph Valid Tree: todas las pruebas pasaron.\n";
125     return 0;
126 }

```

Complejidad: tiempo casi $O(n + e \cdot \alpha(n))$, espacio $O(n)$.

Notas de entrevista (Amazon). Menciona primero el atajo de conteo ($n - 1$ aristas) porque descarta la mayoría de los casos triviales sin tocar el DSU. Explica por qué la unión por rango y la compresión de caminos son necesarias: sin ellas, `find` podría degenerar a $O(n)$ por llamada en el peor caso (una cadena larga), mientras que con ambas optimizaciones la complejidad amortizada es prácticamente constante (α es la inversa de Ackermann). Casos borde a mencionar: $n = 1$ sin aristas (es un árbol trivial válido), grafo con ciclo, grafo desconectado en varios componentes, y aristas de más o de menos respecto a $n - 1$.

323. Number of Connected Components

(M)

Código fuente: [src/01-grafos/323-connected-components.cpp](#)▷ Ver animación · [local](#) | [web](#)

Hay n nodos, numerados de 0 a $n - 1$, y una lista `edges` de pares $[a, b]$ que representan aristas **no dirigidas**. Queremos contar cuántos componentes conexos tiene el grafo.

Intuición. Empezamos suponiendo lo peor: que cada nodo vive solo, aislado, formando su propio componente. Eso nos da n componentes de entrada. Cada arista es una oportunidad de fusión: si conecta dos nodos que ya viven en el mismo componente, no cambia nada (la arista es “redundante” para efectos de contar componentes); pero si conecta dos nodos de componentes distintos, esos dos componentes se convierten en uno solo y el contador baja en 1. Al final de procesar todas las aristas, el contador tiene la respuesta.

$n = 5$, `edges` = $[[0, 1], [1, 2], [3, 4]]$
 0 --- 1 --- 2 3 --- 4
 2 componentes: $\{0, 1, 2\}$ y $\{3, 4\}$

Solución (Union-Find / DSU). La estructura ideal para este “ir fusionando conjuntos” es **Union-Find** (Disjoint Set Union): cada nodo apunta a un padre, y la *raíz* de un nodo identifica a qué componente pertenece. Dos operaciones:

- **find(x)**: sigue los punteros de padre hasta la raíz del conjunto de x . Aplicamos *compresión de caminos*: cada nodo visitado en el camino se re-apunta directamente a la raíz, para que futuras búsquedas sean casi instantáneas.
- **unir(a, b)**: si **find(a)** y **find(b)** son distintas, cuelga una raíz de la otra usando *unión por rango* (la raíz de árbol más pequeño cuelga de la más grande, para mantener los árboles bajos) y decrementa el contador de componentes.

Inicializamos cada nodo como su propio padre (n componentes) y recorremos las aristas una vez, uniendo cuando sea necesario.

```

1 //
  =====
2 // 323. Number of Connected Components in an Undirected Graph (Medium)
3 //
  =====
4 // Hay n nodos, numerados de 0 a n-1, y una lista edges de pares [a, b] que
5 // representan aristas NO dirigidas. Cuenta cuántos componentes conexos tiene
6 // el grafo.
7 //
8 // Ejemplo 1:
9 //   n = 5, edges = [[0,1],[1,2],[3,4]]
10 //
11 //       0 --- 1 --- 2           3 --- 4
12 //
13 //   Hay 2 componentes: {0,1,2} y {3,4}.
14 //
```

```

15 // Ejemplo 2:
16 //   n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
17 //
18 //   0 --- 1 --- 2 --- 3 --- 4
19 //
20 //   Todos quedan conectados en cadena: 1 solo componente.
21 //
22 // Idea (Union-Find / DSU, Disjoint Set Union):
23 // - Empezamos suponiendo que cada nodo es su propio componente: n
24 //   componentes en total, cada uno representado por su propia raíz.
25 // - Por cada arista [a, b], buscamos la raíz de a y la raíz de b (find).
26 //   Si ya comparten raíz, la arista no cambia nada (ya estaban en el mismo
27 //   componente). Si tienen raíces distintas, unimos los dos conjuntos
28 //   (union) y decrementamos el contador de componentes en 1: dos
29 //   componentes se acaban de fusionar en uno solo.
30 // - Al terminar de procesar todas las aristas, el contador tiene la
31 //   respuesta final.
32 // - Usamos unión por rango y compresión de caminos para que find/union
33 //   sean casi  $O(1)$  amortizado ( $O(\alpha(n))$ ), la función inversa de Ackermann).
34 //
35 // Diagrama de una unión:
36 //
37 //   componente(a): [a]--[x]--[raiz_a]           componente(b): [b]--[raiz_b]
38 //
39 //   find(a) = raiz_a, find(b) = raiz_b, son distintas => unimos:
40 //
41 //   raiz_b pasa a colgar de raiz_a (o viceversa, según el rango) y
42 //   --componentes.
43 //
44 // Nota: es hermano del problema 261 "Graph Valid Tree" (mismo DSU: un grafo
45 // es árbol si y solo si tiene exactamente  $n-1$  aristas y termina con un solo
46 // componente, es decir, ninguna arista intenta unir un nodo consigo mismo
47 // en el mismo componente) y del patrón de "contar islas" (200, 695): en
48 // vez de recorrer una cuadrícula con DFS/BFS, aquí contamos componentes de
49 // un grafo general dado por lista de aristas. También se puede resolver con
50 // DFS/BFS marcando visitados y contando cuántas veces arrancamos una
51 // exploración nueva, igual que en Number of Islands.
52 //
53 // Complejidad: tiempo  $O(n + e \cdot \alpha(n))$  (n nodos, e aristas),
54 // espacio  $O(n)$  por los arreglos padre y rango.
55 //
=====

56 #include <vector>
57 #include <cassert>
58 #include <iostream>
59 using namespace std;
60
61 class Solution {
62 public:
63     int countComponents(int n, vector<vector<int>>& edges) {
64         padre.resize(n);
65         rango.assign(n, 0);
66         for (int i = 0; i < n; ++i) padre[i] = i; // cada nodo es su propia

```

```

    raíz
67
68     int componentes = n;    // al inicio, cada nodo es un componente
    aislado
69     for (auto& arista : edges) {
70         int a = arista[0], b = arista[1];
71         int raizA = find(a);
72         int raizB = find(b);
73         if (raizA != raizB) {
74             unir(raizA, raizB);
75             --componentes;    // dos componentes se fusionaron en uno
76         }
77     }
78     return componentes;
79 }
80
81 private:
82     vector<int> padre;
83     vector<int> rango;
84
85     // Encuentra la raíz del conjunto de x, comprimiendo el camino recorrido.
86     int find(int x) {
87         if (padre[x] != x) padre[x] = find(padre[x]);    // compresión de
            caminos
88         return padre[x];
89     }
90
91     // Une los conjuntos de dos raíces distintas usando unión por rango.
92     void unir(int raizA, int raizB) {
93         if (rango[raizA] < rango[raizB]) swap(raizA, raizB);
94         padre[raizB] = raizA;    // la raíz de menor rango cuelga
            de la mayor
95         if (rango[raizA] == rango[raizB]) ++rango[raizA];
96     }
97 };
98
99 int main() {
100     Solution s;
101
102     // Dos componentes: {0,1,2} en cadena y {3,4} aparte.
103     vector<vector<int>> e1 = {{0,1},{1,2},{3,4}};
104     assert(s.countComponents(5, e1) == 2);
105
106     // Cadena que conecta todo: un solo componente.
107     vector<vector<int>> e2 = {{0,1},{1,2},{2,3},{3,4}};
108     assert(s.countComponents(5, e2) == 1);
109
110     // Sin aristas: cada nodo es su propio componente.
111     vector<vector<int>> e3 = {};
112     assert(s.countComponents(4, e3) == 4);
113
114     // Un solo nodo, sin aristas: un componente.
115     vector<vector<int>> e4 = {};
116     assert(s.countComponents(1, e4) == 1);

```

```
117
118     cout << "323 Connected Components: todas las pruebas pasaron.\n";
119     return 0;
120 }
```

Complejidad: tiempo $O(n + e \cdot \alpha(n))$, espacio $O(n)$.

donde $\alpha(n)$ es la función inversa de Ackermann (crece tan lento que para cualquier n práctico es prácticamente una constante), y e es el número de aristas.

Notas de entrevista (Amazon). Este problema es el ejemplo canónico de Union-Find y es hermano directo del **261. Graph Valid Tree**: ahí, en vez de contar componentes, se pregunta si el grafo termina siendo un único árbol (exactamente $n - 1$ aristas y ninguna arista que una dos nodos ya en el mismo componente, es decir, ningún ciclo). También es primo del patrón de “contar islas” (200, 695): en lugar de recorrer una cuadrícula con DFS/BFS marcando visitados, aquí el grafo viene dado explícitamente como lista de aristas, pero la idea de fondo es la misma: contar cuántos grupos conectados existen. De hecho, este problema también se puede resolver con DFS o BFS clásico, construyendo una lista de adyacencia y contando cuántas veces arrancamos una exploración desde un nodo no visitado. Menciona ambos enfoques en la entrevista: DFS/BFS es más directo de explicar, pero Union-Find es más eficiente cuando las aristas llegan de forma incremental (por ejemplo, si el entrevistador pregunta un seguimiento tipo “¿y si las aristas se agregan una a una y quieres saber el número de componentes después de cada una?”). Casos borde: sin aristas (cada nodo es su propio componente), un solo nodo, y aristas repetidas o que no cambian el número de componentes.

128. Longest Consecutive Sequence

(M)

Código fuente: [src/01-grafos/128-longest-consecutive.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo de enteros sin ordenar. Queremos la longitud de la secuencia de enteros *consecutivos* más larga (por ejemplo 3, 4, 5, 6), sin importar el orden en que aparezcan en el arreglo. El algoritmo debe correr en $O(n)$.

Intuición. Aunque no hay un grafo explícito, el patrón es el mismo que recorrer componentes: cada secuencia consecutiva es como una “isla” de números, y sólo queremos visitar cada isla una vez, empezando desde su extremo más pequeño. Metemos todos los números en un `unordered_set` para tener búsquedas en $O(1)$. Luego, para cada número, nos preguntamos: ¿soy el *inicio* de mi secuencia? Un número `num` es inicio si `num - 1` no está en el conjunto. Sólo desde ahí contamos hacia arriba (`num+1`, `num+2`, ...) mientras existan en el conjunto.

```

nums = {100, 4, 200, 1, 3, 2}
100 → ¿existe 99? no → inicio, longitud 1
4 → ¿existe 3? sí → no es inicio, se ignora
200 → ¿existe 199? no → inicio, longitud 1
1 → ¿existe 0? no → inicio, cuenta 1,2,3,4 → longitud 4
3, 2 → no son inicio, se ignoran

```

El máximo es 4, por la secuencia 1, 2, 3, 4.

¿Por qué es $O(n)$ y no $O(n^2)$? Si contáramos hacia arriba desde *cada* número sin filtrar, una secuencia larga dispararía un conteo redundante desde cada uno de sus elementos: el 1 recorrería 1, 2, 3, 4 (4 pasos), pero luego el 2 recorrería 2, 3, 4 (3 pasos), el 3 recorrería 3, 4 (2 pasos), etc. Para una secuencia de tamaño k eso suma $O(k^2)$ pasos, y en el peor caso ($k = n$) el algoritmo completo sería $O(n^2)$.

El truco de exigir que `num - 1` no esté en el conjunto garantiza que *sólo el mínimo de cada secuencia* dispare un conteo, y ese conteo recorre la secuencia completa exactamente una vez. Como las secuencias no se traslapan (cada número pertenece a una sola secuencia consecutiva), la suma de todos los pasos de conteo, sobre *todas* las secuencias, es a lo sumo n . En otras palabras: cada número se visita cuando mucho dos veces en todo el algoritmo: una vez para revisar si es inicio, y a lo sumo una vez más cuando se cuenta como parte de la secuencia de su verdadero inicio. Eso es exactamente lo que mantiene el algoritmo en $O(n)$.

Solución. Construimos el `unordered_set`, y para cada número que sea inicio de secuencia (su antecesor no está en el conjunto) contamos hacia arriba mientras el siguiente exista. Nos quedamos con la longitud máxima encontrada. El código completo (abre el archivo con clic en el título):

```

1 //
  =====
2 // 128. Longest Consecutive Sequence (Medium)
3 //
  -----
4 // Dado un arreglo de enteros sin ordenar, devuelve la longitud de la

```

```

5 // secuencia de enteros CONSECUTIVOS más larga (por ejemplo 3,4,5,6). El
6 // arreglo no está ordenado y el algoritmo debe correr en  $O(n)$ .
7 //
8 // Ejemplo:
9 //   nums = [100, 4, 200, 1, 3, 2]
10 //   La secuencia consecutiva más larga es 1, 2, 3, 4 => longitud 4.
11 //
12 // Diagrama de la idea (metemos todo en un unordered_set):
13 //
14 //   set = { 100, 4, 200, 1, 3, 2 }
15 //
16 //   100 -> ¿existe 99? no  => 100 ES inicio de secuencia. Cuenta: 100 (long
17 //   1)
18 //   4 -> ¿existe 3?  sí  => 4 NO es inicio, se ignora aquí (ya se contará
19 //   cuando lleguemos al verdadero inicio: 1)
20 //   200 -> ¿existe 199? no => 200 ES inicio. Cuenta: 200 (long 1)
21 //   1 -> ¿existe 0?  no  => 1 ES inicio de secuencia. Cuenta hacia arriba:
22 //   1,2,3,4 (existen en el set) -> longitud 4
23 //   3 -> ¿existe 2?  sí  => NO es inicio, se ignora
24 //   2 -> ¿existe 1?  sí  => NO es inicio, se ignora
25 //
26 //   Máximo encontrado: 4
27 //
28 // Idea clave (por qué es  $O(n)$  y no  $O(n^2)$ ): si para CADA número intentáramos
29 // contar hacia arriba su secuencia, cada elemento de una secuencia larga
30 // dispararía un conteo completo desde ahí, y sumaríamos trabajo repetido
31 // (1 dispara conteo de longitud 4, pero 2, 3 y 4 también dispararían sus
32 // propios conteos de longitud 3, 2, 1 =>  $O(n^2)$  en el peor caso, como una
33 // secuencia 1..n completa).
34 //
35 // El truco es filtrar: SOLO contamos hacia arriba cuando el número es un
36 // "inicio de secuencia", es decir, cuando (num - 1) NO está en el set. Así,
37 // cada secuencia consecutiva se cuenta EXACTAMENTE UNA VEZ, empezando desde
38 // su mínimo. Como cada número pertenece a una única secuencia, y esa
39 // secuencia sólo se recorre una vez completa (desde su inicio), la suma total
40 // de pasos de conteo a lo largo de todo el algoritmo es a lo sumo n. Cada
41 // número se visita cuando mucho dos veces: una para meterlo al set / revisar
42 // si es inicio, y otra (a lo sumo) cuando se cuenta como parte de la
43 // secuencia de su verdadero inicio. Por eso el algoritmo completo es  $O(n)$ .
44 //
45 // Complejidad: tiempo  $O(n)$  (cada número se procesa  $O(1)$  veces en promedio
46 // gracias al unordered_set y al filtro de "inicio de secuencia")
47 //
48 //   ,
49 //   espacio  $O(n)$  por el unordered_set.
50 //
51 // =====
52
53 #include <vector>
54 #include <unordered_set>
55 #include <cassert>
56 #include <iostream>
57 using namespace std;
58
59 class Solution {

```

```

55 public:
56     int longestConsecutive(vector<int>& nums) {
57         unordered_set<int> presentes(nums.begin(), nums.end());
58         int mejor = 0;
59
60         for (int num : presentes) {
61             // Sólo arrancamos el conteo si num es el INICIO de su secuencia,
62             // es decir, si num - 1 no pertenece al conjunto. Esto evita
63             // recomtar la misma secuencia varias veces.
64             if (presentes.count(num - 1)) continue;
65
66             int actual = num;
67             int longitud = 1;
68             while (presentes.count(actual + 1)) { // cuenta hacia arriba
69                 ++actual;
70                 ++longitud;
71             }
72             mejor = max(mejor, longitud);
73         }
74         return mejor;
75     }
76 };
77
78 int main() {
79     Solution s;
80
81     vector<int> n1 = {100, 4, 200, 1, 3, 2};
82     assert(s.longestConsecutive(n1) == 4); // secuencia 1,2,3,4
83
84     vector<int> n2 = {0, 3, 7, 2, 5, 8, 4, 6, 0, 1};
85     assert(s.longestConsecutive(n2) == 9); // secuencia 0..8 (el 0 duplicado
86         // no afecta)
87
88     vector<int> vacio = {};
89     assert(s.longestConsecutive(vacio) == 0); // sin elementos, no hay
90         // secuencia
91
92     vector<int> n3 = {1, 2, 2, 3};
93     assert(s.longestConsecutive(n3) == 3); // duplicados no alargan la
94         // secuencia: 1,2,3
95
96     cout << "128 Longest Consecutive: todas las pruebas pasaron.\n";
97     return 0;
98 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

Notas de entrevista (Amazon). El error típico es ordenar el arreglo primero, lo cual da $O(n \log n)$ y no cumple el requisito de $O(n)$ que suele pedirse explícitamente en la entrevista. Enfatiza el filtro de “inicio de secuencia” como la clave para evitar el trabajo repetido: sin él, el algoritmo degrada a $O(n^2)$ en arreglos con una sola secuencia larga. Casos borde a mencionar: arreglo vacío (respuesta 0), duplicados (no alargan la secuencia, por eso usamos un `set` y no un `multiset` o un conteo de frecuencias), y números negativos (el algoritmo funciona igual, no depende del signo).

547. Number of Provinces

(M)

Código fuente: [src/01-grafos/547-number-of-provinces.cpp](#)▷ Ver animación · [local](#) | [web](#)

Hay n ciudades y una matriz de adyacencia `isConnected` de tamaño $n \times n$, donde `isConnected[i][j] == 1` si la ciudad i y la ciudad j están conectadas **directamente** (la matriz es simétrica y siempre `isConnected[i][i] == 1`). Una *provincia* es un grupo de ciudades conectadas directa o indirectamente, sin conexión con ninguna ciudad fuera del grupo. Queremos contar cuántas provincias hay.

Intuición. Esta matriz no es más que otra forma de entregarnos el mismo grafo no dirigido de **323. Number of Connected Components**: en vez de recibir una lista de aristas $[a, b]$, nos dan, para cada par de ciudades (i, j) , si existe o no esa arista. El problema de fondo es idéntico: contar componentes conexos. Basta recorrer la matriz y, cada vez que `isConnected[i][j] == 1` con $i \neq j$, tratar a i y j como dos nodos unidos por una arista.

1	1	0
1	1	0
0	0	1

En esta matriz hay **2** provincias: $\{0, 1\}$ (conectadas directamente) y $\{2\}$ (aislada, solo conectada consigo misma en la diagonal).

Solución (Union-Find / DSU). Igual que en 323, usamos Union-Find: cada ciudad empieza siendo su propia raíz (n provincias). Recorremos únicamente la mitad superior de la matriz (es simétrica, así que revisar $i < j$ basta) y, cuando `isConnected[i][j] == 1`, unimos las raíces de i y j con compresión de caminos y unión por rango; si las raíces eran distintas, dos provincias acaban de fusionarse y decrementamos el contador.

```

1 //
  =====
2 // 547. Number of Provinces (Medium)
3 //
  -----
4 // Hay n ciudades. Nos dan una matriz de adyacencia isConnected de tamaño
5 // n x n, donde isConnected[i][j] == 1 si la ciudad i y la ciudad j están
6 // conectadas DIRECTAMENTE, e isConnected[i][j] == 0 si no lo están (siempre
7 // isConnected[i][i] == 1, la matriz es simétrica). Una "provincia" es un
8 // grupo de ciudades conectadas directa o indirectamente, y que no está
9 // conectado con ninguna ciudad fuera del grupo. Queremos contar cuántas
10 // provincias hay.
11 //
12 // Ejemplo:
13 //     isConnected =
14 //         1 1 0           Ciudad 0 y 1 están conectadas directamente.
15 //         1 1 0           Ciudad 2 no está conectada con nadie más.
16 //         0 0 1
17 //
18 // Hay 2 provincias: {0, 1} y {2}.

```

```

19 //
20 // Idea: esta matriz de adyacencia NO es más que otra forma de darnos el
21 // mismo grafo no dirigido de 323 "Number of Connected Components". En vez de
22 // recibir una lista de aristas [a, b], recibimos, para cada par de ciudades
23 // (i, j), si existe o no la arista. Así que el problema es idéntico: contar
24 // componentes conexos. Usamos Union-Find (DSU): cada ciudad empieza siendo
25 // su propia provincia; cada vez que isConnected[i][j] == 1 con i != j,
26 // unimos las ciudades i y j (si no estaban ya en la misma provincia,
27 // decrementamos el contador de provincias).
28 //
29 // Diagrama de la matriz interpretada como grafo:
30 //
31 //   isConnected[i][j] == 1  <=>  arista (i, j) en el grafo
32 //
33 //       0 --- 1           2 (aislada)
34 //
35 //   (fila/columna 0 y 1 se apuntan entre sí; fila/columna 2 solo se apunta
36 //    a sí misma en la diagonal)
37 //
38 // Complejidad: tiempo  $O(n^2 * \alpha(n))$  (revisamos las  $n^2$  celdas de la
39 // matriz, cada find/unir es casi  $O(1)$  amortizado),
40 // espacio  $O(n)$  por los arreglos padre y rango.
41 //
=====

42 #include <vector>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 class Solution {
48 public:
49     int findCircleNum(vector<vector<int>>& isConnected) {
50         int n = isConnected.size();
51         padre.resize(n);
52         rango.assign(n, 0);
53         for (int i = 0; i < n; ++i) padre[i] = i;    // cada ciudad es su
54             // propia raíz
55
56         int provincias = n;    // al inicio, cada ciudad es una provincia
57             // aislada
58         for (int i = 0; i < n; ++i) {
59             for (int j = i + 1; j < n; ++j) {    // basta la mitad superior (
60                 // matriz simétrica)
61                 if (isConnected[i][j] == 1) {
62                     int raizI = find(i);
63                     int raizJ = find(j);
64                     if (raizI != raizJ) {
65                         unir(raizI, raizJ);
66                         --provincias;    // dos provincias se fusionaron en una
67                     }
68                 }
69             }
70         }
71     }
72 }

```

```

68     return provincias;
69 }
70
71 private:
72     vector<int> padre;
73     vector<int> rango;
74
75     // Encuentra la raíz del conjunto de x, comprimiendo el camino recorrido.
76     int find(int x) {
77         if (padre[x] != x) padre[x] = find(padre[x]); // compresión de
78             caminos
79         return padre[x];
80     }
81
82     // Une los conjuntos de dos raíces distintas usando unión por rango.
83     void unir(int raizA, int raizB) {
84         if (rango[raizA] < rango[raizB]) swap(raizA, raizB);
85         padre[raizB] = raizA; // la raíz de menor rango cuelga
86             de la mayor
87         if (rango[raizA] == rango[raizB]) ++rango[raizA];
88     }
89 };
90
91 int main() {
92     Solution s;
93
94     // Ciudades 0 y 1 conectadas directamente; ciudad 2 aislada => 2
95     provincias.
96     vector<vector<int>> m1 = {
97         {1, 1, 0},
98         {1, 1, 0},
99         {0, 0, 1},
100     };
101     assert(s.findCircleNum(m1) == 2);
102
103     // Ninguna ciudad conectada con otra (solo la diagonal) => 3 provincias.
104     vector<vector<int>> m2 = {
105         {1, 0, 0},
106         {0, 1, 0},
107         {0, 0, 1},
108     };
109     assert(s.findCircleNum(m2) == 3);
110
111     // Todas las ciudades conectadas entre sí => 1 sola provincia.
112     vector<vector<int>> m3 = {
113         {1, 1, 1},
114         {1, 1, 1},
115         {1, 1, 1},
116     };
117     assert(s.findCircleNum(m3) == 1);
118
119     // Una sola ciudad => 1 provincia.
120     vector<vector<int>> m4 = {{1}};
121     assert(s.findCircleNum(m4) == 1);

```

```
119
120     cout << "547 Number of Provinces: todas las pruebas pasaron.\n";
121     return 0;
122 }
```

Complejidad: tiempo $O(n^2 \cdot \alpha(n))$, espacio $O(n)$.

recorremos las n^2 celdas de la matriz (en la práctica, solo la mitad superior), y cada `find/unir` es casi $O(1)$ amortizado gracias a $\alpha(n)$, la inversa de Ackermann.

Notas de entrevista (Amazon). Este problema es hermano directo de **323. Number of Connected Components**: la única diferencia real es el formato de entrada (matriz de adyacencia en vez de lista de aristas), la técnica de fondo es la misma Union-Find. También se puede resolver con DFS/BFS clásico: por cada ciudad no visitada, sumar una provincia y recorrer con DFS toda la fila de la matriz para marcar como visitadas a todas las ciudades conectadas (transitivamente), igual que se “hunde” la tierra en **200. Number of Islands**. Vale la pena mencionar ambos enfoques: DFS/BFS es más directo de explicar dado que la entrada ya es una matriz densa, pero Union-Find generaliza mejor si el entrevistador hace un seguimiento tipo “¿y si las conexiones llegan una a una en vez de venir todas de entrada?”. Cuidado con el costo: si n es grande, $O(n^2)$ ya es el costo de solo leer la matriz de adyacencia, así que ninguna técnica baja de esa cota; con una lista de adyacencia dispersa convendría más recibir aristas explícitas en vez de la matriz completa. Casos borde: una sola ciudad (1 provincia), ninguna ciudad conectada con otra (cada una su propia provincia), y todas las ciudades conectadas entre sí (1 provincia).

743. Network Delay Time

(M)

Código fuente: `src/01-grafos/743-network-delay-time.cpp`▷ Ver animación · [local](#) | [web](#)

Hay n nodos numerados de 1 a n , conectados por aristas *dirigidas y con peso*: `times[i] = [u, v, w]` significa que una señal enviada desde u tarda w unidades de tiempo en llegar a v . Enviamos una señal desde el nodo k . Queremos el tiempo mínimo que tarda en que la señal llegue a *todos* los nodos, o -1 si algún nodo nunca la recibe.

Intuición. Hasta ahora los problemas de grafos del libro usaban BFS/DFS sencillos porque todas las aristas "cuestan" lo mismo (un paso). Aquí las aristas tienen pesos distintos, así que ya no basta con procesar por capas: procesar los nodos en el orden en que el BFS los descubre no garantiza que se descubran en orden de menor a mayor distancia real. Necesitamos el algoritmo de **Dijkstra**: en vez de una cola normal usamos una cola de prioridad (min-heap) que siempre nos da el nodo con la menor distancia acumulada todavía no confirmada.

```
2 --1--> 1
2 --1--> 3 --1--> 4
```

Con `times = [[2,1,1],[2,3,1],[3,4,1]]`, $n = 4$, $k = 2$: desde 2 la señal llega a 1 en tiempo 1, a 3 en tiempo 1 y a 4 (pasando por 3) en tiempo 2. El nodo más lento en enterarse es 4, así que la respuesta es 2.

Solución. Construimos la lista de adyacencia con pares (`vecino`, `peso`) y un arreglo `dist` inicializado a infinito, salvo `dist[k] = 0`. Metemos $(0, k)$ en un min-heap ordenado por distancia. En cada paso sacamos el par con menor distancia; si esa distancia ya quedó obsoleta (mejoramos ese nodo por otro camino mientras tanto), la ignoramos. Si no, *relajamos* cada arista saliente: si llegar al vecino pasando por el nodo actual es más barato que lo que teníamos registrado, actualizamos `dist` del vecino y lo volvemos a meter al heap.

Esto funciona porque, con pesos **no negativos**, la primera vez que sacamos un nodo del heap su distancia ya es la definitiva: cualquier otro camino alternativo tendría que pasar por un nodo cuya distancia parcial ya es mayor o igual, y sumarle más peso solo lo empeora, nunca lo mejora. Si existieran pesos negativos esta garantía se rompe (un camino "más largo" en número de aristas podría terminar siendo más barato) y haría falta Bellman-Ford en su lugar.

Al terminar, si algún nodo se quedó con distancia infinita es porque nunca lo alcanzamos: devolvemos -1 . Si todos los nodos tienen distancia finita, la respuesta es la máxima de esas distancias: la red completa "se entera" hasta que se entera el nodo más lejano.

```
1 //
2 // =====
3 //
4 // 743. Network Delay Time (Medium)
5 //
6 // Hay n nodos numerados de 1 a n. Nos dan times[i] = [u, v, w]: una señal que
7 // sale de u tarda w unidades de tiempo en llegar a v (arista DIRIGIDA y
8 // PONDERADA). Enviamos una señal desde el nodo k. Devuelve el tiempo mínimo
9 // que tarda en que la señal llegue a TODOS los n nodos, o -1 si algún nodo
10 // nunca la recibe.
```



```

9  //
10 // Ejemplo: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
11 //
12 //      2 --1--> 1
13 //      |
14 //      1
15 //      v
16 //      3 --1--> 4
17 //
18 // Desde 2: a 1 le llega en 1, a 3 en 1, a 4 (via 3) en 1+1 = 2.
19 // El nodo que tarda más es 4, con tiempo 2 => respuesta 2.
20 //
21 // Diagrama de Dijkstra (cola de prioridad, min-heap por distancia):
22 //
23 // dist[k] = 0, resto = infinito
24 // heap: (0, k)
25 // mientras el heap no esté vacío:
26 //     sacar (d, u) con menor d
27 //     si d > dist[u] ya conocido, es una entrada obsoleta: ignorar
28 //     para cada arista (u -> v, peso w):
29 //         si dist[u] + w < dist[v]: // relajación de la arista
30 //             dist[v] = dist[u] + w
31 //             heap.push((dist[v], v))
32 //
33 // Idea: es un BFS "por distancia" en vez de por número de aristas. Como los
34 // pesos no son todos iguales, no basta con procesar por capas (como en BFS
35 // normal): hay que procesar siempre el nodo con la MENOR distancia acumulada
36 // todavía no confirmada, y para eso sirve el min-heap. Esto es Dijkstra:
37 // mientras las aristas sean todas de peso >= 0, la primera vez que sacamos
38 // un nodo del heap su distancia ya es la mínima posible (no puede mejorar
39 // después, porque cualquier otro camino que pase por un nodo con distancia
40 // mayor solo puede sumar más peso, nunca restar). Si hubiera pesos negativos
41 // esta garantía se rompe y haría falta Bellman-Ford.
42 // El heap puede tener entradas duplicadas u obsoletas para un mismo nodo
43 // (llegamos a él por varios caminos); las descartamos comparando contra la
44 // distancia ya confirmada en dist[].
45 //
46 // Al final, si algún nodo se quedó en infinito, es inalcanzable => -1.
47 // Si no, la respuesta es la mayor de las distancias mínimas (el último nodo
48 // en enterarse es el que determina cuánto tarda la red completa).
49 //
50 // Complejidad: tiempo  $O(E \log V)$ , espacio  $O(V + E)$ .
51 //
=====
52 #include <vector>
53 #include <queue>
54 #include <climits>
55 #include <cassert>
56 #include <iostream>
57 using namespace std;
58
59 class Solution {
60 public:

```

```

61     int networkDelayTime(vector<vector<int>>& times, int n, int k) {
62         // Lista de adyacencia: adj[u] = {(v, peso), ...}
63         vector<vector<pair<int,int>>> adj(n + 1);
64         for (auto& t : times) {
65             int u = t[0], v = t[1], w = t[2];
66             adj[u].push_back({v, w});
67         }
68
69         vector<int> dist(n + 1, INT_MAX);
70         dist[k] = 0;
71
72         // Min-heap por distancia: pair<distancia, nodo>.
73         priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> heap;
74         heap.push({0, k});
75
76         while (!heap.empty()) {
77             auto [d, u] = heap.top();
78             heap.pop();
79             if (d > dist[u]) continue; // entrada obsoleta, ya mejoramos u
80
81             for (auto& [v, w] : adj[u]) {
82                 if (dist[u] + w < dist[v]) { // relajación de la arista
83                     dist[v] = dist[u] + w;
84                     heap.push({dist[v], v});
85                 }
86             }
87         }
88
89         int maxDist = 0;
90         for (int nodo = 1; nodo <= n; ++nodo) {
91             if (dist[nodo] == INT_MAX) return -1; // nodo inalcanzable
92             maxDist = max(maxDist, dist[nodo]);
93         }
94         return maxDist;
95     }
96 };
97
98 int main() {
99     Solution s;
100
101     // Caso normal: la señal llega a todos, el nodo 4 es el más lento (t=2).
102     vector<vector<int>> t1 = {{2,1,1},{2,3,1},{3,4,1}};
103     assert(s.networkDelayTime(t1, 4, 2) == 2);
104
105     // Nodo 2 es inalcanzable desde 1 (solo hay arista 1 -> 2, no llega a 2
106     // el resto de nodos aunque exista arco 1->2... aquí forzamos un nodo sin
107     // ningún arco de entrada).
108     vector<vector<int>> t2 = {{1,2,1}};
109     assert(s.networkDelayTime(t2, 3, 1) == -1); // nodo 3 nunca recibe la se
        ñal
110
111     // Un solo nodo: la señal ya está en k, tarda 0.
112     vector<vector<int>> t3 = {};
113     assert(s.networkDelayTime(t3, 1, 1) == 0);

```

```
114 // Camino donde el orden de procesamiento del heap importa: hay una ruta
115 // directa cara y una ruta indirecta barata.
116 vector<vector<int>> t4 = {{1,2,5},{1,3,1},{3,2,1}};
117 // Desde 1: directo a 2 cuesta 5, pero via 3 cuesta 1+1=2 (mejor).
118 assert(s.networkDelayTime(t4, 3, 1) == 2);
119
120
121 cout << "743 Network Delay Time: todas las pruebas pasaron.\n";
122 return 0;
123 }
```

Complejidad: tiempo $O(E \log V)$, espacio $O(V + E)$.

Notas de entrevista (Amazon). Este problema es el ejemplo canónico de Dijkstra con cola de prioridad, y suele usarse para verificar que el candidato entiende *por qué* el algoritmo requiere pesos no negativos (vale la pena mencionarlo aunque no lo pregunten). Otro punto que se evalúa es el manejo de entradas obsoletas en el heap: como no existe una operación barata de "disminuir clave" en el `priority_queue` de la STL, simplemente insertamos duplicados y los descartamos comparando contra `dist` al sacarlos, en vez de intentar borrarlos del heap. Casos borde a mencionar: un solo nodo (`k` tarda 0 en llegar a sí mismo), un nodo sin ninguna arista de entrada (inalcanzable, `-1`), y la trampa de asumir que la ruta con menos aristas es siempre la más rápida cuando en realidad hay que comparar el peso acumulado.

787. Cheapest Flights Within K Stops

(M)

Código fuente: `src/01-grafos/787-cheapest-flights-k-stops.cpp`▷ Ver animación · [local](#) | [web](#)

Hay n ciudades numeradas de 0 a $n-1$ y una lista `flights` de vuelos `[from, to, price]` (aristas dirigidas y ponderadas, sin pesos negativos). Dados `src`, `dst` y `k`, buscamos el precio más barato de `src` a `dst` usando **a lo más k escalas** (paradas intermedias), o -1 si no existe tal ruta. Usar a lo más k escalas equivale a usar a lo más $k+1$ vuelos (aristas) en el camino.

Intuición. Dijkstra procesa nodos por distancia mínima acumulada, pero no da control directo sobre *cuántas aristas* usa un camino: una vez que confirmamos la distancia mínima a un nodo, ya no sabemos ni nos importa cuántos saltos costó. Aquí sí importa: dos rutas al mismo destino con el mismo precio pero distinto número de escalas no son intercambiables. Necesitamos un algoritmo que avance *una arista a la vez* y nos deje cortar exactamente después de $k+1$ aristas.

```
n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]
```

```
0 --100--> 1 --100--> 2 --200--> 3
1 -----600-----> 3
```

Con `src=0`, `dst=3`, `k=1`: la ruta $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ cuesta $100 + 100 + 200 = 400$ pero usa 2 escalas (nodos 1 y 2), más de las permitidas. La única ruta con a lo más 1 escala es $0 \rightarrow 1 \rightarrow 3$, que cuesta $100 + 600 = 700$. Respuesta: 700.

Bellman-Ford limitado a rondas (patrón nuevo). Bellman-Ford relaja **todas** las aristas del grafo, ronda tras ronda. Se puede demostrar que, después de r rondas de relajación, `dist[v]` es el costo mínimo para llegar a v usando **a lo más r aristas**. Como k escalas equivalen a $k+1$ aristas, basta con ejecutar **exactamente $k+1$ rondas**: ni una más (usaríamos más escalas de las permitidas) ni una menos.

El truco de la copia. En cada ronda construimos un arreglo `nuevaDist` a partir de una *fotografía fija* de la ronda anterior (`dist`), y solo al terminar de recorrer todas las aristas reemplazamos `dist` por `nuevaDist`. Si en vez de eso actualizáramos `dist` “en el lugar” como en Dijkstra, una sola ronda podría encadenar varias relajaciones seguidas: por ejemplo, relajar $0 \rightarrow 1$ y, en esa misma ronda, usar ya el nuevo `dist[1]` para relajar $1 \rightarrow 2$. Eso colaría de contrabando dos aristas (dos escalas) en una sola ronda, dando una respuesta más barata pero inválida porque excede el límite de k escalas. Al copiar, cada ronda solo puede sumar *exactamente una* arista por camino, sin importar el orden en que iteremos la lista de vuelos.

```
1 //
2 // =====
3 //
4 // 787. Cheapest Flights Within K Stops (Medium)
5 //
6 // Hay n ciudades numeradas de 0 a n-1. Nos dan flights[i] = [from, to, price
7 //   ]:
8 // un vuelo DIRIGIDO de "from" a "to" que cuesta "price" (arista dirigida y
9 // ponderada, sin pesos negativos). Dado src, dst y k, devuelve el precio más
```



```

58 #include <vector>
59 #include <cassert>
60 #include <iostream>
61 using namespace std;
62
63 class Solution {
64 public:
65     int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int
        dst, int k) {
66         const int INF = 1e9;
67         vector<int> dist(n, INF);
68         dist[src] = 0;
69
70         // k+1 rondas: cubrimos caminos de hasta k escalas (k+1 aristas).
71         for (int ronda = 0; ronda <= k; ++ronda) {
72             vector<int> nuevaDist = dist; // copia: fotografía de la ronda
              anterior
73             for (auto& f : flights) {
74                 int u = f[0], v = f[1], precio = f[2];
75                 if (dist[u] == INF) continue; // u todavía inalcanzable en
                  esta ronda
76                 if (dist[u] + precio < nuevaDist[v]) {
77                     nuevaDist[v] = dist[u] + precio;
78                 }
79             }
80             dist = nuevaDist;
81         }
82
83         return dist[dst] == INF ? -1 : dist[dst];
84     }
85 };
86
87 int main() {
88     Solution s;
89
90     // Caso clásico: con k=1 el camino directo 0->1->3 (600+100=700) gana
91     // porque 0->1->2->3 usa 2 escalas y no está permitido.
92     vector<vector<int>> f1 =
93         {{0,1,100},{1,2,100},{2,0,100},{1,3,600},{2,3,200}};
94     assert(s.findCheapestPrice(4, f1, 0, 3, 1) == 700);
95
96     // Con k=0 (sin escalas) y sin vuelo directo 0->3, es imposible.
97     assert(s.findCheapestPrice(4, f1, 0, 3, 0) == -1);
98
99     // Con más escalas permitidas (k=4, sobran rondas) sí se puede aprovechar
100    // la ruta de 2 escalas 0->1->2->3, más barata (100+100+200=400).
101    assert(s.findCheapestPrice(4, f1, 0, 3, 4) == 400);
102
103    // Ejemplo clásico de LeetCode: hay una ruta directa cara y una ruta con
104    // una escala más barata.
105    vector<vector<int>> f2 = {{0,1,100},{1,2,100},{0,2,500}};
106    assert(s.findCheapestPrice(3, f2, 0, 2, 1) == 200); // 0->1->2 = 200
107    assert(s.findCheapestPrice(3, f2, 0, 2, 0) == 500); // solo directo: 500

```

```
108 // Destino totalmente inalcanzable, sin importar cuántas escalas demos.
109 vector<vector<int>> f3 = {{0,1,100}};
110 assert(s.findCheapestPrice(3, f3, 0, 2, 1) == -1);
111
112 // src == dst: el costo es 0 sin necesidad de volar.
113 assert(s.findCheapestPrice(3, f3, 0, 0, 1) == 0);
114
115 cout << "787 Cheapest Flights: todas las pruebas pasaron.\n";
116 return 0;
117 }
```

Complejidad: tiempo $O(k \cdot E)$, espacio $O(n)$.

Notas de entrevista (Amazon). Este problema es la puerta de entrada al patrón de **Bellman-Ford con rondas acotadas**, muy usado cuando el límite del problema no es sobre el costo sino sobre el *número de aristas* del camino (aquí, escalas de vuelo). El error más común es olvidar la copia y relajar sobre el mismo arreglo, lo que permite que una ruta use más escalas de las permitidas en una sola ronda -Amazon suele preguntar explícitamente por qué esa copia es necesaria-. Otro error es confundir “k escalas” con “k aristas”: hay que iterar k+1 rondas, no k. Casos borde a mencionar: `src == dst` (costo 0 sin volar), destino inalcanzable con cualquier número de escalas (-1), y k tan grande que el límite de escalas ya no restringe nada (el algoritmo simplemente converge al Bellman-Ford completo, sin rondas de más que puedan dañar el resultado).

133. Clone Graph

(M)

Código fuente: [src/01-grafos/133-clone-graph.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una referencia a un nodo de un grafo no dirigido y *conexo*. Cada nodo tiene un valor entero (**val**) y una lista de vecinos (**neighbors**). Queremos devolver una **copia profunda** de todo el grafo: cada nodo de la copia debe ser un objeto nuevo, con el mismo valor y las mismas conexiones que el original.

Intuición. Es un DFS normal, con un truco: en vez de un simple arreglo de **visitados**, usamos un mapa **original** → **copia**. La primera vez que vemos un nodo original creamos su copia y la registramos en el mapa *antes* de recorrer sus vecinos. Así, si el grafo tiene un ciclo (muy común, ya que es no dirigido), al volver a toparnos con un nodo ya visitado simplemente devolvemos la copia guardada en el mapa, en lugar de clonarlo de nuevo o caer en una recursión infinita.



Lista de adyacencia de este ejemplo: 1 : [2, 4], 2 : [1, 3], 3 : [2, 4], 4 : [1, 3]. Al clonar desde el nodo 1, el DFS visita 2, que a su vez intenta visitar 1 otra vez: como 1 ya está en el mapa, se reutiliza su copia en lugar de recrearla.

Solución. DFS recursivo con `unordered_map<Node*, Node*>` como memoria de clones. Al entrar a un nodo: si ya tiene copia registrada, la devolvemos; si no, creamos la copia, la registramos de inmediato (esto rompe los ciclos) y luego clonamos recursivamente cada vecino para llenar **neighbors** de la copia.

```

1 //
2 // =====
3 //
4 // 133. Clone Graph (Medium)
5 //
6 //
7 // Nos dan una referencia a un nodo de un grafo no dirigido y CONEXO. Cada
8 // nodo tiene un valor entero y una lista de vecinos. Debemos devolver una
9 // COPIA PROFUNDA (deep copy) de todo el grafo: cada nodo de la copia debe ser
10 // un objeto nuevo (distinta dirección de memoria), con el mismo valor y las
11 // mismas conexiones que su original.
12 //
13 // Ejemplo (grafo cuadrado de 4 nodos):
14 //
15 //      1 -- 2          Lista de adyacencia:
16 //      |    |          1: [2, 4]
17 //      4 -- 3          2: [1, 3]
18 //                      3: [2, 4]
19 //                      4: [1, 3]
20 //
21 // Diagrama del DFS con mapa original -> copia:
22 //

```



```

20 //  visitar(1) -> ¿existe copia de 1? no: la creamos, la guardamos en el mapa
21 //      y recorremos sus vecinos [2, 4]
22 //      visitar(2) -> creamos copia de 2, recorremos [1, 3]
23 //      visitar(1) -> YA existe copia de 1 en el mapa:
24 //      la devolvemos sin recrearla (evita
25 //      recursión infinita en el ciclo)
26 //      visitar(3) -> creamos copia de 3, recorremos [2, 4]
27 //      visitar(2) -> ya existe, se devuelve del mapa
28 //      visitar(4) -> creamos copia de 4, recorremos [1,
29 //      3]
30 //      (1 y 3 ya existen, se devuelven del mapa)
31 //
32 // Idea: DFS clásico, pero en vez de un simple "visitado" usamos un
33 // unordered_map<Node*, Node*> que traduce cada nodo original a su copia. La
34 // primera vez que visitamos un nodo original creamos su copia y la
35 // registramos en el mapa ANTES de recorrer sus vecinos; así, si el grafo
36 // tiene un ciclo (como el cuadrado de arriba), al volver a toparnos con un
37 // nodo ya visitado simplemente reutilizamos la copia guardada en vez de
38 // clonar de nuevo o entrar en un bucle infinito.
39 //
40 // Complejidad: tiempo  $O(V+E)$  (visitamos cada nodo y cada arista una vez),
41 // espacio  $O(V)$  por el mapa y la pila de recursión.
42 //
43 =====
44 #include <vector>
45 #include <unordered_map>
46 #include <cassert>
47 #include <iostream>
48 using namespace std;
49
50 // Definición del nodo del grafo (la da el enunciado de LeetCode).
51 struct Node {
52     int val;
53     vector<Node*> neighbors;
54     Node() : val(0), neighbors(vector<Node*>()) {}
55     Node(int _val) : val(_val), neighbors(vector<Node*>()) {}
56     Node(int _val, vector<Node*> _neighbors) : val(_val), neighbors(_neighbors) {}
57 };
58
59 class Solution {
60 public:
61     Node* cloneGraph(Node* nodo) {
62         if (nodo == nullptr) return nullptr; // grafo vacío => copia vacía
63         unordered_map<Node*, Node*> copias; // original -> copia
64         return clonar(nodo, copias);
65     }
66 private:
67     Node* clonar(Node* original, unordered_map<Node*, Node*>& copias) {
68         auto it = copias.find(original);
69         if (it != copias.end()) return it->second; // ya clonado: reusar

```

```

70     Node* copia = new Node(original->val);
71     copias[original] = copia;    // registrar ANTES de recorrer vecinos
72                                 // (rompe los ciclos del grafo)
73     for (Node* vecino : original->neighbors) {
74         copia->neighbors.push_back(clonar(vecino, copias));
75     }
76     return copia;
77 }
78 };
79
80 // -----
81 // Utilidades de prueba: liberar la memoria de un grafo clonado y comparar
82 // que dos grafos (original y copia) tengan la misma estructura pero nodos
83 // con direcciones de memoria distintas.
84 // -----
85
86 // Verifica recursivamente que "copia" sea una réplica fiel de "original":
87 // mismos valores, misma cantidad de vecinos en el mismo orden, pero cada
88 // nodo de la copia debe ser un puntero DISTINTO al original correspondiente.
89 void verificarClonSonIguales(Node* original, Node* copia,
90                             unordered_map<Node*, Node*>& visitados) {
91     assert(original != nullptr && copia != nullptr);
92     assert(original != copia);           // direcciones distintas
93     assert(original->val == copia->val);  // mismo valor
94     assert(original->neighbors.size() == copia->neighbors.size());
95
96     visitados[original] = copia;
97     for (size_t i = 0; i < original->neighbors.size(); ++i) {
98         Node* vecinoOriginal = original->neighbors[i];
99         Node* vecinoCopia = copia->neighbors[i];
100         if (visitados.count(vecinoOriginal)) {
101             // Ya visitado: sólo confirmamos que apunte a la MISMA copia
102             // (así comprobamos que no se duplicó el clon de un nodo).
103             assert(visitados[vecinoOriginal] == vecinoCopia);
104         } else {
105             verificarClonSonIguales(vecinoOriginal, vecinoCopia, visitados);
106         }
107     }
108 }
109
110 // Libera todos los nodos alcanzables desde "raiz" (evita fugas de memoria
111 // entre las distintas pruebas del main).
112 void liberarGrafo(Node* raiz) {
113     if (raiz == nullptr) return;
114     unordered_map<Node*, bool> visitados;
115     vector<Node*> pila = {raiz};
116     visitados[raiz] = true;
117     vector<Node*> todos;
118     while (!pila.empty()) {
119         Node* actual = pila.back();
120         pila.pop_back();
121         todos.push_back(actual);
122         for (Node* vecino : actual->neighbors) {
123             if (!visitados.count(vecino)) {

```

```

124         visitados[vecino] = true;
125         pila.push_back(vecino);
126     }
127 }
128 }
129 for (Node* n : todos) delete n;
130 }
131
132 int main() {
133     Solution s;
134
135     // --- Caso 1: grafo cuadrado de 4 nodos (1-2-3-4-1), con ciclo ---
136     Node* n1 = new Node(1);
137     Node* n2 = new Node(2);
138     Node* n3 = new Node(3);
139     Node* n4 = new Node(4);
140     n1->neighbors = {n2, n4};
141     n2->neighbors = {n1, n3};
142     n3->neighbors = {n2, n4};
143     n4->neighbors = {n1, n3};
144
145     Node* copia1 = s.cloneGraph(n1);
146     assert(copia1 != nullptr);
147     assert(copia1 != n1);    // el nodo raíz de la copia es un objeto distinto
148     {
149         unordered_map<Node*, Node*> visitados;
150         verificarClonSonIguales(n1, copia1, visitados);
151         assert(visitados.size() == 4);    // se clonaron exactamente 4 nodos
152     }
153     liberarGrafo(copia1);
154     liberarGrafo(n1);
155
156     // --- Caso 2: un solo nodo, sin vecinos ---
157     Node* solo = new Node(1);
158     Node* copiaSolo = s.cloneGraph(solo);
159     assert(copiaSolo != nullptr);
160     assert(copiaSolo != solo);
161     assert(copiaSolo->val == 1);
162     assert(copiaSolo->neighbors.empty());
163     liberarGrafo(copiaSolo);
164     liberarGrafo(solo);
165
166     // --- Caso 3: grafo nulo (vacío) ---
167     assert(s.cloneGraph(nullptr) == nullptr);
168
169     cout << "133 Clone Graph: todas las pruebas pasaron.\n";
170     return 0;
171 }

```

Complejidad: tiempo $O(V + E)$, espacio $O(V)$.

Notas de entrevista (Amazon). El error típico es registrar la copia en el mapa *después* de recorrer los vecinos: eso provoca recursión infinita en cuanto el grafo tiene un ciclo. Menciona

también la alternativa con BFS y una cola (misma idea del mapa, pero iterativa, útil si te preocupa el límite de la pila de recursión en grafos muy grandes). Casos borde a cubrir: nodo nulo (grafo vacío) devuelve `nullptr`, un solo nodo sin vecinos, y verificar que los nodos de la copia sean direcciones de memoria distintas a las del original (una copia profunda real, no un alias).

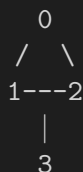
1192. Critical Connections in a Network

(H)

Código fuente: [src/01-grafos/1192-critical-connections.cpp](#)▷ Ver animación · [local](#) | [web](#)

Hay n servidores numerados de 0 a $n - 1$, conectados por una lista de **connections** (aristas **no dirigidas**). El grafo es *conexo*: siempre se puede llegar de cualquier servidor a cualquier otro. Una conexión es **crítica** (un *punte*) si, al quitarla, el grafo se parte en dos o más componentes; es decir, es la única forma en que un grupo de servidores se comunica con el resto. Debemos devolver **todas** las conexiones críticas.

Intuición. Piensa en cada conexión como un cable físico. Si un cable forma parte de un ciclo (hay otra ruta alterna entre sus dos extremos), quitarlo no aísla a nadie: la información puede rodear por el ciclo. Pero si un cable es la *única* ruta hacia cierta parte de la red —como una rama que cuelga de un ciclo— quitarlo sí la desconecta. Ese cable es un puente.



Ejemplo con $n = 4$, **connections** = $[[0, 1], [1, 2], [2, 0], [1, 3]]$: el triángulo 0-1-2 tiene tres rutas distintas entre cualquier par de sus nodos, así que ninguna de sus tres aristas es puente. Pero 1-3 es la *única* forma de llegar al servidor 3: quitarla lo aísla. La única conexión crítica es $\{1, 3\}$.

Solución: puentes de Tarjan. Este es un patrón nuevo en el libro, así que vale la pena explicarlo con calma. Hacemos un DFS desde cualquier nodo (el grafo es conexo) y llevamos dos arreglos:

- **disc[u]**: el “número de orden” en que el DFS *descubrió* al nodo u por primera vez. Es un contador global que aumenta cada vez que visitamos un nodo nuevo (el primero visitado tiene **disc** = 0, el segundo **disc** = 1, ...). Una vez asignado, **disc[u]** ya no cambia.
- **low[u]**: el **disc** *más pequeño* al que se puede “regresar” desde el subárbol de u , usando aristas del árbol de DFS hacia descendientes y, como máximo, **una** arista de retroceso (*back-edge*) que salte hacia un ancestro ya visitado. Este valor sí se actualiza (sólo puede bajar) conforme el DFS regresa de sus hijos.

Si al procesar la arista de árbol (u, v) —con v hijo de u — encontramos una arista de retroceso desde el subárbol de v hacia u o algún ancestro de u , entonces v y su subárbol tienen una “cuerda” que los mantiene conectados al resto del grafo aunque quitemos (u, v) : no es puente. Eso se traduce en la condición clave:

$$(u, v) \text{ es puente} \iff \text{low}[v] > \text{disc}[u]$$

$\text{low}[v] > \text{disc}[u]$ significa que, desde el subárbol de v , es *imposible* alcanzar a u o a cualquier ancestro de u salvo pasando otra vez por la arista (u, v) : esa arista es la única puerta de entrada y salida del subárbol, y por lo tanto es un puente.

En el ejemplo: al visitar 2 (hijo de 1) encontramos una arista de retroceso hacia 0 (ya visitado y no es su padre), así que **low**[2] = **disc**[0] = 0; ese valor se propaga hacia arriba: **low**[1] = 0 y **low**[0] = 0. Ni (0, 1) ni (1, 2) son puente porque $0 \not> \text{disc}[0] = 0$ ni $0 \not> \text{disc}[1] = 1$. En cambio 3 no tiene ninguna arista de retroceso (su único vecino es su padre, 1), así que **low**[3] se queda en su propio **disc** = 3, y $3 > \text{disc}[1] = 1$: la arista (1, 3) sí es puente.


```

50 //      |
51 //      (back-edge a 0)
52 //
53 // Al procesar 2, encontramos una arista de retroceso hacia 0 (ya visitado,
54 // y no es el padre de 2). Eso significa que 2 puede "regresar" hasta
55 // disc=0 sin usar la arista 1-2 ni la 0-1: por eso low[2] = 0. Ese 0 se
56 // propaga hacia arriba: low[1] = min(low[1], low[2]) = 0, y luego
57 // low[0] = min(low[0], low[1]) = 0. Así, ni la arista 0-1 ni la 1-2 son
58 // puente, porque el subárbol de abajo tiene una "cuerda" (back-edge) que
59 // lo mantiene conectado al resto incluso sin esa arista del árbol.
60 //
61 // En cambio, 3 no tiene ninguna arista de retroceso (su único vecino es su
62 // padre, 1), así que low[3] se queda en su propio disc = 3. Ese 3 NO puede
63 // "escapar" hacia arriba de disc[1] = 1.
64 //
65 // La condición clave: para una arista de árbol (u, v) donde v es hijo de u,
66 //
67 //      (u,v) es puente  <=>  low[v] > disc[u]
68 //
69 // ¿Por qué funciona? low[v] > disc[u] significa que, desde el subárbol de v
70 // (usando aristas del árbol hacia abajo y a lo más una arista de retroceso),
71 // es IMPOSIBLE alcanzar a u o a cualquier ancestro de u. La única forma en
72 // que v y su subárbol se comunican con el resto del grafo es a través de la
73 // arista (u, v). Quitárla los desconecta: es un puente. Si en cambio
74 // low[v] <= disc[u], existe una "cuerda" que rodea la arista (u, v) y la
75 // vuelve prescindible.
76 //
77 // Aplicando esto al ejemplo: low[3] = 3 > disc[1] = 1 => (1,3) es puente.
78 // low[2] = 0, disc[1] = 1 => 0 > 1 es falso => (1,2) NO es puente.
79 // low[1] = 0, disc[0] = 0 => 0 > 0 es falso => (0,1) NO es puente.
80 //
81 // Al final del DFS habremos revisado cada arista del árbol exactamente una
82 // vez con esta prueba, y habremos recolectado todos los puentes.
83 //
84 // Complejidad: tiempo  $O(V + E)$  (un DFS normal, cada nodo y arista se visitan
85 // una vez), espacio  $O(V + E)$  para la lista de adyacencia más
86 //  $O(V)$  de los arreglos disc/low y la pila de recursión.
87 //
=====
88 #include <vector>
89 #include <cassert>
90 #include <algorithm>
91 #include <iostream>
92 using namespace std;
93
94 class Solution {
95 public:
96     vector<vector<int>> criticalConnections(int n, vector<vector<int>>&
97         connections) {
98         vector<vector<int>> adj(n);
99         for (auto& c : connections) {
100             adj[c[0]].push_back(c[1]);

```

```

101     }
102
103     disc.assign(n, -1);
104     low.assign(n, -1);
105     timer = 0;
106     vector<vector<int>> puentes;
107
108     // El grafo es conexo (garantía del enunciado), así que un solo DFS
109     // desde el nodo 0 recorre todos los servidores.
110     dfs(0, -1, adj, puentes);
111
112     return puentes;
113 }
114
115 private:
116     vector<int> disc;    // disc[u]: momento de descubrimiento de u
117     vector<int> low;    // low[u]: disc mínimo alcanzable desde el subárbol de
118         u
119     int timer;
120
121     void dfs(int u, int padre, vector<vector<int>>& adj, vector<vector<int>>&
122         puentes) {
123         disc[u] = low[u] = timer++;
124
125         for (int v : adj[u]) {
126             if (v == padre) continue;    // no regresar por la misma arista al
127                 padre
128
129             if (disc[v] == -1) {
130                 // v aún no visitado: es hijo de u en el árbol de DFS.
131                 dfs(v, u, adj, puentes);
132                 low[u] = min(low[u], low[v]);    // propagar hacia arriba
133                 if (low[v] > disc[u]) {
134                     // Ninguna cuerda desde el subárbol de v llega hasta u
135                     // (ni más arriba): la arista (u, v) es un puente.
136                     puentes.push_back({min(u, v), max(u, v)});
137                 }
138             } else {
139                 // v ya visitado y no es el padre: es una arista de retroceso
140                 // (back-edge) hacia un ancestro. Actualiza low[u] con el
141                 // disc de ese ancestro (no con su low, por eso se usa disc[v
142                 ]).
143                 low[u] = min(low[u], disc[v]);
144             }
145         }
146     }
147 }
148
149 int main() {
150     Solution s;
151
152     // Normaliza (ya vienen como [min,max] desde el algoritmo) y ordena el
153     // vector de puentes para comparar sin depender del orden de recorrido.
154     auto ordenar = [](vector<vector<int>> v) {

```



```

151         sort(v.begin(), v.end());
152         return v;
153     };
154
155     // Caso 1 (el del enunciado): triángulo 0-1-2 con una arista colgante 1-3.
156     // El triángulo no tiene puentes; la única conexión crítica es 1-3.
157     vector<vector<int>> c1 = {{0,1},{1,2},{2,0},{1,3}};
158     vector<vector<int>> esperado1 = {{1,3}};
159     assert(ordemar(s.criticalConnections(4, c1)) == esperado1);
160
161     // Caso 2: camino lineal 0-1-2-3. Sin ciclos, así que TODAS las aristas
162     // son puentes (quitar cualquiera desconecta el grafo en dos pedazos).
163     vector<vector<int>> c2 = {{0,1},{1,2},{2,3}};
164     vector<vector<int>> esperado2 = {{0,1},{1,2},{2,3}};
165     assert(ordemar(s.criticalConnections(4, c2)) == esperado2);
166
167     // Caso 3: dos triángulos (0-1-2 y 3-4-5) unidos por una sola arista
168     // puente (2-3). Ningún lado del triángulo es puente, pero la conexión
169     // que los une sí lo es.
170     vector<vector<int>> c3 = {{0,1},{1,2},{2,0},{2,3},{3,4},{4,5},{5,3}};
171     vector<vector<int>> esperado3 = {{2,3}};
172     assert(ordemar(s.criticalConnections(6, c3)) == esperado3);
173
174     // Caso 4: anillo (ciclo) de 4 nodos. Cada arista tiene una ruta
175     // alternativa por el resto del anillo, así que no hay puentes.
176     vector<vector<int>> c4 = {{0,1},{1,2},{2,3},{3,0}};
177     assert(ordemar(s.criticalConnections(4, c4)).empty());
178
179     cout << "1192 Critical Connections: todas las pruebas pasaron.\n";
180     return 0;
181 }

```

Complejidad: tiempo $O(V + E)$, espacio $O(V + E)$.

Notas de entrevista (Amazon). Este problema evalúa si conoces algoritmos de grafos más allá de BFS/DFS básico, así que vale la pena explicar el porqué de `low[]`, no sólo memorizar la fórmula. Puntos clave para mencionar: (1) al visitar una arista de retroceso, actualiza `low[u]` con `disc[v]` (el momento de descubrimiento), *no* con `low[v]` — esa distinción es la que evita "atravesar" ciclos que no existen. (2) Cuidado con las aristas paralelas hacia el padre: aquí se evita re-visitar al padre comparando el nodo, pero si el grafo permitiera múltiples conexiones idénticas entre los mismos dos servidores habría que excluir sólo *una* arista hacia el padre (por ejemplo, usando el índice de la arista en vez del nodo padre); el enunciado de LeetCode garantiza que no hay conexiones repetidas. (3) El orden en que se devuelven los puentes no importa, pero normalizar cada arista como (min, max) facilita comparar resultados. Casos borde: $n = 1$ sin conexiones (ningún puente posible), y un árbol puro (sin ciclos), donde absolutamente todas las aristas son puentes.

Capítulo 2

Árboles

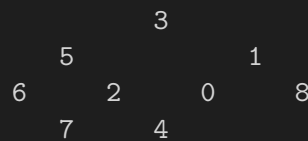
236. Lowest Common Ancestor of a Binary Tree

(M)

Código fuente: `src/02-arboles/236-lowest-common-ancestor.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan un árbol binario y dos nodos p y q que están garantizados presentes en el árbol. Queremos encontrar su *ancestro común más bajo* (LCA, *Lowest Common Ancestor*): el nodo más profundo que tiene tanto a p como a q entre sus descendientes (un nodo se considera descendiente de sí mismo).

Intuición. Piensa en el árbol como un árbol genealógico. Para encontrar al ancestro común de dos personas, bajas desde la raíz preguntando en cada nodo: "¿alguno de los dos que busco vive en mi rama izquierda?" y "¿en mi rama derecha?". Si uno vive a la izquierda y el otro a la derecha, tú (el nodo actual) eres el punto exacto donde sus caminos se separan: eres el LCA. Si ambos viven del mismo lado, la respuesta está más abajo, así que dejas que la rama que sí encontró algo te lo diga.



En este árbol, $LCA(5, 1) = 3$ porque cada uno cuelga de una rama distinta de la raíz. En cambio $LCA(5, 4) = 5$ porque 5 es ancestro directo de 4 (4 vive dentro del propio subárbol de 5). Y $LCA(6, 4) = 5$ porque tanto 6 como 4 cuelgan del subárbol de 5, aunque en ramas distintas dentro de él.

Solución. Hacemos DFS recursivo tipo post-order. Cada llamada devuelve, para el subárbol que analiza, "el nodo relevante encontrado": p , q , el LCA ya identificado, o `nullptr` si ninguno de los dos está ahí.

Si el nodo actual es `nullptr`, p o q , lo devolvemos de inmediato: no hace falta seguir bajando por esa rama, ya encontramos uno de los dos objetivos (o llegamos al final del árbol). Si la recursión a la izquierda y la de la derecha devuelven ambas algo no nulo, p y q se repartieron uno a cada lado, así que el nodo actual es el LCA. Si solo un lado devuelve algo, ese resultado se propaga hacia arriba sin cambios. El código completo (abre el archivo con clic en el título):

```

1 //
2 // =====
3 // 236. Lowest Common Ancestor of a Binary Tree (Medium)
4 //
5 // -----
6 // Dado un árbol binario y dos nodos p y q que están garantizados presentes en
7 // el árbol, encuentra su ancestro común más bajo (LCA, Lowest Common Ancestor
8 // ).
9 // El LCA es el nodo más profundo que tiene tanto a p como a q como
10 // descendientes (un nodo se considera descendiente de sí mismo).
11 //
12 // Ejemplo:
13 //
14 // 3
  
```

```

11 //      /      \
12 //      5      1
13 //     / \    / \
14 //    6  2 0  8
15 //       / \
16 //      7  4
17 //
18 //  LCA(5, 1) = 3  (están en subárboles distintos de la raíz)
19 //  LCA(5, 4) = 5  (5 es ancestro directo de 4)
20 //  LCA(6, 4) = 5  (ambos cuelgan del subárbol de 5)
21 //
22 // Diagrama de la recursión DFS: en cada nodo preguntamos "¿p o q vive en mi
23 // subárbol izquierdo?" y "¿en el derecho?":
24 //
25 //      nodo
26 //     /    \
27 //  buscar p/q  buscar p/q
28 //  en izq      en der
29 //
30 // - Si el nodo actual ES p o q, lo devolvemos de inmediato (no hace falta
31 // seguir bajando por ese lado: ya encontramos uno de los dos objetivos).
32 // - Si izquierda y derecha devuelven ambas algo no nulo, quiere decir que
33 // p y q se repartieron uno a cada lado: el nodo actual es el LCA.
34 // - Si solo un lado devuelve algo no nulo, ese resultado se propaga hacia
35 // arriba (el LCA está más arriba, o es el propio nodo encontrado).
36 //
37 // Idea: DFS recursivo tipo post-order. Cada llamada devuelve, para el
38 // subárbol que analiza, "el nodo relevante encontrado" (p, q, el LCA, o
39 // nullptr si ninguno de los dos está ahí).
40 //
41 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
42 // espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
43 //
=====
44 #include <cassert>
45 #include <iostream>
46 using namespace std;
47
48 struct TreeNode {
49     int val;
50     TreeNode *left;
51     TreeNode *right;
52     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
53 };
54
55 class Solution {
56 public:
57     TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
58         if (root == nullptr || root == p || root == q) return root;
59
60         TreeNode* izq = lowestCommonAncestor(root->left, p, q);
61         TreeNode* der = lowestCommonAncestor(root->right, p, q);
62

```

```

63         if (izq != nullptr && der != nullptr) return root; // p y q en ramas
           distintas
64         return (izq != nullptr) ? izq : der; // propaga el que
           exista
65     }
66 };
67
68 int main() {
69     Solution s;
70
71     // Árbol de prueba:
72     //           3
73     //        /  \
74     //       5    1
75     //      / \  / \
76     //     6  2 0  8
77     //        / \
78     //       7  4
79     TreeNode* n3 = new TreeNode(3);
80     TreeNode* n5 = new TreeNode(5);
81     TreeNode* n1 = new TreeNode(1);
82     TreeNode* n6 = new TreeNode(6);
83     TreeNode* n2 = new TreeNode(2);
84     TreeNode* n0 = new TreeNode(0);
85     TreeNode* n8 = new TreeNode(8);
86     TreeNode* n7 = new TreeNode(7);
87     TreeNode* n4 = new TreeNode(4);
88
89     n3->left = n5;    n3->right = n1;
90     n5->left = n6;    n5->right = n2;
91     n1->left = n0;    n1->right = n8;
92     n2->left = n7;    n2->right = n4;
93
94     // Caso 1: p y q en subárboles distintos de la raíz => LCA es la raíz.
95     TreeNode* r1 = s.lowestCommonAncestor(n3, n5, n1);
96     assert(r1 != nullptr && r1->val == 3);
97
98     // Caso 2: uno es ancestro directo del otro => LCA es el ancestro (5).
99     TreeNode* r2 = s.lowestCommonAncestor(n3, n5, n4);
100    assert(r2 != nullptr && r2->val == 5);
101
102    // Caso 3: ambos cuelgan del subárbol de 2 (hijo de 5) => LCA es 2.
103    TreeNode* r3 = s.lowestCommonAncestor(n3, n7, n4);
104    assert(r3 != nullptr && r3->val == 2);
105
106    // Caso 4: árbol pequeño de dos nodos, p es la raíz y q su único hijo.
107    TreeNode* a = new TreeNode(1);
108    TreeNode* b = new TreeNode(2);
109    a->left = b;
110    TreeNode* r4 = s.lowestCommonAncestor(a, a, b);
111    assert(r4 != nullptr && r4->val == 1);
112
113    // Caso 5: dos hojas hermanas => LCA es su padre directo (1).
114    TreeNode* r5 = s.lowestCommonAncestor(n3, n0, n8);

```

```
115     assert(r5 != nullptr && r5->val == 1);
116
117     delete n6; delete n7; delete n4; delete n2; delete n5;
118     delete n0; delete n8; delete n1; delete n3;
119     delete b; delete a;
120
121     cout << "236 Lowest Common Ancestor: todas las pruebas pasaron.\n";
122     return 0;
123 }
```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

Notas de entrevista (Amazon). Pregunta muy común. Puntos clave a mencionar: la solución no necesita comparar valores (funciona incluso con valores repetidos) porque compara punteros de nodo directamente. Si el árbol fuera un *árbol de búsqueda binario* (BST), existe una solución más simple en $O(h)$ sin recursión doble, aprovechando el orden de los valores para decidir hacia qué lado bajar. Casos borde: p es ancestro de q (o viceversa), y p/q son hojas hermanas directas.

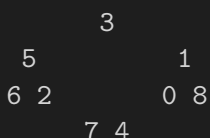
1644. Lowest Common Ancestor of a Binary Tree II

(M)

Código fuente: [src/02-arboles/1644-lca-ii.cpp](#)▷ Ver animación · [local](#) | [web](#)

Es la variante del problema 236 donde **p y/o q pueden NO existir** en el árbol. Si alguno de los dos falta, debemos devolver `nullptr` en vez de un ancestro cualquiera.

Intuición. En 236 podíamos cortar la búsqueda apenas encontrábamos p o q, porque el enunciado garantizaba que ambos vivían en el árbol: encontrar uno era suficiente para confiar en el resultado. Aquí esa garantía desaparece. Si cortáramos temprano y el otro nodo resultara no existir, devolveríamos un LCA falso. La solución: recorrer el árbol *completo*, sin atajos, y llevar un contador de cuántos de los dos nodos objetivo se encontraron de verdad. Sólo si el contador llega a 2 confiamos en el candidato a LCA; si es 0 o 1, alguno falta y respondemos `nullptr`.



Con este árbol: $LCA(5, 1) = 3$, $LCA(5, 4) = 5$, pero $LCA(5, 99) = \text{nullptr}$ porque 99 no está en el árbol, y $LCA(99, 88) = \text{nullptr}$ porque ninguno de los dos existe.

Solución. Usamos el mismo DFS post-order de 236 (`dfs` devuelve, para cada subárbol, el candidato a LCA), pero SIEMPRE evaluamos ambos hijos antes de revisar el nodo actual (sin retorno anticipado), y cada vez que el nodo actual es p o q incrementamos un contador pasado por referencia. Al final, el resultado sólo es válido si el contador es exactamente 2:

```

1 //
2 // =====
3 //
4 // 1644. Lowest Common Ancestor of a Binary Tree II (Medium)
5 //
6 // -----
7 //
8 // Dado un árbol binario y dos nodos p y q, encuentra su ancestro común más
9 // bajo (LCA). A diferencia del problema 236, aquí p y/o q PUEDEN NO EXISTIR
10 // en el árbol. Si alguno de los dos falta, debemos devolver nullptr.
11 //
12 // Ejemplo (mismo árbol que en 236):
13 //
14 //      3
15 //     / \
16 //    5   1
17 //   / \ / \
18 //  6  2 0  8
19 //   / \
20 //  7  4

```

```

21 //
22 // Diferencia clave con 236: en 236 podíamos cortar la búsqueda en cuanto
23 // encontrábamos p o q ("if root == p or root == q, devuélvelo YA"), porque
24 // el enunciado GARANTIZABA que ambos existían en el árbol. Aquí esa garantía
25 // no existe: si cortáramos temprano y resultara que el otro nodo no está en
26 // el árbol, devolveríamos un LCA falso. Por eso NO podemos confiar en la
27 // primera coincidencia; hay que recorrer el árbol COMPLETO y, además, llevar
28 // la cuenta de cuántos de los dos nodos {p, q} encontramos de verdad.
29 //
30 // Diagrama de la técnica (DFS post-order con contador por referencia):
31 //
32 //           nodo
33 //          /  \
34 //   dfs(izq)  dfs(der)    <- ambas llamadas SIEMPRE se hacen completas
35 //          \  /           (no hay retorno anticipado)
36 //         ?nodo es p o q?
37 //         sí -> contador++, este nodo es candidato a LCA
38 //         no -> si izq y der no son nulos, este nodo es el LCA;
39 //                si sólo uno no es nulo, se propaga hacia arriba
40 //
41 // Al final, sólo confiamos en el candidato a LCA si el contador llegó a 2
42 // (es decir, se confirmó que TANTO p COMO q existen en el árbol). Si el
43 // contador es 0 o 1, alguno de los dos nodos no existe: devolvemos nullptr.
44 //
45 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo exactamente una vez, sin
46 // atajos, porque necesitamos confirmar la existencia de ambos),
47 // espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
48 //
=====
49 #include <cassert>
50 #include <iostream>
51 using namespace std;
52
53 struct TreeNode {
54     int val;
55     TreeNode *left;
56     TreeNode *right;
57     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
58 };
59
60 class Solution {
61 public:
62     TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
63         int contador = 0;
64         TreeNode* candidato = dfs(root, p, q, contador);
65         return (contador == 2) ? candidato : nullptr; // sólo válido si ambos
66                                                         existen
67     }
68 private:
69     // Recorre TODO el subárbol (sin retorno anticipado) y va sumando en
70     // "contador" cada vez que encuentra a p o a q. Devuelve el candidato a
71     // LCA para este subárbol, igual que en 236.

```



```

72     TreeNode* dfs(TreeNode* nodo, TreeNode* p, TreeNode* q, int& contador) {
73         if (nodo == nullptr) return nullptr;
74
75         TreeNode* izq = dfs(nodo->left, p, q, contador);
76         TreeNode* der = dfs(nodo->right, p, q, contador);
77
78         if (nodo == p || nodo == q) {
79             ++contador;           // confirmamos que este nodo objetivo existe
80             return nodo;
81         }
82
83         if (izq != nullptr && der != nullptr) return nodo; // p y q en ramas
84             distintas
85         return (izq != nullptr) ? izq : der;                // propaga el que
86             exista
87     }
88 };
89
90 int main() {
91     Solution s;
92
93     // Árbol de prueba (el mismo que en 236):
94     //
95     //           3
96     //          / \
97     //         5   1
98     //        / \ / \
99     //       6  2 0  8
100      //      / \
101      //     7   4
102
103     TreeNode* n3 = new TreeNode(3);
104     TreeNode* n5 = new TreeNode(5);
105     TreeNode* n1 = new TreeNode(1);
106     TreeNode* n6 = new TreeNode(6);
107     TreeNode* n2 = new TreeNode(2);
108     TreeNode* n0 = new TreeNode(0);
109     TreeNode* n8 = new TreeNode(8);
110     TreeNode* n7 = new TreeNode(7);
111     TreeNode* n4 = new TreeNode(4);
112
113     n3->left = n5;    n3->right = n1;
114     n5->left = n6;    n5->right = n2;
115     n1->left = n0;    n1->right = n8;
116     n2->left = n7;    n2->right = n4;
117
118     // Caso 1: p y q existen y están en subárboles distintos de la raíz
119     // => LCA es la raíz (idéntico al caso feliz de 236).
120     TreeNode* r1 = s.lowestCommonAncestor(n3, n5, n1);
121     assert(r1 != nullptr && r1->val == 3);
122
123     // Caso 2: p y q existen, uno es ancestro directo del otro => LCA es 5.
124     TreeNode* r2 = s.lowestCommonAncestor(n3, n5, n4);
125     assert(r2 != nullptr && r2->val == 5);
126
127     // Caso 3: q NO existe en el árbol (nodo suelto, no enlazado a nada).

```

```

124 // Aunque p (n5) sí existe, como q falta debemos devolver nullptr.
125 TreeNode* suelto99 = new TreeNode(99);
126 TreeNode* r3 = s.lowestCommonAncestor(n3, n5, suelto99);
127 assert(r3 == nullptr);
128
129 // Caso 4: NINGUNO de los dos existe en el árbol => nullptr.
130 TreeNode* suelto88 = new TreeNode(88);
131 TreeNode* r4 = s.lowestCommonAncestor(n3, suelto99, suelto88);
132 assert(r4 == nullptr);
133
134 // Caso 5: dos hojas hermanas que sí existen => LCA es su padre directo
135 // (1).
136 TreeNode* r5 = s.lowestCommonAncestor(n3, n0, n8);
137 assert(r5 != nullptr && r5->val == 1);
138
139 delete n6; delete n7; delete n4; delete n2; delete n5;
140 delete n0; delete n8; delete n1; delete n3;
141 delete suelto99; delete suelto88;
142
143 cout << "1644 LCA II: todas las pruebas pasaron.\n";
144 return 0;
145 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

Notas de entrevista (Amazon). Deja claro en voz alta la diferencia con 236: aquí no hay garantía de existencia, así que el corte temprano (`if root == p or root == q: return root`) es incorrecto porque puede devolver un LCA falso cuando el otro nodo no está. La técnica del contador por referencia es la forma limpia de confirmar existencia sin un segundo recorrido; también es válida una alternativa de dos pasadas (una para verificar que p y q existen, otra para calcular el LCA como en 236), pero es menos elegante y hace doble trabajo. Casos borde: p o q ausentes, ambos ausentes, p es ancestro de q (o viceversa), y árbol de un solo nodo.

103. Binary Tree Zigzag Level Order Traversal

(M)

Código fuente: [src/02-arboles/103-zigzag-level-order.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dada la raíz de un árbol binario, queremos su recorrido por niveles, pero alternando la dirección: el nivel 0 se lee de izquierda a derecha, el nivel 1 de derecha a izquierda, el nivel 2 otra vez de izquierda a derecha, y así sucesivamente.

Intuición. Es el mismo BFS por niveles que usamos en 994 (Rotting Oranges): procesamos la cola capa por capa, y el tamaño de la cola al inicio de cada vuelta nos dice cuántos nodos hay en ese nivel. La única novedad es la dirección: en los niveles pares guardamos los valores en orden normal, y en los impares los guardamos en orden inverso. Para no pagar el costo de invertir un vector completo, colocamos cada valor directamente en su posición final dentro del nivel (índice i o $\text{capa}-1-i$ según la dirección).

3 9 20 15 7

Árbol `[3,9,20,null,null,15,7]`: el nivel 0 es `[3]` (izquierda a derecha), el nivel 1 se recorre de derecha a izquierda dando `[20, 9]`, y el nivel 2 vuelve a izquierda a derecha con `[15, 7]`. Resultado: `[[3], [20, 9], [15, 7]]`.

Solución. BFS estándar por niveles con una cola. Mantenemos una bandera booleana que alterna en cada vuelta del `while`; cuando está activa, insertamos cada valor en la posición espejo del nivel en vez de simplemente hacer `push_back`. Casos borde: árbol vacío devuelve `[]` de inmediato; un solo nodo devuelve `[[valor]]` sin alternar nada, porque sólo hay un nivel.

```

1 //
  =====
2 // 103. Binary Tree Zigzag Level Order Traversal (Medium)
3 //
  -----
4 // Dado la raíz de un árbol binario, devuelve el recorrido por niveles de sus
5 // valores en zigzag: el nivel 0 va de izquierda a derecha, el nivel 1 va de
6 // derecha a izquierda, el nivel 2 de nuevo izquierda a derecha, y así
7 // alternando.
8 //
9 // Ejemplo:
10 //      3
11 //     / \
12 //    9  20
13 //     / \
14 //    15  7
15 //
16 // nivel 0 (izq->der): [3]
17 // nivel 1 (der->izq): [20, 9]      <- se invierte el orden normal [9, 20]
18 // nivel 2 (izq->der): [15, 7]
19 //
20 // Resultado: [[3], [20, 9], [15, 7]]
21 //

```

```

22 // Diagrama del BFS por niveles (igual que 994, pero con struct.size() como
23 // tamaño de la capa):
24 //
25 //   cola: [3]
26 //   nivel 0 -> saco 3, encolo 9 y 20           -> capa = [3]
27 //   nivel 1 -> saco 9, 20, encolo 15 y 7       -> capa = [9, 20] (se invierte)
28 //   nivel 2 -> saco 15, 7, no hay hijos        -> capa = [15, 7]
29 //
30 // Idea: BFS estándar por niveles (igual que en 994-rotting-oranges). La única
31 // diferencia es que, en los niveles impares, invertimos el vector del nivel
32 // antes de agregarlo al resultado. Usamos un contador de nivel (o el tamaño
33 // del resultado) para saber si toca invertir.
34 //
35 // Complejidad: tiempo  $O(n)$  (cada nodo se visita una vez),
36 //               espacio  $O(n)$  por la cola y el resultado.
37 //
=====

38 #include <vector>
39 #include <queue>
40 #include <cassert>
41 #include <iostream>
42 #include <algorithm>
43 using namespace std;
44
45 struct TreeNode {
46     int val;
47     TreeNode* left;
48     TreeNode* right;
49     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
50 };
51
52 class Solution {
53 public:
54     vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
55         vector<vector<int>> resultado;
56         if (root == nullptr) return resultado;    // árbol vacío => []
57
58         queue<TreeNode*> cola;
59         cola.push(root);
60         bool derechaAIzquierda = false;    // nivel 0 va izquierda a derecha
61
62         while (!cola.empty()) {
63             int capa = cola.size();    // nodos en este nivel
64             vector<int> nivel(capa);
65             for (int i = 0; i < capa; ++i) {
66                 TreeNode* nodo = cola.front(); cola.pop();
67
68                 // Posición dentro del nivel: normal o invertida según direcci
69                 // ón.
70                 int idx = derechaAIzquierda ? (capa - 1 - i) : i;
71                 nivel[idx] = nodo->val;
72
73                 if (nodo->left != nullptr) cola.push(nodo->left);

```

```

73         if (nodo->right != nullptr) cola.push(nodo->right);
74     }
75     resultado.push_back(nivel);
76     derechaAIzquierda = !derechaAIzquierda;    // alterna para el
77         siguiente nivel
78     }
79     return resultado;
80 };
81
82 // Construye el árbol de ejemplo:
83 //      3
84 //    /  \
85 //   9   20
86 //  /  \
87 // 15   7
88
89 TreeNode* construirArbolEjemplo() {
90     TreeNode* raiz = new TreeNode(3);
91     raiz->left = new TreeNode(9);
92     raiz->right = new TreeNode(20);
93     raiz->right->left = new TreeNode(15);
94     raiz->right->right = new TreeNode(7);
95     return raiz;
96 }
97
98 int main() {
99     Solution s;
100
101     // Caso clásico: [3,9,20,null,null,15,7] => [[3],[20,9],[15,7]]
102     TreeNode* arbol1 = construirArbolEjemplo();
103     vector<vector<int>> esperado1 = {{3}, {20, 9}, {15, 7}};
104     assert(s.zigzagLevelOrder(arbol1) == esperado1);
105
106     // Árbol vacío => [].
107     assert(s.zigzagLevelOrder(nullptr).empty());
108
109     // Un solo nodo => [[x]].
110     TreeNode* unNodo = new TreeNode(1);
111     vector<vector<int>> esperadoUnNodo = {{1}};
112     assert(s.zigzagLevelOrder(unNodo) == esperadoUnNodo);
113
114     cout << "103 Zigzag Level Order: todas las pruebas pasaron.\n";
115     return 0;
116 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

Notas de entrevista (Amazon). El error típico es invertir el vector del nivel con una llamada aparte (`reverse`) en lugar de calcular la posición correcta al insertar; ambas son válidas en complejidad, pero calcular el índice directamente evita una pasada extra y demuestra que entiendes bien el layout de la capa. Vale la pena mencionar también la variante con dos pilas (una por dirección) como alternativa clásica al BFS con cola. Casos borde a mencionar: árbol vacío (`[]`), un solo nodo (`[[x]]`), y árboles muy desbalanceados donde un nivel tiene un solo nodo.

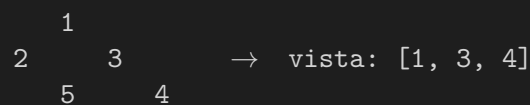
199. Binary Tree Right Side View

(M)

Código fuente: [src/02-arboles/199-right-side-view.cpp](#)▷ Ver animación · [local](#) | [web](#)

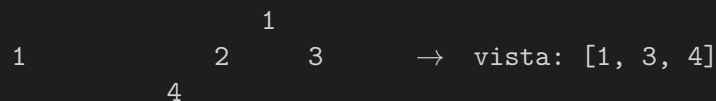
Dado un árbol binario, imagina que te paras del lado *derecho* del árbol y miras hacia él. Queremos los valores de los nodos que alcanzas a ver, de arriba hacia abajo (uno por cada nivel: el nodo más a la derecha de ese nivel).

Intuición. Si recorremos el árbol nivel por nivel (como en el BFS de 994-Rotting-Oranges) y en cada nivel leemos los nodos de izquierda a derecha, el nodo que se ve desde afuera es simplemente el *último* que leemos en ese nivel. No importa si ese nodo cuelga de la rama izquierda o derecha de su padre; lo único que importa es su posición dentro del nivel: el último en orden izquierda-a-derecha es, por definición, el más a la derecha, y por lo tanto el visible.



Nótese el nivel 2: el nodo 5 (que cuelga de 2) queda oculto detrás del nodo 4 (que cuelga de 3), simplemente porque 4 está más a la derecha dentro de ese nivel.

Caso importante. Un nivel puede tener su nodo visible colgando de una rama *izquierda*, si la rama derecha de ese nivel ya se quedó sin hijos (es más corta):



Aquí 4 cuelga de la izquierda de 2, pero en el nivel 2 no existe ningún nodo del lado derecho: 4 es el único nodo de ese nivel y, por lo tanto, el último leído de izquierda a derecha. Sí es visible desde la derecha aunque provenga de una rama izquierda.

Solución. BFS por niveles: en cada nivel guardamos el valor del último nodo procesado (el de más a la derecha). También es posible resolverlo con DFS visitando primero la rama derecha y luego la izquierda, registrando el primer nodo encontrado en cada profundidad (el primero en ese orden de visita es el más a la derecha; si ya hay un valor registrado para esa profundidad, el nodo actual queda oculto detrás de uno visitado antes).

```

1 //
  =====
2 // 199. Binary Tree Right Side View (Medium)
3 //
  -----
4 // Dado un árbol binario, imagina que te paras del lado DERECHO del árbol y
5 // miras hacia él. Devuelve, de arriba hacia abajo, los valores de los nodos
6 // que alcanzas a ver (uno por cada nivel).
7 //
8 // Ejemplo clásico:
  
```

```

9  //
10 //      1
11 //     / \
12 //    2   3
13 //     \   \
14 //      5   4
15 //  Vista derecha: [1, 3, 4]
16 //  Nivel 0: solo 1      -> visible: 1
17 //  Nivel 1: 2 y 3, gana 3 -> visible: 3 (el más a la derecha)
18 //  Nivel 2: solo 5      -> visible: 4 (!ojo! el 5 queda oculto
19 //                          detrás del 4, que está en
20 //                          el mismo nivel pero cuelga
21 //                          de la rama derecha de 3)
22 //
23 // Caso con nivel donde el nodo visible viene de la rama IZQUIERDA porque la
24 // rama derecha de ese nivel es más corta (termina antes):
25 //
26 //      1
27 //     / \
28 //    2   3
29 //   /
30 //  4
31 //  Nivel 0: [1]          -> visible: 1
32 //  Nivel 1: [2, 3]       -> visible: 3
33 //  Nivel 2: [4]          -> visible: 4 (4 cuelga de la IZQUIERDA de 2; en
34 //                          este nivel no hay ningún nodo del
35 //                          lado derecho, así que el último nodo
36 //                          leído de izquierda a derecha SÍ es
37 //                          visible desde la derecha)
38 //  Vista derecha: [1, 3, 4]
39 //
40 // Idea: BFS por niveles (como en 994-rotting-oranges). En cada nivel
41 // recorremos todos sus nodos de izquierda a derecha y nos quedamos con el
42 // ÚLTIMO: ese es, por definición, el más a la derecha del nivel y por lo
43 // tanto el que se ve desde afuera. No importa de qué rama cuelgue (izquierda
44 // o derecha del padre); lo único que importa es su posición dentro del
45 // nivel.
46 //
47 // Alternativa equivalente: DFS visitando primero la rama derecha y luego la
48 // izquierda, registrando el primer nodo que se visita en cada profundidad
49 // (si ya hay un valor registrado para esa profundidad, el nodo actual queda
50 // oculto detrás de uno visitado antes, que por el orden de visita está más a
51 // la derecha).
52 //
53 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
54 // espacio  $O(n)$  en el peor caso (la cola puede llegar a
55 // contener hasta el último nivel completo, que en un árbol
56 // balanceado es  $\sim n/2$  nodos; el resultado también usa  $O(n)$ ).
57 //
=====
58 #include <vector>
59 #include <queue>
60 #include <cassert>

```

```

61 #include <iostream>
62 using namespace std;
63
64 struct TreeNode {
65     int val;
66     TreeNode *left;
67     TreeNode *right;
68     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
69 };
70
71 class Solution {
72 public:
73     vector<int> rightSideView(TreeNode* root) {
74         vector<int> vista;
75         if (root == nullptr) return vista;    // árbol vacío => nada que ver
76
77         queue<TreeNode*> cola;
78         cola.push(root);
79
80         while (!cola.empty()) {
81             int nivel = cola.size();    // cuántos nodos hay en este nivel
82             for (int i = 0; i < nivel; ++i) {
83                 TreeNode* actual = cola.front(); cola.pop();
84
85                 if (i == nivel - 1) vista.push_back(actual->val);    // último
                                // del nivel
86
87                 if (actual->left != nullptr) cola.push(actual->left);
88                 if (actual->right != nullptr) cola.push(actual->right);
89             }
90         }
91         return vista;
92     }
93 };
94
95 int main() {
96     Solution s;
97
98     // Caso 1 (clásico): [1,2,3,null,5,null,4] => [1,3,4]
99     //
100     //      1
101     //     / \
102     //    2   3
103     //   \   \
104     //    5   4
105     TreeNode* n1 = new TreeNode(1);
106     TreeNode* n2 = new TreeNode(2);
107     TreeNode* n3 = new TreeNode(3);
108     TreeNode* n5 = new TreeNode(5);
109     TreeNode* n4 = new TreeNode(4);
110     n1->left = n2;  n1->right = n3;
111     n2->right = n5;
112     n3->right = n4;
113
114     vector<int> r1 = s.rightSideView(n1);

```



```

114     assert((r1 == vector<int>{1, 3, 4}));
115
116     delete n5; delete n4; delete n2; delete n3; delete n1;
117
118     // Caso 2: el nodo visible del último nivel cuelga de la rama IZQUIERDA
119     // porque la rama derecha de ese nivel ya terminó (es más corta).
120     //           1
121     //        /  \
122     //       2    3
123     //      /
124     //     4
125     TreeNode* m1 = new TreeNode(1);
126     TreeNode* m2 = new TreeNode(2);
127     TreeNode* m3 = new TreeNode(3);
128     TreeNode* m4 = new TreeNode(4);
129     m1->left = m2;  m1->right = m3;
130     m2->left = m4;
131
132     vector<int> r2 = s.rightSideView(m1);
133     assert((r2 == vector<int>{1, 3, 4}));
134
135     delete m4; delete m2; delete m3; delete m1;
136
137     // Caso 3: árbol vacío => vista vacía.
138     vector<int> r3 = s.rightSideView(nullptr);
139     assert(r3.empty());
140
141     // Caso 4: un solo nodo => se ve únicamente él.
142     TreeNode* solo = new TreeNode(42);
143     vector<int> r4 = s.rightSideView(solo);
144     assert((r4 == vector<int>{42}));
145     delete solo;
146
147     cout << "199 Right Side View: todas las pruebas pasaron.\n";
148     return 0;
149 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

Notas de entrevista (Amazon). El error típico es asumir que el nodo visible de cada nivel siempre cuelga de una rama derecha; como muestra el segundo diagrama, puede colgar de una izquierda si esa rama derecha terminó antes. Aclarar esto en la entrevista demuestra que entendiste el problema y no memorizaste una solución superficial. Casos borde a mencionar: árbol vacío (vista vacía), árbol de un solo nodo (la vista es ese único valor), y árboles muy desbalanceados donde toda una rama es más profunda que la otra.

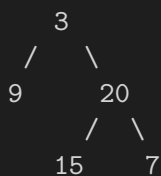
987. Vertical Order Traversal of a Binary Tree

(H)

Código fuente: [src/02-arboles/987-vertical-order.cpp](#)▷ Ver animación · [local](#) | [web](#)

A cada nodo del árbol le asignamos una *columna* y una *fila*: la raíz vive en columna 0, fila 0; el hijo izquierdo se mueve una columna a la izquierda y una fila hacia abajo; el hijo derecho, una columna a la derecha y una fila hacia abajo. Debemos devolver los valores agrupados de columna menor a mayor; dentro de cada columna, ordenados por fila ascendente; y si dos nodos caen exactamente en la misma columna y la misma fila, se desempatan por **valor ascendente**. Este último detalle es justo lo que distingue a este problema del más sencillo #314 y lo que lo hace Hard: no basta con recorrer el árbol, hay que ordenar con un criterio compuesto de tres niveles.

Intuición. Si en vez de pensar en "izquierda/derecha" pensamos en coordenadas, cada nodo termina siendo una terna (columna, fila, valor). Una vez que tenemos todas las ternas, el problema se reduce a *ordenarlas*: por columna, luego por fila, luego por valor. El orden natural de una tupla ya hace exactamente eso. No hace falta ninguna estructura de datos sofisticada: un DFS que acumula ternas y un `sort` al final resuelven el problema completo.



columna	fila	valor
-1	1	9
0	0	3
0	2	15
1	1	20
2	2	7

Al agrupar por columna (ya ordenado) obtenemos `[[9],[3,15],[20],[7]]`: exactamente el resultado esperado.

Solución. Un DFS (o BFS, es indistinto) recorre el árbol y por cada nodo guarda la terna (columna, fila, valor) en un vector. El hijo izquierdo se visita con `columna-1, fila+1`; el derecho con `columna+1, fila+1`. Al terminar, ordenamos el vector de ternas: como `tuple<int,int,int>` ya compara componente a componente, un solo `sort` deja todo en el orden pedido, incluido el desempate por valor. Por último recorreremos el vector ordenado agrupando los valores consecutivos que comparten columna.

El caso donde el desempate por valor realmente importa: en el árbol completo `[1,2,3,4,5,6,7]`, el nodo 5 (hijo derecho de 2) y el nodo 6 (hijo izquierdo de 3) caen ambos en columna 0, fila 2. Sin el desempate por valor el orden entre ellos sería ambiguo; con él, 5 siempre va antes que 6.

```

1 //
  =====
2 // 987. Vertical Order Traversal of a Binary Tree (Hard)

```

```

3 // -----
4 // A cada nodo le asignamos una columna y una fila: la raíz está en columna 0,
5 // fila 0; el hijo izquierdo va una columna a la izquierda (columna-1) y una
6 // fila más abajo; el hijo derecho, una columna a la derecha (columna+1) y una
7 // fila más abajo. Debemos devolver los valores agrupados de columna menor a
8 // mayor; dentro de cada columna, ordenados por fila ascendente; y si dos
9 // nodos caen en la MISMA columna y MISMA fila, se desempata por valor
10 // ascendente (esto es justo lo que distingue a este problema del más
11 // sencillo 314, y lo que lo hace Hard).
12 //
13 // Ejemplo:
14 //           3                      columna -1: [9]
15 //          / \                    columna 0: [3, 15]
16 //         9   20                  columna 1: [20]
17 //          / \                    columna 2: [7]
18 //        15   7
19 //
20 // (3 en c0,f0) (9 en c-1,f1) (20 en c1,f1) (15 en c0,f2) (7 en c2,f2)
21 //
22 // Diagrama del recorrido DFS acumulando (columna, fila, valor):
23 //
24 //     dfs(nodo, col, fila)
25 //         |-- guarda (col, fila, nodo->val)
26 //         |-- dfs(nodo->left, col-1, fila+1)
27 //         |-- dfs(nodo->right, col+1, fila+1)
28 //
29 // Idea: recorreremos el árbol completo (DFS o BFS, da igual) guardando una
30 // terna (columna, fila, valor) por cada nodo. Al final ordenamos todas las
31 // ternas: primero por columna, luego por fila, luego por valor (el orden
32 // natural de una tupla ya hace exactamente esto). Después agrupamos los
33 // valores consecutivos que comparten columna.
34 //
35 // Complejidad: tiempo  $O(n \log n)$  por el ordenamiento de las  $n$  ternas,
36 // espacio  $O(n)$  para guardarlas.
37 // =====
38 #include <vector>
39 #include <tuple>
40 #include <algorithm>
41 #include <functional>
42 #include <climits>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 struct TreeNode {
48     int val;
49     TreeNode *left;
50     TreeNode *right;
51     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
52 };

```

```

53
54 class Solution {
55 public:
56     vector<vector<int>> verticalTraversal(TreeNode* root) {
57         vector<tuple<int,int,int>> nodos;    // (columna, fila, valor)
58
59         function<void(TreeNode*, int, int)> dfs = [&](TreeNode* nodo, int col,
60             int fila) {
61             if (!nodo) return;
62             nodos.emplace_back(col, fila, nodo->val);
63             dfs(nodo->left, col - 1, fila + 1);
64             dfs(nodo->right, col + 1, fila + 1);
65         };
66         dfs(root, 0, 0);
67
68         // El orden natural de tuple<int,int,int> ya compara componente a
69         // componente: columna, luego fila, luego valor. Justo lo que pide
70         // el problema (incluido el desempate por valor).
71         sort(nodos.begin(), nodos.end());
72
73         vector<vector<int>> resultado;
74         int colActual = INT_MIN;
75         bool primera = true;
76         for (auto& [col, fila, val] : nodos) {
77             if (primera || col != colActual) {
78                 resultado.push_back({});
79                 colActual = col;
80                 primera = false;
81             }
82             resultado.back().push_back(val);
83         }
84         return resultado;
85     };
86
87 int main() {
88     Solution s;
89
90     // Caso clásico: [3,9,20,null,null,15,7] => [[9],[3,15],[20],[7]]
91     TreeNode* raiz1 = new TreeNode(3);
92     raiz1->left = new TreeNode(9);
93     raiz1->right = new TreeNode(20);
94     raiz1->right->left = new TreeNode(15);
95     raiz1->right->right = new TreeNode(7);
96     vector<vector<int>> esperado1 = {{9}, {3, 15}, {20}, {7}};
97     assert(s.verticalTraversal(raiz1) == esperado1);
98
99     // Árbol completo [1,2,3,4,5,6,7]: los nodos 5 y 6 comparten columna 0 y
100     // fila 2 (5 = hijo derecho de 2, que está en columna -1; 6 = hijo
101     // izquierdo de 3, que está en columna 1; ambos terminan en columna 0).
102     // Se desempatan por valor ascendente: 5 antes que 6.
103     // Esperado: [[4],[2],[1,5,6],[3],[7]]
104     TreeNode* raiz2 = new TreeNode(1);
105     raiz2->left = new TreeNode(2);

```

```

106     raiz2->right = new TreeNode(3);
107     raiz2->left->left = new TreeNode(4);
108     raiz2->left->right = new TreeNode(5);
109     raiz2->right->left = new TreeNode(6);
110     raiz2->right->right = new TreeNode(7);
111     vector<vector<int>> esperado2 = {{4}, {2}, {1, 5, 6}, {3}, {7}};
112     assert(s.verticalTraversal(raiz2) == esperado2);
113
114     // Un solo nodo => [[x]]
115     TreeNode* raiz3 = new TreeNode(42);
116     vector<vector<int>> esperado3 = {{42}};
117     assert(s.verticalTraversal(raiz3) == esperado3);
118
119     cout << "987 Vertical Order: todas las pruebas pasaron.\n";
120     return 0;
121 }

```

Complejidad: tiempo $O(n \log n)$, espacio $O(n)$.

Notas de entrevista (Amazon). El error típico es asumir que un recorrido en orden (BFS por niveles o DFS in-order) ya entrega el resultado final: no es así, porque nodos de columnas distintas pueden intercalarse en el mismo nivel del recorrido. La forma robusta es separar completamente las dos fases: primero *recolectar* toda la información posicional de cada nodo (columna, fila, valor) sin preocuparse por el orden, y después *ordenar* con el criterio compuesto correcto. Vale la pena mencionar en la entrevista que este problema es la versión "difícil" de LeetCode 314 (Binary Tree Vertical Order Traversal): la diferencia entera está en el desempate por valor cuando coinciden columna y fila, algo fácil de pasar por alto si se resuelve de memoria.

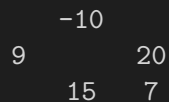
124. Binary Tree Maximum Path Sum

(H)

Código fuente: [src/02-arboles/124-max-path-sum.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan un árbol binario cuyos valores pueden ser negativos. Un *camino* es cualquier secuencia de nodos donde cada par consecutivo está conectado por una arista, sin repetir ningún nodo. El camino **no** necesita pasar por la raíz ni terminar en una hoja: puede empezar y terminar en cualquier nodo. Queremos la suma máxima de valores que puede lograr algún camino.

Intuición. Párate en cada nodo y pregúntate dos cosas distintas. Primero: "si quiero seguir subiendo este camino hacia mi padre, ¿cuánto es lo más que puedo aportar bajando por una sola de mis dos ramas?" (subir por las dos ramas a la vez no tiene sentido: el camino se bifurcaría). Segundo: "si mi camino ya no va a subir más y va a *doblar* justo aquí, usando mis dos ramas a la vez, ¿cuánto sumaría?". La primera pregunta es lo que le reportas a tu padre; la segunda solo sirve para actualizar, en silencio, la mejor respuesta que hemos visto en todo el árbol.



En este árbol el camino óptimo es $15 \rightarrow 20 \rightarrow 7 = 42$: conviene ignorar por completo la raíz -10 y la rama de 9, porque sumarlas solo empeoraría el resultado.

Solución. Hacemos DFS recursivo tipo post-order con una función auxiliar `gananciaMaxima(nodo)` que devuelve, para el subárbol de ese nodo, la ganancia máxima que aporta a un camino que sigue subiendo hacia el padre (una sola rama):

```

gananciaIzq = max(0, gananciaMaxima(nodo->left))
gananciaDer = max(0, gananciaMaxima(nodo->right))
devuelve = nodo->val + max(gananciaIzq, gananciaDer)

```

El `max(0, ...)` descarta una rama si su ganancia es negativa: es mejor no extender el camino por ahí, porque sumar un número negativo solo lo empeora.

Es crucial no confundir lo que la recursión *devuelve* (una sola rama, porque un camino que sube hacia el padre solo puede continuar por un lado) con lo que actualiza la *respuesta global* (las dos ramas a la vez, para el camino que dobla justo en ese nodo y ya no puede crecer más hacia arriba):

```

candidato = nodo->val + gananciaIzq + gananciaDer
mejorGlobal = max(mejorGlobal, candidato)

```

Recorremos en post-order (primero los hijos, luego el nodo actual) para que cada nodo ya conozca la ganancia de sus dos subárboles antes de calcular su propio candidato. El código completo (abre el archivo con clic en el título):

```

1 //
  =====
2 // 124. Binary Tree Maximum Path Sum (Hard)
3 //
  -----

```



```

    )
58 //
59 //   - Lo que se DEVUELVE al padre: val + max(gananciaIzq, gananciaDer)
60 //     (una sola rama, para que el padre pueda seguir extendiendo el camino).
61 //   - Lo que actualiza la RESPUESTA GLOBAL: val + gananciaIzq + gananciaDer
62 //     (las dos ramas juntas, el camino "dobla" aquí y ya no puede crecer más
63 //     hacia arriba).
64 //
65 // Recorremos el árbol en post-order (primero hijos, luego nodo actual) para
66 // que cada nodo ya conozca la ganancia de sus dos subárboles antes de
67 // calcular su propio candidato.
68 //
69 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
70 //               espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
71 //
=====

72 #include <climits>
73 #include <cassert>
74 #include <iostream>
75 using namespace std;
76
77 struct TreeNode {
78     int val;
79     TreeNode *left;
80     TreeNode *right;
81     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
82 };
83
84 class Solution {
85 public:
86     int maxPathSum(TreeNode* root) {
87         mejorGlobal = INT_MIN;
88         gananciaMaxima(root);
89         return mejorGlobal;
90     }
91
92 private:
93     int mejorGlobal;
94
95     // Devuelve la ganancia máxima que aporta este subárbol a un camino que
96     // sigue subiendo hacia el padre (una sola rama). De paso, actualiza
97     // mejorGlobal con el mejor camino que "dobla" en este nodo (dos ramas).
98     int gananciaMaxima(TreeNode* nodo) {
99         if (nodo == nullptr) return 0;
100
101         // Ganancia de cada rama; si es negativa, mejor no tomarla (0).
102         int gananciaIzq = max(0, gananciaMaxima(nodo->left));
103         int gananciaDer = max(0, gananciaMaxima(nodo->right));
104
105         // Camino que dobla en este nodo: usa las dos ramas a la vez.
106         // Solo sirve para la respuesta global, nunca se propaga al padre.
107         int candidato = nodo->val + gananciaIzq + gananciaDer;
108         mejorGlobal = max(mejorGlobal, candidato);

```



```

109         // Lo que sí se propaga al padre: este nodo más, a lo más, UNA rama.
110         return nodo->val + max(gananciaIzq, gananciaDer);
111     }
112 };
113
114
115 int main() {
116     Solution s;
117
118     // Caso 1: árbol pequeño sin valores negativos.
119     //      1
120     //    / \
121     //   2  3
122     // Camino óptimo: 2 -> 1 -> 3 = 6.
123     TreeNode* n1 = new TreeNode(1);
124     TreeNode* n2 = new TreeNode(2);
125     TreeNode* n3 = new TreeNode(3);
126     n1->left = n2; n1->right = n3;
127     assert(s.maxPathSum(n1) == 6);
128     delete n2; delete n3; delete n1;
129
130     // Caso 2: la raíz es negativa y conviene descartarla junto con la rama
131     // de 9; el camino óptimo vive por completo en el subárbol derecho.
132     //      -10
133     //    /   \
134     //   9    20
135     //  /  \
136     // 15   7
137     // Camino óptimo: 15 -> 20 -> 7 = 42.
138     TreeNode* r = new TreeNode(-10);
139     TreeNode* rIzq = new TreeNode(9);
140     TreeNode* rDer = new TreeNode(20);
141     TreeNode* rDerIzq = new TreeNode(15);
142     TreeNode* rDerDer = new TreeNode(7);
143     r->left = rIzq; r->right = rDer;
144     rDer->left = rDerIzq; rDer->right = rDerDer;
145     assert(s.maxPathSum(r) == 42);
146     delete rIzq; delete rDerIzq; delete rDerDer; delete rDer; delete r;
147
148     // Caso 3: un solo nodo negativo. No hay otra opción de camino más que
149     // el nodo solo, así que la respuesta debe poder ser negativa.
150     TreeNode* neg = new TreeNode(-3);
151     assert(s.maxPathSum(neg) == -3);
152     delete neg;
153
154     // Caso 4: árbol con una rama completamente negativa que conviene
155     // descartar (max(0, ...) la corta) en vez de arrastrarla al camino.
156     //      5
157     //    / \
158     //   -8  9
159     //  / \
160     // -2 -3
161     // El mejor camino ignora por completo el subárbol de -8: la respuesta
162     // es 5 -> 9 = 14, no 5 + (-8) + 9 ni nada que incluya -2 o -3.

```

```

163     TreeNode* raiz2 = new TreeNode(5);
164     TreeNode* izq = new TreeNode(-8);
165     TreeNode* der = new TreeNode(9);
166     TreeNode* izqIzq = new TreeNode(-2);
167     TreeNode* izqDer = new TreeNode(-3);
168     raiz2->left = izq; raiz2->right = der;
169     izq->left = izqIzq; izq->right = izqDer;
170     assert(s.maxPathSum(raiz2) == 14);
171     delete izqIzq; delete izqDer; delete izq; delete der; delete raiz2;
172
173     // Caso 5: todos los valores negativos; la mejor respuesta es el nodo
174     // individual menos negativo (un camino de un solo nodo).
175     //      -3
176     //      / \
177     //     -2 -1
178     TreeNode* n = new TreeNode(-3);
179     TreeNode* nIzq = new TreeNode(-2);
180     TreeNode* nDer = new TreeNode(-1);
181     n->left = nIzq; n->right = nDer;
182     assert(s.maxPathSum(n) == -1);
183     delete nIzq; delete nDer; delete n;
184
185     cout << "124 Max Path Sum: todas las pruebas pasaron.\n";
186     return 0;
187 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

Notas de entrevista (Amazon). El error más común es devolver `val + gananciaIzq + gananciaDer` hacia el padre en vez de `val + max(gananciaIzq, gananciaDer)`: eso rompería la invariante de que un camino no se bifurca. Otro punto clave: la respuesta debe poder ser negativa (por ejemplo un árbol de un solo nodo con valor negativo), así que `mejorGlobal` debe inicializarse en `INT_MIN` y nunca forzarse a 0. Casos borde: nodo único, todos los valores negativos, y una rama negativa que conviene podar con `max(0, ...)` en vez de arrastrarla al camino final.

543. Diameter of Binary Tree

(Fácil)

Código fuente: [src/02-arboles/543-diameter-of-binary-tree.cpp](#)▷ Ver animación · [local](#) | [web](#)**Complejidad:** tiempo $O(N)$, espacio $O(H)$.

En este problema, se nos pide encontrar la longitud del camino más largo entre dos nodos de un árbol binario. Es un error común pensar que este camino debe pasar forzosamente por la raíz; podría darse el caso de que un subárbol muy profundo tenga su propio camino que no interactúe con la raíz.

La observación fundamental es notar que cualquier camino entre dos nodos puede visualizarse subiendo de un nodo hasta un ancestro común y luego bajando hacia el otro. Visto desde la perspectiva de ese ancestro, la longitud total del camino será la altura de su rama izquierda más la altura de su rama derecha.

Entonces, la solución consiste en hacer un recorrido *post-order* (profundidad primero). Para cada nodo calculamos recursivamente la altura de sus dos ramas. En ese momento, calculamos la suma de ambas alturas; este es el diámetro si el camino “doblará” en este nodo. Llevamos una variable global que mantiene el registro del diámetro máximo encontrado en todo el árbol. Al mismo tiempo, hacia el padre de la recursión, le devolvemos $1 + \max(\text{alturaIzq}, \text{alturaDer})$, informando así el camino más largo posible desde la hoja hasta el padre en una sola dirección.

```

1 //
2 // =====
3 //
4 // 543. Diameter of Binary Tree (Easy)
5 //
6 // -----
7 // Dado la raíz de un árbol binario, devuelve la longitud del diámetro del
8 // árbol.
9 //
10 // El diámetro de un árbol binario es la longitud del camino más largo entre
11 // DOS NODOS CUALESQUIERA del árbol. Este camino puede o no pasar por la raíz.
12 //
13 // La longitud del camino entre dos nodos se representa por el número de
14 // aristas (edges) entre ellos, no el número de nodos.
15 //
16 // Ejemplo 1:
17 //      1
18 //     / \
19 //    2   3
20 //   / \
21 //  4   5
22 // Diámetro óptimo: 4 -> 2 -> 1 -> 3 o 5 -> 2 -> 1 -> 3 (3 aristas).
23 //
24 // Ejemplo 2:
25 //      1
26 //     /
27 //    2
28 // Diámetro óptimo: 1 -> 2 (1 arista).
29 //
30 // Idea clave:
31 // El camino más largo en el árbol puede o no pasar por la raíz.

```

```

29 // Sin embargo, si analizamos cualquier nodo como si fuera el punto más
30 // alto (o el "doblez") de un camino, la longitud del camino que pasa por él
31 // es exactamente la altura de su subárbol izquierdo más la altura de su
32 // subárbol derecho.
33 //
34 // Por lo tanto, en cada nodo, calculamos la "altura" (o profundidad) de sus
35 // hijos.
36 // - Lo que devolvemos hacia arriba (al padre) es 1 + max(alturaIzq,
    alturaDer).
37 // - Lo que actualiza nuestra respuesta global (el máximo diámetro visto
    hasta
38 //     ahora) es alturaIzq + alturaDer.
39 //
40 // Al recorrer el árbol en post-order, garantizamos que al procesar un nodo,
41 // ya tenemos las alturas calculadas de sus subárboles.
42 //
43 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
44 //                 espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
45 //

```

```

46 #include <algorithm>
47 #include <cassert>
48 #include <iostream>
49 using namespace std;
50
51 struct TreeNode {
52     int val;
53     TreeNode *left;
54     TreeNode *right;
55     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
56 };
57
58 class Solution {
59 public:
60     int diameterOfBinaryTree(TreeNode* root) {
61         maxDiametro = 0;
62         altura(root);
63         return maxDiametro;
64     }
65
66 private:
67     int maxDiametro;
68
69     int altura(TreeNode* nodo) {
70         if (nodo == nullptr) return 0;
71
72         int alturaIzq = altura(nodo->left);
73         int alturaDer = altura(nodo->right);
74
75         // Camino que "dobla" en este nodo: suma de las aristas a izquierda y
            derecha
76         maxDiametro = max(maxDiametro, alturaIzq + alturaDer);
77

```

```

78         // Se propaga la altura máxima al padre + 1 (por la arista que conecta
79         // a este nodo)
80         return 1 + max(alturaIzq, alturaDer);
81     }
82 };
83 int main() {
84     Solution s;
85
86     // Caso 1: árbol básico.
87     //      1
88     //     /\
89     //    2 3
90     //   /\
91     //  4 5
92     // Diámetro: 3 (4->2->1->3 o 5->2->1->3).
93     TreeNode* r1 = new TreeNode(1);
94     r1->left = new TreeNode(2);
95     r1->right = new TreeNode(3);
96     r1->left->left = new TreeNode(4);
97     r1->left->right = new TreeNode(5);
98     assert(s.diameterOfBinaryTree(r1) == 3);
99
100    delete r1->left->left;
101    delete r1->left->right;
102    delete r1->left;
103    delete r1->right;
104    delete r1;
105
106    // Caso 2: árbol muy pequeño.
107    //      1
108    //     /
109    //    2
110    // Diámetro: 1 (1->2).
111    TreeNode* r2 = new TreeNode(1);
112    r2->left = new TreeNode(2);
113    assert(s.diameterOfBinaryTree(r2) == 1);
114
115    delete r2->left;
116    delete r2;
117
118    // Caso 3: árbol que dobla en un hijo y no pasa por la raíz.
119    //      1
120    //     /
121    //    2
122    //   /\
123    //  3 4
124    // /\  \
125    // 5   6
126    // El diámetro más largo es entre 5 y 6: 5->3->2->4->6 (4 aristas).
127    // Pasa por 2, no por 1.
128    TreeNode* r3 = new TreeNode(1);
129    r3->left = new TreeNode(2);
130    r3->left->left = new TreeNode(3);

```

```
131     r3->left->right = new TreeNode(4);
132     r3->left->left->left = new TreeNode(5);
133     r3->left->right->right = new TreeNode(6);
134     assert(s.diameterOfBinaryTree(r3) == 4);
135
136     delete r3->left->left->left;
137     delete r3->left->right->right;
138     delete r3->left->left;
139     delete r3->left->right;
140     delete r3->left;
141     delete r3;
142
143     cout << "543 Diameter of Binary Tree: todas las pruebas pasaron.\n";
144     return 0;
145 }
```

863. All Nodes Distance K in Binary Tree

(Media)

Código fuente: [src/02-arboles/863-all-nodes-distance-k-in-binary-tree.cpp](#)▷ Ver animación · [local](#) | [web](#)**Complejidad:** tiempo $O(N)$, espacio $O(N)$.

En los árboles binarios estándar, los nodos solo tienen referencias a sus hijos. Esto significa que podemos viajar hacia abajo fácilmente, pero no podemos “subir” hacia los ancestros. Sin embargo, cuando se nos pide encontrar todos los nodos a una distancia K de un nodo objetivo, la respuesta podría estar tanto en los subárboles de ese nodo como en otras partes del árbol a las que solo se puede llegar pasando por sus ancestros.

Para resolver esto, podemos dividir el problema en dos fases. Primero, transformamos el árbol en un grafo no dirigido. Hacemos un recorrido inicial (por ejemplo, DFS) para construir un diccionario (mapa hash) que asocie cada nodo con su padre. Con esto, cada nodo ahora conoce a sus tres vecinos potenciales: hijo izquierdo, hijo derecho y padre.

En la segunda fase, el problema se reduce a encontrar los nodos a distancia K en un grafo. Para ello, utilizamos una Búsqueda en Anchura (BFS). Comenzamos desde el nodo objetivo, insertándolo en una cola y marcándolo como visitado (usando un conjunto o *set*). Expandimos nuestra búsqueda nivel por nivel: en cada paso, desencolamos un nodo y encolamos a sus hijos y a su padre, siempre que no hayan sido visitados antes. Cuando alcanzamos el nivel K de nuestra BFS, todos los nodos que se encuentren actualmente en la cola son exactamente los que están a distancia K del objetivo.

```

1 //
2 // =====
3 //
4 // 863. All Nodes Distance K in Binary Tree (Medium)
5 //
6 // -----
7 //
8 // Dado la raíz de un árbol binario, un nodo objetivo (target) y un entero K,
9 // devuelve los valores de todos los nodos que están a una distancia K del
10 // nodo objetivo.
11 //
12 // Ejemplo:
13 //
14 //      3
15 //     /\
16 //    5  1
17 //   /\ /\
18 //  6 2 0 8
19 //   /\
20 //  7  4
21 // target = 5, K = 2
22 // Salida: [7, 4, 1]
23 // Explicación: Los nodos a distancia 2 de 5 son 7 y 4 (hacia abajo) y 1
24 // (hacia arriba, pasando por la raíz 3).
25 //
26 // Idea clave:
27 // En un árbol binario normal, solo podemos movernos hacia abajo (hijos).
28 // Sin embargo, para encontrar todos los nodos a distancia K, necesitamos
29 // expandirnos en todas direcciones: hacia los hijos y hacia los ancestros.
30 //
31 // La solución tiene dos pasos:

```

```

27 // 1. Recorrer el árbol (DFS o BFS) para construir un mapa que asocie cada
28 //    nodo con su padre: unordered_map<TreeNode*, TreeNode*> padres. Esto
29 //    convierte el árbol en un grafo no dirigido.
30 // 2. Hacer una búsqueda en anchura (BFS) comenzando desde el nodo 'target'.
31 //    Llevamos un conjunto (set) de nodos visitados para no retroceder. En
32 //    cada paso de la BFS, nos expandimos a: hijo izquierdo, hijo derecho y
33 //    padre. Cuando el nivel de la BFS sea exactamente K, los nodos en la
34 //    cola son nuestra respuesta.
35 //
36 // Complejidad: tiempo  $O(N)$  para construir el mapa y hacer la BFS, espacio
37 //                $O(N)$  para el mapa, el conjunto de visitados y la cola.
38 //
=====
39 #include <vector>
40 #include <unordered_map>
41 #include <unordered_set>
42 #include <queue>
43 #include <algorithm>
44 #include <cassert>
45 #include <iostream>
46 using namespace std;
47
48 struct TreeNode {
49     int val;
50     TreeNode *left;
51     TreeNode *right;
52     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
53 };
54
55 class Solution {
56 public:
57     vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
58         // Paso 1: Mapear padres
59         unordered_map<TreeNode*, TreeNode*> padre;
60         mapearPadres(root, nullptr, padre);
61
62         // Paso 2: BFS desde el target
63         queue<TreeNode*> q;
64         unordered_set<TreeNode*> visitados;
65
66         q.push(target);
67         visitados.insert(target);
68         int distanciaActual = 0;
69
70         while (!q.empty()) {
71             if (distanciaActual == k) {
72                 vector<int> resultado;
73                 while (!q.empty()) {
74                     resultado.push_back(q.front()->val);
75                     q.pop();
76                 }
77                 return resultado;
78             }

```



```

79
80     int tamanoNivel = q.size();
81     for (int i = 0; i < tamanoNivel; ++i) {
82         TreeNode* actual = q.front();
83         q.pop();
84
85         // Intentar hijo izquierdo
86         if (actual->left && visitados.find(actual->left) == visitados.
            end()) {
87             visitados.insert(actual->left);
88             q.push(actual->left);
89         }
90         // Intentar hijo derecho
91         if (actual->right && visitados.find(actual->right) ==
            visitados.end()) {
92             visitados.insert(actual->right);
93             q.push(actual->right);
94         }
95         // Intentar padre
96         if (padre[actual] && visitados.find(padre[actual]) ==
            visitados.end()) {
97             visitados.insert(padre[actual]);
98             q.push(padre[actual]);
99         }
100     }
101     distanciaActual++;
102 }
103
104     return {};
105 }
106
107 private:
108     void mapearPadres(TreeNode* nodo, TreeNode* p, unordered_map<TreeNode*,
        TreeNode*>& padre) {
109         if (!nodo) return;
110         padre[nodo] = p;
111         mapearPadres(nodo->left, nodo, padre);
112         mapearPadres(nodo->right, nodo, padre);
113     }
114 };
115
116 int main() {
117     Solution s;
118
119     // Caso 1
120     //      3
121     //    / \
122     //   5  1
123     //  / \ / \
124     // 6  2 0 8
125     //   / \
126     //  7  4
127     TreeNode* r1 = new TreeNode(3);
128     TreeNode* n5 = new TreeNode(5);

```

```
129     TreeNode* n1 = new TreeNode(1);
130     r1->left = n5; r1->right = n1;
131     n5->left = new TreeNode(6);
132     n5->right = new TreeNode(2);
133     n5->right->left = new TreeNode(7);
134     n5->right->right = new TreeNode(4);
135     n1->left = new TreeNode(0);
136     n1->right = new TreeNode(8);
137
138     vector<int> res1 = s.distanceK(r1, n5, 2);
139     sort(res1.begin(), res1.end());
140     assert((res1 == vector<int>{1, 4, 7}));
141
142     delete n5->right->left;
143     delete n5->right->right;
144     delete n5->right;
145     delete n5->left;
146     delete n1->left;
147     delete n1->right;
148     delete n1;
149     delete n5;
150     delete r1;
151
152     // Caso 2: target es raíz
153     TreeNode* r2 = new TreeNode(1);
154     r2->left = new TreeNode(2);
155     r2->right = new TreeNode(3);
156
157     vector<int> res2 = s.distanceK(r2, r2, 1);
158     sort(res2.begin(), res2.end());
159     assert((res2 == vector<int>{2, 3}));
160
161     delete r2->left;
162     delete r2->right;
163     delete r2;
164
165     cout << "863 All Nodes Distance K: todas las pruebas pasaron.\n";
166     return 0;
167 }
```

98. Validate Binary Search Tree

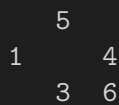
(M)

Código fuente: [src/02-arboles/98-validate-binary-search-tree.cpp](#)

▷ Ver animación · [local](#) | [web](#)

Dada la raíz de un árbol binario, determina si es un árbol de búsqueda binaria (BST) válido. Para que un BST sea válido, el subárbol izquierdo de cada nodo debe contener solo nodos con claves **menores** que la clave del nodo, el subárbol derecho debe contener solo nodos con claves **mayores**, y ambos subárboles deben ser también BST válidos.

Intuición. El error más común al resolver este problema es verificar solamente que el hijo izquierdo sea menor y el hijo derecho sea mayor que el nodo actual. Esto *no* es suficiente. Un nodo que esté muy abajo en el subárbol derecho de la raíz podría tener un valor menor que la raíz, violando así la propiedad fundamental del BST (donde **todo** el subárbol derecho debe ser mayor que la raíz).



En este ejemplo, $3 < 4$ y $6 > 4$, por lo que el nodo 4 y sus hijos parecen correctos localmente. Sin embargo, el nodo 3 está en el subárbol derecho de la raíz 5, pero no es mayor que 5. ¡El árbol es inválido!

Solución. En lugar de verificar solo relaciones padre-hijo, debemos mantener un *rango válido* (mínimo, máximo) a medida que bajamos por el árbol en nuestra recursión. Todo nodo que visitemos debe caer estrictamente dentro de ese rango.

- Al principio, la raíz puede tener cualquier valor: (LONG_MIN, LONG_MAX).
- Cuando bajamos al hijo izquierdo, el límite **superior** del rango se actualiza al valor del nodo actual.
- Cuando bajamos al hijo derecho, el límite **inferior** del rango se actualiza al valor del nodo actual.

Usamos el tipo `long` de C++ para los límites mínimo y máximo porque los valores de los nodos pueden ser `INT_MIN` o `INT_MAX`, y si usáramos `int`, fallarían las comparaciones en esos casos extremos (overflow/underflow). El código:

```

1 //
  =====
2 // 98. Validate Binary Search Tree (Medium)
3 //
  -----
4 // Dada la raíz de un árbol binario, determina si es un árbol de búsqueda
   binaria
5 // (BST) válido.
6 //
7 // Un BST válido se define como sigue:
8 // - El subárbol izquierdo de un nodo contiene solo nodos con claves MENORES
   que
  
```

```

9  // la clave del nodo.
10 // - El subárbol derecho de un nodo contiene solo nodos con claves MAYORES que
11 // la clave del nodo.
12 // - Ambos subárboles (izquierdo y derecho) deben ser también BST válidos.
13 //
14 // Ejemplo 1:
15 //      2
16 //     /\
17 //    1  3
18 // Es un BST válido. Todo a la izquierda de 2 es < 2, todo a la derecha es >
19 // 2.
20 // Ejemplo 2:
21 //      5
22 //     /\
23 //    1  4
24 //     /\
25 //    3  6
26 // NO es válido. Aunque en el nodo 4, 3 < 4 y 6 > 4, el nodo 3 está en el
27 // subárbol derecho de 5 y 3 NO es mayor que 5.
28 //
29 // Idea clave: un error común es comparar solo el nodo actual con sus dos
30 // hijos inmediatos. Esto falla porque un nodo muy abajo en el subárbol
31 // derecho
32 // podría violar la condición de ser mayor que la raíz original.
33 // Solución correcta: recursión bajando un RANGO PERMITIDO (min, max).
34 // - Al ir a la izquierda, el valor máximo permitido se actualiza al valor
35 //   actual.
36 // - Al ir a la derecha, el valor mínimo permitido se actualiza al valor
37 //   actual.
38 // (Usamos el tipo long long para evitar problemas de overflow con INT_MAX/
39 // INT_MIN).
40 //
41 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
42 // espacio  $O(h)$  por la pila de recursión ( $h$  = altura).
43 //
44 =====
45
46 #include <climits>
47 #include <cassert>
48 #include <iostream>
49 using namespace std;
50
51 struct TreeNode {
52     int val;
53     TreeNode *left;
54     TreeNode *right;
55     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
56 };
57
58 class Solution {
59 public:
60     bool isValidBST(TreeNode* root) {

```

```

56     return validar(root, LLONG_MIN, LLONG_MAX);
57 }
58
59 private:
60     bool validar(TreeNode* nodo, long long minimo, long long maximo) {
61         if (nodo == nullptr) return true;
62
63         // Si el valor actual está fuera del rango permitido, no es un BST.
64         if (nodo->val <= minimo || nodo->val >= maximo) {
65             return false;
66         }
67
68         // El subárbol izquierdo debe tener valores menores que nodo->val.
69         // El subárbol derecho debe tener valores mayores que nodo->val.
70         return validar(nodo->left, minimo, nodo->val) &&
71             validar(nodo->right, nodo->val, maximo);
72     }
73 };
74
75 int main() {
76     Solution s;
77
78     // Caso 1: Árbol válido simple.
79     //      2
80     //    / \
81     //   1  3
82     TreeNode* n1 = new TreeNode(2);
83     n1->left = new TreeNode(1);
84     n1->right = new TreeNode(3);
85     assert(s.isValidBST(n1) == true);
86     delete n1->left; delete n1->right; delete n1;
87
88     // Caso 2: Árbol inválido por nodo profundo.
89     //      5
90     //    / \
91     //   1  4
92     //    / \
93     //   3  6
94     TreeNode* n2 = new TreeNode(5);
95     n2->left = new TreeNode(1);
96     n2->right = new TreeNode(4);
97     n2->right->left = new TreeNode(3);
98     n2->right->right = new TreeNode(6);
99     assert(s.isValidBST(n2) == false);
100    delete n2->right->left; delete n2->right->right;
101    delete n2->left; delete n2->right; delete n2;
102
103    // Caso 3: Solo un nodo (siempre válido).
104    TreeNode* n3 = new TreeNode(10);
105    assert(s.isValidBST(n3) == true);
106    delete n3;
107
108    // Caso 4: Valores en el límite de los enteros de 32 bits (INT_MAX).
109    TreeNode* n4 = new TreeNode(INT_MAX);

```

```
110     assert(s.isValidBST(n4) == true);  
111     delete n4;  
112  
113     cout << "98 Validate BST: todas las pruebas pasaron.\n";  
114     return 0;  
115 }
```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

Notas de entrevista (Amazon). Cuidado especial con los casos borde como los límites de los enteros de 32 bits (`INT_MAX` y `INT_MIN`). Es preferible usar `long` o parámetros de puntero nulo para representar el infinito. Además, asegúrate de mencionar que el espacio auxiliar de la recursión es $O(h)$ donde h es la altura del árbol, lo cual se convierte en $O(n)$ en el peor caso (árbol degenerado como una lista ligada) y $O(\log n)$ en el mejor caso (árbol balanceado).

105. Construct Binary Tree from Preorder and Inorder Traversal (M)

Código fuente: [src/02-arboles/105-construct-binary-tree-from-preorder-and-inorder-traversal.cpp](#)

▷ Ver animación · [local](#) | [web](#)

Dados dos arreglos de enteros `preorder` e `inorder`, donde el primero es el recorrido en preorden de un árbol binario y el segundo es su recorrido en inorden, reconstruye y devuelve el árbol binario original. (Se garantiza que no hay valores duplicados en el árbol).

Intuición. Analicemos la naturaleza de los dos recorridos:

- **Preorden (Preorder):** El formato es [RAÍZ, [Elementos del Subárbol Izquierdo], [Elementos del Subárbol Derecho]].
- **Inorden (Inorder):** El formato es [[Elementos del Subárbol Izquierdo], RAÍZ, [Elementos del Subárbol Derecho]].

El arreglo `preorder` siempre nos da un regalo inestimable: su primer elemento es **garantizadamente** la raíz del árbol actual. Una vez que sabemos quién es la raíz, podemos buscar ese valor en el arreglo `inorder`. Todo lo que quede a la izquierda de ese valor en `inorder` pertenecerá al subárbol izquierdo, y todo lo que quede a la derecha será del subárbol derecho.

Por ejemplo, si la raíz es 3 y la encontramos en el índice i de `inorder`, sabemos que hay exactamente i elementos en el subárbol izquierdo. Esto nos indica que los siguientes i elementos en `preorder` forman el subárbol izquierdo, y el resto forman el derecho.

Solución. Podemos aplicar recursión, "consumiendo" elementos de `preorder` uno por uno como las raíces de los subárboles que vamos construyendo. Para que la búsqueda de la raíz en `inorder` no tome $O(n)$ en cada paso (lo que haría la solución $O(n^2)$), creamos un mapa hash al inicio que nos dice el índice de cualquier valor en $O(1)$.

```

1 //
  =====
2 // 105. Construct Binary Tree from Preorder and Inorder Traversal (Medium)
3 //
  -----
4 // Dados dos arreglos de enteros preorder e inorder donde preorder es el
5 // recorrido en preorden de un árbol binario e inorder es el recorrido en
6 // inorden del mismo árbol, construye y devuelve el árbol binario.
7 //
8 // Ejemplo 1:
9 //      3
10 //    / \
11 //   9  20
12 //  / \
13 // 15  7
14 // Entrada: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
15 // Salida: [3,9,20,null,null,15,7] (representación en niveles)
16 //
17 // Idea clave:
18 // 1. El primer elemento de 'preorder' siempre es la RAÍZ del árbol actual.
```

```

19 // 2. Buscamos el valor de esa raíz en el arreglo 'inorder'.
20 // 3. Todo lo que está a la izquierda de la raíz en 'inorder' pertenece al
21 //    subárbol izquierdo. Todo lo que está a la derecha pertenece al subárbol
22 //    derecho.
23 // 4. Con el tamaño del subárbol izquierdo (digamos, k elementos), sabemos que
24 //    los siguientes k elementos en 'preorder' (después de la raíz) son
25 //    exactamente el preorden del subárbol izquierdo.
26 //
27 // Para que la búsqueda en 'inorder' sea rápida, usamos un mapa hash (O(1)) en
28 // lugar de hacer escaneo lineal (O(n)).
29 //
30 // Complejidad: tiempo O(n) (hash map + recursión lineal), espacio O(n)
31 //               (mapa hash y pila de recursión).
32 //
=====

33 #include <vector>
34 #include <unordered_map>
35 #include <cassert>
36 #include <iostream>
37 using namespace std;
38
39 struct TreeNode {
40     int val;
41     TreeNode *left;
42     TreeNode *right;
43     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
44 };
45
46 class Solution {
47 public:
48     TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
49         // Mapa para encontrar el índice de cualquier valor en inorder en O(1)
50         unordered_map<int, int> inorderIndex;
51         for (int i = 0; i < inorder.size(); i++) {
52             inorderIndex[inorder[i]] = i;
53         }
54
55         // Puntero global para avanzar en el arreglo preorder
56         int preIndex = 0;
57         return construir(preorder, inorderIndex, preIndex, 0, inorder.size() -
58             1);
59
60 private:
61     TreeNode* construir(const vector<int>& preorder,
62                         unordered_map<int, int>& inorderIndex,
63                         int& preIndex, int inStart, int inEnd) {
64
65         // Caso base: no hay elementos para este subárbol
66         if (inStart > inEnd) {
67             return nullptr;
68         }
69

```



```

70      // El elemento actual en preorden es la raíz de este subárbol
71      int rootVal = preorder[preIndex++];
72      TreeNode* root = new TreeNode(rootVal);
73
74      // Encontramos dónde está esta raíz en el arreglo inorden
75      int mid = inorderIndex[rootVal];
76
77      // Construimos recursivamente los subárboles izquierdo y derecho.
78      // Importante: primero construimos el izquierdo porque los siguientes
79      // elementos en preorder pertenecen al subárbol izquierdo.
80      root->left = construir(preorder, inorderIndex, preIndex, inStart, mid
81                          - 1);
82      root->right = construir(preorder, inorderIndex, preIndex, mid + 1,
83                          inEnd);
84
85      return root;
86  };
87
88  // Función auxiliar para liberar la memoria del árbol
89  void liberarArbol(TreeNode* root) {
90      if (!root) return;
91      liberarArbol(root->left);
92      liberarArbol(root->right);
93      delete root;
94  }
95
96  int main() {
97      Solution s;
98
99      // Caso 1:
100      //      3
101      //     /\
102      //    9 20
103      //   /\  \
104      //  15 7
105      vector<int> pre1 = {3, 9, 20, 15, 7};
106      vector<int> in1 = {9, 3, 15, 20, 7};
107      TreeNode* root1 = s.buildTree(pre1, in1);
108
109      assert(root1->val == 3);
110      assert(root1->left->val == 9);
111      assert(root1->right->val == 20);
112      assert(root1->left->left == nullptr);
113      assert(root1->left->right == nullptr);
114      assert(root1->right->left->val == 15);
115      assert(root1->right->right->val == 7);
116      liberarArbol(root1);
117
118      // Caso 2: Un solo nodo
119      vector<int> pre2 = {-1};
120      vector<int> in2 = {-1};
121      TreeNode* root2 = s.buildTree(pre2, in2);
122      assert(root2->val == -1);

```

```
122     assert(root2->left == nullptr && root2->right == nullptr);
123     liberarArbol(root2);
124
125     cout << "105 Construct Binary Tree: todas las pruebas pasaron.\n";
126     return 0;
127 }
```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

Notas de entrevista (Amazon). Este es un problema clásico de "divide y vencerás". El detalle técnico más importante es pasar el índice de **preorder** por **referencia** (`int& preIndex`) para que el avance secuencial de la raíz se mantenga sincronizado a través de todas las llamadas recursivas. Si lo pasas por valor, los subárboles derechos procesarán las raíces equivocadas. La memoria extra $O(n)$ viene del mapa hash y de la pila recursiva.

337. House Robber III

(M)

Código fuente: [src/02-arboles/337-house-robber-iii.cpp](#)▷ Ver animación · [local](#) | [web](#)

El ladrón ha descubierto un nuevo vecindario para robar. Las casas en este vecindario forman un árbol binario. La regla de oro (y única forma de alertar a la policía) es que **no puedes robar dos casas que estén conectadas directamente**. Es decir, si robas un nodo, no puedes robar a sus hijos inmediatos.

Debes determinar la cantidad máxima de dinero que el ladrón puede llevarse sin alertar a la policía.

Intuición. Si estás parado en cualquier casa (nodo) tomando una decisión, solo tienes dos opciones: o la robas, o no la robas.

- **Opción 1: Robas la casa actual.** Si haces esto, la alarma de la policía limita tus siguientes movimientos: tienes *estrictamente prohibido* robar las casas de los hijos directos. Así que la ganancia de esta rama sería el valor de la casa actual, más el dinero máximo que podrías obtener de los subárboles de los hijos *asumiendo que no robaste esos hijos*.
- **Opción 2: No robas la casa actual.** Al mantenerte alejado de esta casa, la policía no sospecha. Ahora eres completamente *libre* de hacer lo que quieras con las casas de los hijos: puedes decidir robarlas o no robarlas. Obviamente, elegirás la opción que te dé más dinero de cada hijo de forma independiente.

El problema de calcular estas dos opciones de forma independiente para cada nodo con recursión simple es que terminarías calculando los mismos subárboles una y otra vez, con una complejidad de $O(2^n)$.

Solución. Para evitar el recálculo (y sin necesidad de usar un mapa hash de memoización), podemos hacer que nuestra función recursiva (DFS) devuelva **ambas** respuestas a la vez para cada subárbol. En C++, podemos devolver un `std::pair<int, int>` con la estructura: `{max_SIN_robear, max_ROBANDO}`.

De esta forma, visitamos cada nodo del árbol exactamente una vez en post-orden (visitamos a los hijos primero para tener sus resultados, y luego tomamos la decisión para el padre).

```

1 //
2 // =====
3 //
4 // 337. House Robber III (Medium)
5 //
6 // -----
7 //
8 // El ladrón ha descubierto un nuevo vecindario para robar. Hay una sola
9 // entrada a este lugar, llamada "raíz". Además de la raíz, cada casa tiene
10 // exactamente una casa matriz (un padre). Después de un recorrido, el ladrón
11 // se da cuenta de que todas las casas en este lugar forman un árbol binario.
12 //
13 // Automáticamente se contactará a la policía si se roban dos casas unidas
14 // directamente en la misma noche (es decir, no puedes robar un nodo y a
15 // su padre).
16 //

```

```

13 // Determina la cantidad máxima de dinero que el ladrón puede robar esta noche
14 // sin alertar a la policía.
15 //
16 // Ejemplo 1:
17 //      3
18 //     /\
19 //    2 3
20 //     /\
21 //    3 1
22 // Robo óptimo = 3 (raíz) + 3 (hijo de 2) + 1 (hijo de 3) = 7.
23 //
24 // Ejemplo 2:
25 //      3
26 //     /\
27 //    4 5
28 //   /\  \
29 //  1 3  1
30 // Robo óptimo = 4 (hijo izq) + 5 (hijo der) = 9.
31 //
32 // Idea clave:
33 // Para cada nodo, tenemos dos opciones:
34 // 1. Robar el nodo actual. Si lo hacemos, NO podemos robar sus hijos,
35 //    así que sumamos el valor del nodo más lo que sacaríamos de los "nietos"
36 //    (es decir, robando los subárboles de los hijos, pero eligiendo no
37 //    robar los hijos inmediatos).
38 // 2. NO robar el nodo actual. Si hacemos esto, somos libres de decidir si
39 //    robar o no robar cada uno de los hijos (tomando el máximo entre ambas
40 //    opciones para cada hijo).
41 //
42 // Para evitar recalcular estados (que daría  $O(2^n)$ ), usamos DFS en post-order
43 // , devolviendo para cada nodo un par: [max_sin_robar_raiz, max_robando_raiz].
44 //
45 // Complejidad: tiempo  $O(n)$ , espacio  $O(h)$  para la pila de recursión.
46 //
47 =====
48
49 #include <algorithm>
50 #include <cassert>
51 #include <iostream>
52 using namespace std;
53
54 struct TreeNode {
55     int val;
56     TreeNode *left;
57     TreeNode *right;
58     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
59 };
60
61 class Solution {
62 public:
63     int rob(TreeNode* root) {
64         pair<int, int> resultado = robarDFS(root);
65         return max(resultado.first, resultado.second);
66     }
67
68 private:
69     pair<int, int> robarDFS(TreeNode* node) {
70         if (!node) return {0, 0};
71         pair<int, int> left = robarDFS(node->left);
72         pair<int, int> right = robarDFS(node->right);
73         int rob = node->val + max(left.second, right.second);
74         int noRob = max(left.first, left.second) + max(right.first, right.second);
75         return {rob, noRob};
76     }
77 };

```

```

64     }
65
66 private:
67     // Devuelve un par {maximo_SIN_robear_el_nodo, maximo_ROBANDO_el_nodo}
68     pair<int, int> robarDFS(TreeNode* nodo) {
69         if (!nodo) {
70             return {0, 0};
71         }
72
73         pair<int, int> izq = robarDFS(nodo->left);
74         pair<int, int> der = robarDFS(nodo->right);
75
76         // Si decidimos NO robar el nodo actual, somos libres de elegir
77         // la mejor opción (robar o no robar) para cada uno de sus hijos.
78         int sinRobar = max(izq.first, izq.second) + max(der.first, der.second)
79         ;
80
81         // Si decidimos ROBAR el nodo actual, obligatoriamente NO podemos
82         // a sus hijos directos.
83         int robando = nodo->val + izq.first + der.first;
84
85         return {sinRobar, robando};
86     };
87
88 int main() {
89     Solution s;
90
91     // Caso 1:
92     //      3
93     //    / \
94     //   2  3
95     //    \  \
96     //     3  1
97     TreeNode* r1 = new TreeNode(3);
98     r1->left = new TreeNode(2);
99     r1->right = new TreeNode(3);
100    r1->left->right = new TreeNode(3);
101    r1->right->right = new TreeNode(1);
102    assert(s.rob(r1) == 7);
103
104    // Caso 2:
105    //      3
106    //    / \
107    //   4  5
108    //  / \  \
109    // 1  3  1
110    TreeNode* r2 = new TreeNode(3);
111    r2->left = new TreeNode(4);
112    r2->right = new TreeNode(5);
113    r2->left->left = new TreeNode(1);
114    r2->left->right = new TreeNode(3);
115    r2->right->right = new TreeNode(1);

```

```
116     assert(s.rob(r2) == 9);
117
118     cout << "337 House Robber III: todas las pruebas pasaron.\n";
119
120     // Cleanup manual (solo para no ensuciar la salida, en CP a veces se omite
121     // pero aquí somos limpios).
122     delete r1->left->right; delete r1->right->right;
123     delete r1->left; delete r1->right; delete r1;
124
125     delete r2->left->left; delete r2->left->right; delete r2->right->right;
126     delete r2->left; delete r2->right; delete r2;
127
128     return 0;
129 }
```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

Notas de entrevista (Amazon). Muchos candidatos se lanzan inmediatamente a escribir una recursión simple con memoización usando un `unordered_map<TreeNode*, int>`. Aunque funciona, la solución óptima es devolver los dos estados en un par (o arreglo pequeño). Mencionar la evolución de una solución ingenua ($O(2^n)$) a una memoizada y, finalmente, a esta de estado compuesto ($O(n)$ de tiempo y sin la sobrecarga de un mapa hash) da una excelente impresión en la entrevista de Amazon.

297. Serialize and Deserialize Binary Tree

(D)

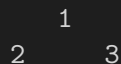
Código fuente: [src/02-arboles/297-serialize-and-deserialize-binary-tree.cpp](#)

► Ver animación · [local](#) | [web](#)

El proceso de *serializar* consiste en transformar una estructura de datos (como un árbol) en un formato lineal (como una cadena de texto) que pueda ser enviado por la red o guardado en disco. El proceso inverso, *deserializar*, toma esa cadena y reconstruye la estructura original en memoria de forma exacta. Diseña el algoritmo para lograr esto con un árbol binario.

Intuición. Existen varias formas válidas de recorrer un árbol y plasmarlo en texto (BFS por niveles como lo hace LeetCode internamente, o DFS). El enfoque más elegante y rápido de programar es el recorrido DFS en **preorden** (Raíz, Izquierda, Derecha).

Si recorremos el árbol así, y guardamos un carácter especial (digamos "N") cada vez que nos topemos con un puntero nulo, la cadena resultante tendrá toda la información necesaria para reconstruirse a sí misma sin ambigüedades.



Usando preorden y comas como separadores, el árbol de arriba genera la cadena: "1,2,N,3,N,N,".

Solución. Para la serialización, usamos una recursión simple en preorden, concatenando el valor del nodo y las llamadas recursivas.

Para la deserialización, el truco está en *consumir* la cadena en el mismo orden (preorden). El primer valor que leamos será obligatoriamente la raíz. Después de crear la raíz, llamamos a la recursión para que consuma de la cadena todo lo que pertenezca al subárbol izquierdo, y luego lo que pertenezca al subárbol derecho. Usar un `stringstream` de C++ (pasado por referencia) hace que "consumir tokens separados por comas" sea extremadamente sencillo y elegante.

```

1 //
2 // =====
3 //
4 // 297. Serialize and Deserialize Binary Tree (Hard)
5 //
6 // -----
7 //
8 // La serialización es el proceso de convertir una estructura de datos o un
9 // objeto en una secuencia de bits para que pueda almacenarse en un archivo o
10 // búfer de memoria, o transmitirse a través de una conexión de red para
11 // reconstruirse posteriormente en el mismo entorno o en otro entorno.
12 //
13 // Diseña un algoritmo para serializar y deserializar un árbol binario. No hay
14 // restricciones sobre cómo tu algoritmo de serialización/deserialización debe
15 // funcionar. Solo necesitas asegurarte de que un árbol binario se pueda
16 // serializar en una cadena y esta cadena se pueda deserializar en la
17 // estructura de árbol original.
18 //
19 // Ejemplo 1:
20 //      1
21 //     /\
22 //    2 3
  
```

```

19 //      / \
20 //     4   5
21 // Entrada: root = [1,2,3,null,null,4,5]
22 // Salida: [1,2,3,null,null,4,5]
23 //
24 // Idea clave:
25 // Usaremos un recorrido en Preorden (Preorder: Raíz, Izquierda, Derecha).
26 // - Serializar: Recorremos el árbol y agregamos el valor del nodo seguido de
27 //   una coma (e.g., "1,"). Si encontramos un puntero nulo, agregamos un
28 //   marcador especial como "N,".
29 // - Deserializar: Leemos los tokens separados por coma uno por uno. Como
30 //   guardamos el árbol en preorden, el primer token es garantizadamente la
31 //   raíz. Luego construimos recursivamente el subárbol izquierdo y luego el
32 //   derecho usando el estado compartido (por ejemplo, pasando un stringstream
33 //   por referencia).
34 //
35 // Complejidad: tiempo  $O(n)$ , espacio  $O(n)$  tanto para serializar como para
36 //               deserializar (por la cadena generada y la pila de recursión).
37 //
=====

38 #include <string>
39 #include <sstream>
40 #include <cassert>
41 #include <iostream>
42 using namespace std;
43
44 struct TreeNode {
45     int val;
46     TreeNode *left;
47     TreeNode *right;
48     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
49 };
50
51 class Codec {
52 public:
53     // Codifica un árbol a una cadena de texto.
54     string serialize(TreeNode* root) {
55         if (!root) return "N,";
56
57         string res = to_string(root->val) + ",";
58         res += serialize(root->left);
59         res += serialize(root->right);
60
61         return res;
62     }
63
64     // Decodifica tu cadena de texto a un árbol.
65     TreeNode* deserialize(string data) {
66         stringstream ss(data);
67         return decodificar(ss);
68     }
69
70 private:

```



```

71     TreeNode* decodificar(stringstream& ss) {
72         string token;
73         // Leemos el siguiente token hasta la coma
74         if (!getline(ss, token, ',')) {
75             return nullptr;
76         }
77
78         // Si el token es "N", es un puntero nulo
79         if (token == "N") {
80             return nullptr;
81         }
82
83         // De lo contrario, es un número. Lo convertimos y creamos el nodo.
84         TreeNode* nodo = new TreeNode(stoi(token));
85
86         // Como serializamos en preorden, lo siguiente pertenece a la
87             izquierda
88         // y luego a la derecha.
89         nodo->left = decodificar(ss);
90         nodo->right = decodificar(ss);
91
92         return nodo;
93     };
94
95     // Función auxiliar para verificar si dos árboles son idénticos
96     bool sonIden(TreeNode* r1, TreeNode* r2) {
97         if (!r1 && !r2) return true;
98         if (!r1 || !r2) return false;
99         return (r1->val == r2->val) &&
100             sonIden(r1->left, r2->left) &&
101             sonIden(r1->right, r2->right);
102     }
103
104     // Función auxiliar para liberar memoria
105     void liberarArbol(TreeNode* root) {
106         if (!root) return;
107         liberarArbol(root->left);
108         liberarArbol(root->right);
109         delete root;
110     }
111
112     int main() {
113         Codec codec;
114
115         // Caso 1: Árbol normal
116         //      1
117         //     /\
118         //    2 3
119         //   /\
120         //  4 5
121         TreeNode* root1 = new TreeNode(1);
122         root1->left = new TreeNode(2);
123         root1->right = new TreeNode(3);

```

```
124     root1->right->left = new TreeNode(4);
125     root1->right->right = new TreeNode(5);
126
127     string ser1 = codec.serialize(root1);
128     TreeNode* ans1 = codec.deserialize(ser1);
129     assert(sonIden(root1, ans1));
130     liberarArbol(root1);
131     liberarArbol(ans1);
132
133     // Caso 2: Árbol vacío
134     TreeNode* root2 = nullptr;
135     string ser2 = codec.serialize(root2);
136     TreeNode* ans2 = codec.deserialize(ser2);
137     assert(ans2 == nullptr);
138
139     cout << "297 Serialize/Deserialize BST: todas las pruebas pasaron.\n";
140     return 0;
141 }
```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

Notas de entrevista (Amazon). Al ser un problema "Hard", los entrevistadores buscan ver tu dominio de las herramientas del lenguaje. Usar `std::stringstream` con `getline` en C++ demuestra madurez. Otro punto a favor: menciona por qué el recorrido BFS (por niveles) también es válido pero suele requerir más código (colas y manejo de nulos por nivel), mientras que DFS preorden brilla por la brevedad de su implementación recursiva. Evita usar variables globales enteras para el índice del string; en su lugar, pasa el estado (el stream o un iterador) por referencia.

Capítulo 3

Ruta de problemas

Lista objetivo por bloque. Los marcados con ✓ ya tienen capítulo; el resto son las siguientes iteraciones.

Bloque 1 — Grafos. ✓ 200 Number of Islands • ✓ 994 Rotting Oranges • ✓ 695 Max Area of Island • ✓ 542 01 Matrix • ✓ 1091 Shortest Path in Binary Matrix • ✓ 417 Pacific Atlantic • ✓ 79 Word Search • ✓ 207 Course Schedule • ✓ 210 Course Schedule II • ✓ 269 Alien Dictionary • ✓ 127 Word Ladder • ✓ 126 Word Ladder II • ✓ 261 Graph Valid Tree • ✓ 323 Connected Components • ✓ 128 Longest Consecutive • ✓ 547 Number of Provinces • ✓ 743 Network Delay Time • ✓ 787 Cheapest Flights K Stops • ✓ 133 Clone Graph • ✓ 1192 Critical Connections.

Bloque 2 — Árboles. ✓ 236 LCA • ✓ 1644 LCA II • ✓ 103 Zigzag Level Order • ✓ 199 Right Side View • ✓ 987 Vertical Order • ✓ 124 Max Path Sum • ✓ 543 Diameter • ✓ 863 Distance K • ✓ 98 Validate BST • ✓ 105 Build Tree Pre+In • ✓ 337 House Robber III • ✓ 297 Serialize/Deserialize.

Bloque 3 — Intervalos. 56 Merge Intervals • 253 Meeting Rooms II • 2402 Meeting Rooms III • 1094 Car Pooling.

Bloque 4 — Heaps / Top-K. 215 Kth Largest • 347 Top K Frequent • 973 K Closest Points • 295 Find Median • 23 Merge k Sorted Lists • 621 Task Scheduler.

Bloque 5 — Design. 146 LRU Cache • 348 Design Tic-Tac-Toe • 642 Search Autocomplete.

Bloque 6 — Sliding Window / Stack. 3 Longest Substring • 76 Minimum Window • 42 Trapping Rain Water • 84 Largest Rectangle • 394 Decode String.

Bloque 7 — DP. 322 Coin Change • 139 Word Break • 55 Jump Game • 5 Longest Palindromic Substring.